

Macro Tree Transducers

JOOST ENGELFRIET* AND HEIKO VOGLER*[†]

*Twente University of Technology, Department of Computer Science,
P.O. Box 217, 7500 AE Enschede, The Netherlands*

Received October 29, 1982; accepted April 23, 1985

Macro tree transducers are a combination of top-down tree transducers and macro grammars. They serve as a model for syntax-directed semantics in which context information can be handled. In this paper the formal model of macro tree transducers is studied by investigating typical automata theoretical topics like composition, decomposition, domains, and ranges of the induced translation classes. The extension with regular look-ahead is considered. © 1985 Academic Press, Inc.

1. INTRODUCTION

Macro tree transducers were introduced in [15, 8] combining the features of top-down tree transducers [34, 36, 12, 4] and macro grammars [23, 20]. In fact, macro tree transducers are the generalization of top-down tree-to-string transducers to trees according to the generalization method described in [13, 9, 15]. Conceptually, they serve as a model for syntax-directed translations in which context information can be handled. Hence, it is reasonable to consider macro tree transducers as a formalization of certain aspects of denotational semantics and attribute grammars. From the program schematic point of view macro tree transducers are systems of recursive procedures with parameters: the first ("syntactic") parameter is of type T_{Σ} (i.e., tree over alphabet Σ), representing a derivation of an underlying context-free grammar, and all other ("context") parameters are of type $T_{\mathcal{A}}$, representing elements of a semantic domain. In [8] these systems are called primitive recursive schemes with parameters.

To make the connections of macro tree transducers to the above mentioned concepts more transparent we first consider, informally, the transducer model. A macro tree transducer consists of ranked alphabets Q , Σ , and $\mathcal{A}$ of states, input, and output symbols, respectively, an initial state q^{in} of rank 1, and a finite number of rules R of the form

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \quad (*)$$

* Current address: University of Leiden, Institute of Applied Mathematics and Computer Science, P.O. Box 9512, 2300 RA Leiden, The Netherlands.

[†] Supported by the "Deutscher Akademischer Austauschdienst" under Grant 311/018/509/2.

where q is a state of rank $n+1$ with $n \geq 0$, σ is an input symbol of rank m with $m \geq 0$, and $x_1, \dots, x_m$ and $y_1, \dots, y_n$ are called (input) variables and (formal) parameters, respectively. The right-hand side t is a tree built up with output symbols, the parameters $y_1, \dots, y_n$ (of rank 0) and constructs like $q'(x_i, t_1, \dots, t_r)$, where q' is a state of rank $r+1$, x_i is a variable from $\{x_1, \dots, x_m\}$, and $t_1, \dots, t_r$ are trees built up in the same way as t . An example of a typical rule is

$$q(\sigma(x_1, x_2), y_1, y_2) \rightarrow \delta(y_1, a, q_1(x_2, q_2(x_2, \beta(y_2)), b)),$$

where δ, β, a , and b are output symbols (of rank 3, 1, 0, and 0, respectively) and q_1 and q_2 are states (of rank 3 and 2, respectively). A macro tree transducer is total deterministic if for every state q of rank $n+1$ ($n \geq 0$) and every input symbol σ of rank m ($m \geq 0$) there is exactly one rule with left-hand side $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n)$ in R .

The rule (*) is applied as a subtree rewriting rule, as usual, by replacing the variables and the parameters by trees in T_{Σ} and sentential forms (SF), respectively.

Sentential forms are the trees occurring during a derivation and they can be defined as follows: (i) For $\delta \in \Delta_k$, $k \geq 0$, and $t_1, \dots, t_k \in \text{SF}$, $\delta(t_1, \dots, t_k) \in \text{SF}$, and (ii) for $q \in Q$ with rank $k+1$, $k \geq 0$, $s \in T_{\Sigma}$, and $t_1, \dots, t_k \in \text{SF}$, $q(s, t_1, \dots, t_k) \in \text{SF}$.

Denoting a derivation step by $\Rightarrow$, as usual, M realizes the translation $\{(s, t) \in T_{\Sigma} \times T_A \mid q^{\text{in}}(s) \xRightarrow{*} t\}$. Just as for macro grammars (and context-free tree grammars), the derivations may be restricted to inside-out (IO) derivations (trees in T_A should be substituted for the parameters) or outside-in (OI) derivations (a subtree may not be rewritten if it is in a context parameter of a state). Actually, writing $\langle q, s \rangle(t_1, \dots, t_n)$ instead of $q(s, t_1, \dots, t_n)$ a macro tree transducer turns into a context-free grammar with (infinitely many) nonterminals in $Q \times T_{\Sigma}$. Hence, as for context-free tree grammars, OI-derivations are equivalent to unrestricted derivations, but the IO-derivations produce less. The classes of translations realized by total deterministic and nondeterministic macro tree transducers using IO- and OI-derivations are denoted by D_tMT , MT_{IO} , and MT_{OI} , respectively. A total deterministic macro tree transducer realizes the same mapping in both the IO- and OI-derivation mode.

Thus, on the one hand, a macro tree transducer turns into a context-free tree grammar by dropping the syntactic parameter (e.g., incorporating it into the state). On the other hand, by dropping the context parameters, i.e., by restricting the ranks of all states of the macro tree transducer to be 1, the usual top-down tree transducer is obtained. Finally, macro tree transducers are the appropriate generalization to trees of top-down tree-to-string transducers (obtained by taking the yield of the output trees of top-down tree transducers, see, e.g., [19], in the same way as top-down tree transducers are generalized² sequential machines [36, 12, 19] and context-free tree grammars are generalized context-free grammars [35]. The generalization consists of first viewing a string (for instance, the right-hand side $w_0 q_1(x_1) w_1 \dots q_n(x_n) w_n$ of a top-down tree-to-string transducer) as a monadic tree (giving each output symbol and each "symbol" $q_i(x_i)$ rank 1, i.e., q_i

has rank 2), and then generalizing such trees by allowing the symbols to have rank > 1 . Thus the context parameters of the states are created.

Hence, macro tree transducers generalize context-free tree grammars, top-down tree transducers, and top-down tree-to-string transducers. A picture of this is given in Fig. 1, where the dimensions (1), (2), and (3) can be explained as follows. Following direction (1) means to drop the syntactic parameter (i.e., the input). Note that a top-down tree transducer turns into a regular tree grammar (see [13]; it is a regular set of equations to be solved in the algebra of trees, see [31]). Direction (2) means to drop the context parameters. Finally, direction (3) means to restrict all output symbols to have rank 1 or 0, and to allow at most one context parameter. This means that the right-hand sides of rules are monadic trees, to be viewed as strings. The diagram can be generalized (in the direction opposite to (2)) as discussed in [9, 15], by observing that one can follow the diagonals from *RTG* to *CFG* and from *T* to *yT* by taking the yield of the output trees; doing the same for *CFTG* and *MT* (obtaining the macro grammar [23] and the macro tree-to-string transducer), and then going up (in the direction opposite to (3)), one obtains the next part of the diagram. Iterating this process the higher level tree grammars [30, 20, 9] and the higher level macro tree transducer [10] are obtained.

We now turn to the connections of macro tree transducers with syntax-directed translations. The semantic of a programming language the syntax of which is given by a context-free grammar is mostly described in a syntax-directed way: the meaning of a syntactic construct is defined in terms of the meanings of its direct components using certain operations on a semantic domain. In top-down tree transducers (considered as models for this way of providing semantics) the meanings are represented by the states. In general, the meaning of a construct may also depend on the context in which it occurs. Hence, it is reasonable to have means of representing context information and updating it. In macro tree transducers the for-

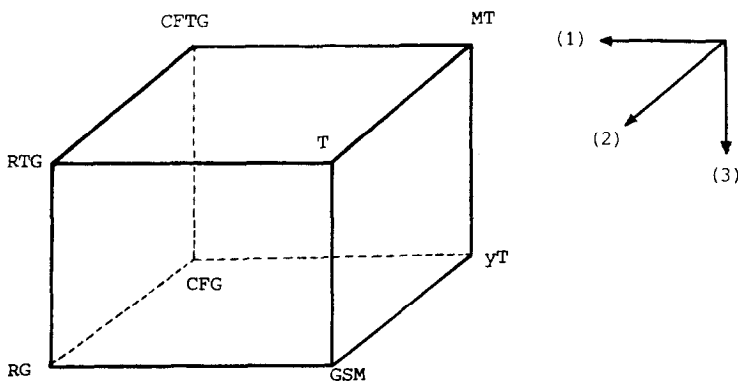


FIG. 1. The macro tree transducer (*MT*) generalizes the context-free tree grammar (*CFTG*), the top-down tree transducer (*T*), the top-down tree-to-string transducer (*yT*), the context-free grammar (*CFG*), the generalized sequential machine (*GSM*), the regular tree grammar (*RTG*) and the regular or right-linear grammar (*RG*). For an explanation of the three dimensions of the cube, see the text.

mal parameters serve this purpose: they can “store” context information (i.e., trees in T_A) and this information can be changed with each derivation step (due to the trees with parameters and output symbols in the context parameter positions of states in the right-hand sides of rules). Hence, macro tree transducers formalize the concept of syntax-directed semantics “with context.”

For the modelling of aspects of denotational semantics total deterministic macro tree transducers are appropriate. For convenience, the (uniquely determined) tree in $T_A(Y)$ derived from $q(s, y_1, \dots, y_n)$ by a total deterministic macro tree transducer is denoted by $M_q(s)$ (treating the $y_1, \dots, y_n$ as output symbols of rank 0). Intuitively, $M_q(s)$ is the q -meaning or q -translation of (the syntactic construct) s in the formal context $(y_1, \dots, y_n)$. By substituting in $M_q(s)$, $u_1, \dots, u_n \in T_A$ for $y_1, \dots, y_n$, respectively, the q -translation of s in the actual context $(u_1, \dots, u_n)$ is obtained. Hence, $M_q(s)$ may be viewed as a (“derived”) function $T_A^n \rightarrow T_A$ and the tree obtained from the above substitution may be denoted by $M_q(s)(u_1, \dots, u_n)$. Consider now, for instance, the semantic clause of a compound statement in a programming language: $\mathcal{C}[\langle S_1; S_2 \rangle](\rho, \sigma) = \mathcal{C}[S_2](\rho, \mathcal{C}[S_1](\rho, \sigma))$ where $\mathcal{C}$ is a semantic function of type $\text{statement} \rightarrow (\text{environment} \times \text{state} \rightarrow \text{state})$, S_1 and S_2 are syntactic variables and ρ and σ are variables over the corresponding semantic domain [26]. This semantic clause can be formulated as a rule for a macro tree transducer: $\mathcal{C}(\langle S_1; S_2 \rangle(x_1, x_2), y_1, y_2) \rightarrow \mathcal{C}(x_2, y_1, \mathcal{C}(x_1, y_1, y_2))$, where $\mathcal{C}$ denotes now a state of rank 3 and $\langle S_1; S_2 \rangle$ an input symbol of rank 2. Let t_ρ and $t_\sigma \in T_A$ denote an actual environment and a state, respectively, and let both s_1 and s_2 denote derivation trees of statements of the programming language. Then, using the notion of $\mathcal{C}$ -translation we obtain $M_{\mathcal{C}}(\langle S_1; S_2 \rangle(s_1, s_2))(t_\rho, t_\sigma) = M_{\mathcal{C}}(s_2)(t_\rho, M_{\mathcal{C}}(s_1)(t_\rho, t_\sigma))$, which corresponds closely to the above-mentioned semantic clause.

In spite of these close connections, a macro tree transducer can only handle context information as basic objects, i.e., elements of T_A . Thus, for instance, the environment ρ and the state σ , which are in general functions, should be considered as objects of some ground type; in general, parameters of higher type, which are more suitable for the modelling of denotational semantics of programming languages, cannot be expressed in this model. Furthermore, using the model of the present paper the semantic clause of recursive constructs like the while statement cannot be handled. This control structure would require e -moves (i.e., rules in which no input is consumed) or the explicit use of a symbol denoting the least fixed-point of a function. In [10] higher level tree transducers are studied in order to solve these problems. We note that, modelling denotational semantics, it is more natural to use many-sorted alphabets (as done in [8]) rather than ranked ones. However, since the difference is not very large and since the formalism gets rather complicated in the many-sorted case, we decided to stick to ranked alphabets.

In attribute grammars [29] synthesized attributes are used to represent the meanings of syntactic constructs and the inherited attributes represent the context information. Simulating synthesized and inherited attributes by states and formal parameters, respectively, attribute grammars can be translated into macro tree transducers. In fact, in [8] an algorithm is provided how to simulate an attribute

grammar by means of a primitive recursive scheme with parameters. Since the simulation is schematic, i.e., does not depend on the semantic domain, the algorithm translates an attribute grammar (viewed as a tree transducer, see [15, 18]) into a macro tree transducer, i.e., a restricted type of denotational semantics. Thus, macro tree transducers are a sufficiently powerful model to study attribute grammars.

In this paper we are concerned with total deterministic macro tree transducers as well as with their nondeterministic versions. Our main interest is focused on typical automata theoretical questions. Hence, as a continuation of [12, 14] we study the decomposition of the classes D , MT , MT_{IO} , and MT_{OI} into simpler classes of translations. It turns out that every macro tree transducer can be decomposed into top-down tree transducers and YIELD-mappings (a YIELD-mapping is a mapping from trees to trees that translates an expression in which the operation of tree substitution is used symbolically, into the tree it denotes; see [30, 20, 9, 8]).

This implies that the domain of a macro tree transducer is recognizable. In order to study the power of macro tree transducers we consider compositions $K_1 \circ K_2$, where K_1 and K_2 are classes of macro tree transductions. Unfortunately, the classes D , MT , MT_{IO} , and MT_{OI} are not closed under composition. Hence, positive results appear only, if one of the classes K_1 and K_2 contains top-down tree transductions only. Moreover, K_1 should be a class of deterministic transductions, because, otherwise, we would run into the usual problem of nondeterminism followed by copying. (This combination cannot be handled by top-down tree transducers; the other "solution," to restrict K_2 to noncopying transductions is not considered in this paper). We show in particular that total deterministic macro tree transductions are closed under (left and right) composition with total deterministic top-down tree transductions. Finally, we enlarge the transducer class in question by regular look-ahead (as was done in [14] for top-down tree transducers; see also [19]) and show that most of the classes of macro tree transductions are closed under regular look-ahead, i.e., the addition of the regular look-ahead does not increase the transformational power of the macro tree transducer. This enlargement facilitates a comparison with bottom-up tree transducers.

This paper is divided into 7 sections. Section 2 provides some preliminaries. In Section 3 macro tree transducers are defined and the connections to top-down tree transducers and context-free tree grammars are discussed. Inductive characterizations are given for the derivation relation of a macro tree transducer (similar to the least fixed-point characterization of context-free tree languages, see [20]) and, using these, finiteness properties are proved concerning the exponential height of output trees and the double exponential length of derivations (in terms of the height of the input tree). Decomposition and composition results for total deterministic macro tree transductions are studied in Section 4. Furthermore the addition of regular look-ahead is investigated. Section 5 is concerned with IO macro tree transductions. A decomposition is proved and it is shown how bottom-up tree transductions [36, 12] can be simulated by macro tree transducers using IO-derivation mode. This section concludes with a discussion about regular

look-ahead for IO macro tree transducers. In Section 6 OI macro tree transductions are decomposed and the question of regular look-ahead is investigated. Section 7 gives some conclusions and remarks.

2. PRELIMINARIES

We recall and invent some notations, basic definitions, and terminology which will be used in the rest of the paper. Nevertheless the reader is assumed to be familiar with the basic concepts of tree language theory (see, e.g., [34, 37, 12, 13]). Hence, we sometimes only recall the notations of objects.

2.1. General Notations

In the proofs of this paper the reference to the local induction hypothesis is abbreviated by I.H. Let $[n]$ denote the subset $\{1, \dots, n\}$ of the set of natural numbers that is denoted by $\mathbb{N}$. If A is a set and $v \in A^n$ is an n -tuple of elements of A , then for every $i \in [n]$, $v(i)$ denotes the i th component of v .

The empty set is denoted by $\emptyset$. For a set A , $\mathcal{P}(A)$ is the set of subsets of A . For two sets A and B a relation R from A into B is any subset of $A \times B$. The inverse of R is denoted by R^{-1} . The composition of two relations R_1 and R_2 , denoted by $R_1 \circ R_2$, is the set of pairs $\{(x, z) \mid (x, y) \in R_1 \text{ and } (y, z) \in R_2 \text{ for some } y\}$. R^n is the n -fold composition of R with itself (R^0 is the identity), R^* and R^+ denote the reflexive, transitive closure and the transitive closure of R , respectively. The domain and range of R are denoted by $\text{dom}(R)$ and $\text{out}(R)$, respectively. Furthermore, the image of a set C under R , denoted by $R(C)$, is the set $\{y \mid (x, y) \in R \text{ and } x \in C\}$. All these notions can be easily transferred to classes of relations for which we use the same notations.

2.2. Trees

A ranked alphabet is an alphabet in which each symbol has a unique rank in $\{0, 1, 2, \dots\}$. The subset Σ_m of the ranked alphabet Σ is the set of symbols of rank m ($m \geq 0$). Note that, for $i \neq j$, Σ_i and Σ_j are disjoint. In constructions and proofs of the present paper we use two mechanisms to define ranked alphabets and even sometimes a mixture of both.

We can define a ranked alphabet Σ by enumerating its finite subsets Σ_m that are not empty, or we can define Σ by giving a set of symbols that are indexed with their (unique) rank. For example, if $\Sigma = \{a, b, c\}$ and a, b , and c are of rank 0, 2, and 7, respectively, then we can describe Σ by $\Sigma_0 = \{a\}$, $\Sigma_2 = \{b\}$, and $\Sigma_7 = \{c\}$, or by $\{a^{(0)}, b^{(2)}, c^{(7)}\}$.

The set of labeled trees over a ranked alphabet Σ is denoted by T_Σ ; $\sigma(s_1, \dots, s_m)$ is the tree with root labeled $\sigma \in \Sigma_m$ and direct subtrees $s_1, \dots, s_m \in T_\Sigma$ (if $m = 0$, we often write σ rather than $\sigma()$). Monadic trees, i.e., trees in which only symbols of rank 1 and one symbol of rank 0 occur, are written as strings; for example, $b(c(d(a)))$ and $f(f(f(a)))$ are denoted by $bcda$ and f^3a , respectively.

For a set S of symbols or trees, $T_{\Sigma}(S)$ is the set of trees defined inductively as follows: (i) $S \subseteq T_{\Sigma}(S)$ and (ii) for $\sigma \in \Sigma_k$, $k \geq 0$, and $t_1, \dots, t_k \in T_{\Sigma}(S)$, $\sigma(t_1, \dots, t_k) \in T_{\Sigma}(S)$. Thus $T_{\Sigma} = T_{\Sigma}(\emptyset)$. For a subset S' of S , a tree $t \in T_{\Sigma}(S)$ is said to be *linear* in S' if no symbol of S' occurs more than once in t . The *height* of a tree t , denoted by $\text{height}(t)$ or $h(t)$, is provided by the function $\text{height}: T_{\Sigma} \rightarrow \mathbb{N}$ which is defined inductively on the structure of t as follows: (i) For $\sigma \in \Sigma_0$, $\text{height}(\sigma) = 1$ and (ii) for $\sigma \in \Sigma_m$ with $m \geq 1$ and $s_1, \dots, s_m \in T_{\Sigma}$, $\text{height}(\sigma(s_1, \dots, s_m)) = 1 + \max\{\text{height}(s_i) \mid i \in [m]\}$. The *yield* of a tree t , denoted by $\text{yield}(t)$, is the sequence of its leaf labels.

Any subset of T_{Σ} is called a *tree language* over Σ . The class of *recognizable tree languages* is denoted by RECOG.

2.3. Tree Substitution

Let $X = \{x_1, x_2, \dots\}$ be the set of (*input*) *variables* and $Y = \{y_1, y_2, \dots\}$ the set of (*formal*) *parameters*; $X_k = \{x_1, \dots, x_k\}$ and $Y_k = \{y_1, \dots, y_k\}$ for $k \geq 0$ (thus $X_0 = Y_0 = \emptyset$). For a set $Z_k = \{z_1, \dots, z_k\}$ of symbols and a tree $t \in T_{\Sigma}(Z_k)$ with $k \geq 0$ and trees $t_1, \dots, t_k \in T_{\Sigma}$, we denote by $t[z_1/t_1, \dots, z_k/t_k]$ the result of substituting t_i for every occurrence of z_i in t . Note that for $k=0$, $t[z_1/t_1, \dots, z_k/t_k] = t[\] = t$. We will use the same definition in case $z_1, \dots, z_k$ are trees (such that for no $i \neq j$, z_i is a subtree of z_j). If the "substitution variables" $z_1, \dots, z_k$ are understood from the context, we write $t[t_1, \dots, t_k]$ rather than $t[z_1/t_1, \dots, z_k/t_k]$; this will usually be the case if $z_1, \dots, z_k \in Y$. For $\pi = (s_1, \dots, s_m) \in T_{\Sigma}^m$ and $\rho = (t_1, \dots, t_n) \in T_{\Sigma}^n$ with $m, n \geq 0$ and a tree $t \in T_{\Sigma}(X_m \cup Y_n)$ we often use $t\langle \pi, \rho \rangle$ to abbreviate $t[x_1/s_1, \dots, x_m/s_m, y_1/t_1, \dots, y_n/t_n]$.

For tree languages $L \in \mathcal{P}(T_{\Sigma}(Y_k))$ and $L_1, \dots, L_k \in \mathcal{P}(T_{\Sigma}(Y_m))$ with $k \geq 0$ and $m \geq 0$, we recall the definition of IO- and OI-substitution of $L_1, \dots, L_k$ into L , denoted by $L \xleftarrow{\text{IO}} (L_1, \dots, L_k)$ and $L \xleftarrow{\text{OI}} (L_1, \dots, L_k)$, respectively, from [20] (Definition 2.1.1 with Y instead of X),

$$L \xleftarrow{\text{IO}} (L_1, \dots, L_k) = \{t[t_1, \dots, t_k] \mid t \in L \text{ and } t_i \in L_i \text{ for } i \in [k]\};$$

$L \xleftarrow{\text{OI}} (L_1, \dots, L_k)$ is defined inductively as follows:

- (i) For $\sigma \in \Sigma_0$, $\sigma \xleftarrow{\text{OI}} (L_1, \dots, L_k) = \{\sigma\}$.
- (ii) For $i \in [k]$, $y_i \xleftarrow{\text{OI}} (L_1, \dots, L_k) = L_i$.
- (iii) For $m \geq 1$, $\sigma \in \Sigma_m$, and $t_1, \dots, t_m \in T_{\Sigma}(Y_k)$, $\sigma(t_1, \dots, t_m) \xleftarrow{\text{OI}} (L_1, \dots, L_k) = \{\sigma(s_1, \dots, s_m) \mid \text{for } i \in [m], s_i \in t_i \xleftarrow{\text{OI}} (L_1, \dots, L_k)\}$.
- (iv) For $L \in T_{\Sigma}(Y_k)$, $L \xleftarrow{\text{OI}} (L_1, \dots, L_k) = \bigcup \{t \xleftarrow{\text{OI}} (L_1, \dots, L_k) \mid t \in L\}$.

Note that both $L \xleftarrow{\text{IO}} (L_1, \dots, L_k)$ and $L \xleftarrow{\text{OI}} (L_1, \dots, L_k)$ are in $\mathcal{P}(T_{\Sigma}(Y_m))$.

Viewing tree substitution as a symbolic operation (denoted, e.g., by sub) the meaning of trees containing sub can be obtained by means of a YIELD-mapping (see, e.g., [30, 20, 9]) which is defined as follows: Let f be a mapping from Σ_0 to $T_{\Sigma}(Y)$. Then f is extended to a mapping $\text{YIELD}_f: T_{\Sigma} \rightarrow T_{\Sigma}(Y)$ by (i) for $\sigma \in \Sigma_0$, $\text{YIELD}_f(\sigma) = f(\sigma)$, and (ii) for $\sigma \in \Sigma_{k+1}$ with $k \geq 0$ and $s_0, s_1, \dots, s_k \in T_{\Sigma}$, $\text{YIELD}_f(\sigma(s_0, s_1, \dots, s_k)) = \text{YIELD}_f(s_0) [\text{YIELD}_f(s_1), \dots, \text{YIELD}_f(s_k)]$ (note that

$\text{YIELD}_f(s_0)$ may contain formal parameters y_i with $i > k$; these remain the same). The class of these mappings is denoted by YIELD . Note that this definition of YIELD is more general than the one in the literature, where YIELD_f is only defined on T_Σ if Σ is a "derived alphabet" containing explicit sub-symbols; here, we interpret every symbol as substitution, just as the yield of a tree is obtained by interpreting every symbol as concatenation. Most results in the literature remain true after this slight extension of YIELD .

2.4. Bottom-Up Tree Transducers

A *bottom-up tree transducer* is a construct $M = (Q, \Sigma, \Delta, Q_f, R)$ with states (of rank 1), ranked input and output alphabets, final states, and rules. Every rule in R is of the form $\sigma(q_1(x_1), \dots, q_k(x_k)) \rightarrow q(t)$ with $\sigma \in \Sigma_k$ ($k \geq 0$), $q_1, \dots, q_k, q \in Q$, and $t \in T_\Delta(X_k)$. These rules turn into tree rewriting rules, as usual, by substituting trees in T_Σ for the x s. The *translation realized by M* , denoted $\tau(M)$, is $\{(s, t) \in T_\Sigma \times T_\Delta \mid s \xrightarrow{*} q(t) \text{ for some } q \in Q_f\}$. (For more precise definitions, see, e.g., [12, 13]).

The class of bottom-up tree transducers is denoted by B_a with the prefixes D , t , and D_t to express determinism, totality, and total determinism, respectively. A bottom-up tree transducer is called a *bottom-up finite tree automaton*, if $\Delta = \Sigma$ and every rule looks like $\sigma(q_1(x_1), \dots, q_k(x_k)) \rightarrow q(\sigma(x_1, \dots, x_k))$. The class of bottom-up finite tree automata is denoted by $B\text{-FTA}_a$. If R_a is a class of bottom-up tree transducers, R denotes the class of induced translations. Thus, $\text{dom}(B\text{-FTA}) = \text{dom}(DB\text{-FTA}) = \text{RECOG}$.

We will use the (classical) way of specifying a finite tree automaton in $D_t B\text{-FTA}_a$ by a triple (P, Σ, δ) , where P is the finite set of states, Σ is the ranked input alphabet, and δ is a family $\{\delta_\sigma\}$ of mappings $\delta_\sigma: P^k \rightarrow P$ with $\sigma \in \Sigma_k$, $k \geq 0$. The function $\delta: T_\Sigma \rightarrow P$ is defined as usual (cf. [38]) recursively on the structure of the tree in T_Σ : for $\sigma \in \Sigma_k$ with $k \geq 0$ and $t_1, \dots, t_k \in T_\Sigma$, $\delta(\sigma(t_1, \dots, t_k)) = \delta_\sigma(\delta(t_1), \dots, \delta(t_k))$. Intuitively, $\delta(t)$ is the state in which the tree automaton arrives at the root of t . An automaton in $DB\text{-FTA}_a$ is specified by a 4-tuple (P, Σ, δ, F) , where $F \subseteq P$ is the set of final states.

2.5. Simultaneous Induction

A great number of definitions and proofs in this paper are done by a kind of double induction on trees that will be called simultaneous induction. Although this is only technical stuff, we want to convince the reader that this concept really provides definitions and proofs.

Let Σ be a ranked alphabet and A a set. The mappings $g: \bigcup \{T_\Sigma^m \mid m \geq 0\} \rightarrow A$ and $f: T_\Sigma \rightarrow A$ are said to be *defined by simultaneous induction* if the following holds:

- (a) For all $\sigma \in \Sigma_m$ with $m \geq 0$ and $s_1, \dots, s_m \in T_\Sigma$, $g((s_1, \dots, s_m))$ is used to define $f(\sigma(s_1, \dots, s_m))$.
- (b) For all $s_1, \dots, s_m \in T_\Sigma$ with $m \geq 0$, $f(s_1), \dots, f(s_m)$ are used to define $g((s_1, \dots, s_m))$.

Using this concept the mappings f and g are well defined. In fact, by (b), for $m=0$, $g(())$ can be defined immediately (without the use of f), where $()$ is the empty sequence. Then, by (a), $f(\sigma)$ can be defined for every $\sigma \in \Sigma_0$. Now assume that $f(s_1), \dots, f(s_m)$ for $m \geq 0$ are already defined. Then, by (b), $g((s_1, \dots, s_m))$ can be defined and therefore, using (a) again, $f(\sigma(s_1, \dots, s_m))$ can be defined for every $\sigma \in \Sigma_m$. Hence, f is defined everywhere. Then (b) implies that $g((s_1, \dots, s_m))$ can be defined for every tuple $(s_1, \dots, s_m)$.

Let Q and P be two predicates which range over $\bigcup \{T_\Sigma^m | m \geq 0\}$ and T_Σ , respectively. Q and P are said to be *proved by simultaneous induction* if (a) and (b) are proved:

(a) For all $m \geq 0$, $\sigma \in \Sigma_m$, and $s_1, \dots, s_m \in T_\Sigma$, if $Q((s_1, \dots, s_m))$ holds, then $P(\sigma(s_1, \dots, s_m))$ holds.

(b) For all $m \geq 0$ and $s_1, \dots, s_m \in T_\Sigma$, if $P(s_1)$ and $P(s_2)$ and $\dots$ and $P(s_m)$ hold, then $Q((s_1, \dots, s_m))$ holds.

Using a similar argumentation as for the definition by simultaneous induction, it is easy to see that, if P and Q are proved by simultaneous induction, then for all $s, s_1, \dots, s_m \in T_\Sigma$ with $m \geq 0$, $P(s)$ and $Q((s_1, \dots, s_m))$ hold.

In fact, the above induction principles are just a special case of the ordinary induction principles for trees over a many-sorted alphabet (see [27] and Theorem 6.5 of [6]). To explain this, consider, for a ranked alphabet Σ , the (infinite) many-sorted alphabet $s(\Sigma)$ with sorts t and t_m (for every $m \geq 0$) and operation symbols tup_m and app_σ (for every $m \geq 0$ and $\sigma \in \Sigma_m$) of type $t^m \rightarrow t_m$ and $t_m \rightarrow t$, respectively. Consider now the tree algebra F with carriers T_Σ and T_Σ^m of sort t and t_m , respectively, and with the operations $(\text{tup}_m)_F(s_1, \dots, s_m) = (s_1, \dots, s_m)$ and $(\text{app}_\sigma)_F((s_1, \dots, s_m)) = \sigma(s_1, \dots, s_m)$ for $s_1, \dots, s_m \in T_\Sigma$. Clearly, F is the free $s(\Sigma)$ -algebra, and the above induction principles are the usual ones for $T_{s(\Sigma)}$ (note that Q is the union of predicates Q_m on T_Σ^m).

3. DEFINITIONS AND ELEMENTARY PROPERTIES

In this section we will formally define the macro tree transducer and associate a tree translation with it by means of a derivation relation. Context-free tree grammars are simulated by macro tree transducers and conversely macro tree transducers modelled by context-free tree grammars (cf. Sect. 1). The translation of a macro tree transducer is characterized inductively (in two ways). Finally, two bounds on derivations are pointed out: the height of output trees is always exponentially bounded and the length of a derivation always double exponentially bounded with respect to the height of the input tree.

3.1. Definition of Macro Tree Transducer

Although the new transducer model can be considered as a natural extension of the top-down tree transducer, the right-hand sides of rules are a bit more com-

plicated. To avoid a complex definition of macro tree transducers we put the definition of the set of possible right-hand sides in front.

Building up a right-hand side for a macro tree transducer rule we may use output symbols, formal parameters $y_1, \dots, y_n$ (for some $n \geq 0$), and constructs $q(x_i, t_1, \dots, t_k)$ with $x_i \in X_m$. The arguments $t_1, \dots, t_k$ of q may again contain all the mentioned constructs. Thus, for instance, nested states may occur which correspond to nested procedure calls in programming languages. Of course, in a right-hand side the allowed input variables and formal parameters are prescribed by the left-hand side of the rule.

3.1. DEFINITION. Let Q and Δ be ranked alphabets and $m, n \geq 0$. The set of *right-hand sides* $\text{RHS}(Q, \Delta, m, n)$ over Q and Δ with m variables and n parameters is the smallest set $\text{rhs} \subseteq T_{Q \cup \Delta}(X_m \cup Y_n)$ satisfying the following three conditions:

- (i) $Y_n \subseteq \text{rhs}$.
- (ii) For $\delta \in \Delta_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{rhs}$, $\delta(\xi_1, \dots, \xi_k) \in \text{rhs}$.
- (iii) For $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$ and $\xi_1, \dots, \xi_k \in \text{rhs}$, $q(x_i, \xi_1, \dots, \xi_k) \in \text{rhs}$.

If Q, Δ, m , and n are understood from the context we write RHS rather than $\text{RHS}(Q, \Delta, m, n)$. Note that in case (ii) $k = 0$ is included. As an example, for $m = 3$ and $n = 4$, $\delta(a, q(x_1, \gamma(q'(x_1, y_2, a))), y_1)$ is in $\text{RHS}(Q, \Delta, 3, 4)$, where $a \in \Delta_0$, $\gamma \in \Delta_1$, $\delta \in \Delta_3$, $q \in Q_2$, and $q' \in Q_3$.

Now we are in the position to define the macro tree transducer [15, 8].

3.2. DEFINITION. (i) A *macro tree transducer* M is a tuple $(Q, \Sigma, \Delta, q^{\text{in}}, R)$ consisting of three ranked alphabets Q, Σ , and Δ of states, input and output symbols, respectively (each state has at least rank 1), an initial state q^{in} of rank 1, and a finite set R of rules of the form

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \quad (*)$$

where $m, n \geq 0$, $q \in Q_{n+1}$, $\sigma \in \Sigma_m$, and $t \in \text{RHS}(Q, \Delta, m, n)$.

(ii) A macro tree transducer M is *deterministic (total)*, if for each pair $q \in Q_{n+1}$ and $\sigma \in \Sigma_m$ there is at most (at least) one rule in R with left-hand side $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n)$.

(iii) A macro tree transducer is (input) *linear*, if in the right-hand side of any rule in R a variable of the left-hand side occurs at most once, i.e., if $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t$ is in R , then t is linear in X_m . (The analogous concept of linearity with respect to the parameters will not be considered.)

The class of macro tree transducers is denoted by MT_a . The class of deterministic {total, total deterministic, linear} macro tree transducers is denoted by $DMT_a\{, MT_a, D_t MT_a, LMT_a\}$, and similarly for other combinations.

Rules like (*) in Definition 3.2 are called (q, σ) -rules. Intuitively, $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n)$ computes the q -translation of the subtree $\sigma(x_1, \dots, x_m)$ in the environment $y_1, \dots, y_n$.

As a special case of MT_a one can easily obtain the class of top-down tree transducers (denoted by T_a): a top-down tree transducer is a macro tree transducer such that every state has rank 1. Then the rules are of the form

$$q(\sigma(x_1, \dots, x_m)) \rightarrow t,$$

where $t \in \text{RHS}(Q, A, m, 0) = T_a(Q(X_m))$. Thus, $T_a \subseteq MT_a$ (cf. Sect. 1) and furthermore the restrictions of Definition 3.2 are the usual ones (cf. [34, 12]).

A top-down tree transducer is called a *top-down finite tree automaton*, if $A = \Sigma$ and every rule is of the form

$$q(\sigma(x_1, \dots, x_m)) \rightarrow \sigma(q_1(x_1), \dots, q_m(x_m)),$$

for $q, q_1, \dots, q_m \in Q$ with $m \geq 0$ and $\sigma \in \Sigma_m$. The class of top-down finite tree automata is denoted by $T\text{-FTA}_a$.

As customary we associate a tree translation with a macro tree transducer by means of a derivation relation. By substituting trees of T_Σ and sentential forms (these are trees in $T_{Q \cup \Sigma \cup A}$) for the variables and the parameters, respectively, a rule (*) turns into a subtree rewriting rule with which the derivation relation can be defined. To make this clearer let us look at the following example.

3.3. EXAMPLE. Let $M = (Q, \Sigma, A, q^{\text{in}}, R) \in MT_a$, and let $p: q(\sigma(x_1, x_2), y_1, y_2, y_3) \rightarrow g(a, y_1, q'(x_1, f(y_2), y_1))$ be a rule in R (see also Fig. 2a). Now consider a tree ϕ in which the structure of the left-hand side of p is present, for instance, $\phi = \beta[z_1/q(\sigma(s_1, s_2), t_1, t_2, t_3)]$ (see also Fig. 2b), where $\beta \in T_{Q \cup \Sigma \cup A}(\{z_1\})$ and z_1 occurs exactly once in β ; $s_1, s_2 \in T_\Sigma$ and t_1, t_2 , and $t_3 \in T_{Q \cup \Sigma \cup A}$ (β contains z_1 only for the purpose of representing ϕ in a suitable way). According to the rule p we may replace the subtree $q(\sigma(s_1, s_2), t_1, t_2, t_3)$ of ϕ by the right-hand side of p in which

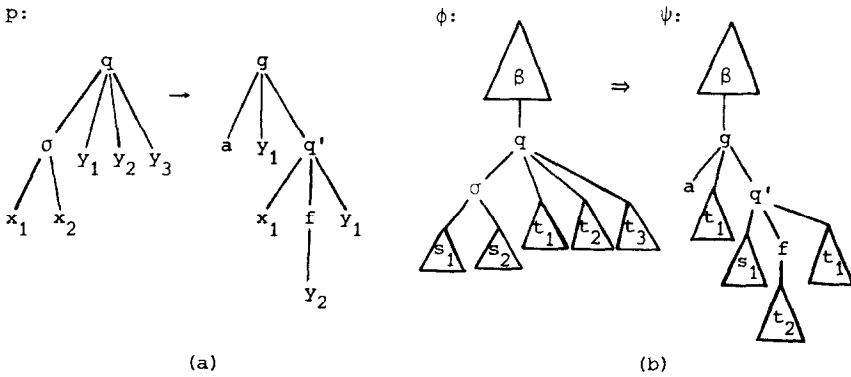


FIG. 2. A rule and a derivation step of a macro tree transducer.

the trees s_1, s_2 , and t_1, t_2, t_3 are substituted for x_1, x_2 , and y_1, y_2, y_3 , respectively, i.e., by $g(a, y_1, q'(x_1, f(y_2), y_1)) \langle (s_1, s_2), (t_1, t_2, t_3) \rangle$. Hence, we obtain the tree $\psi = \beta[z_1/g(a, t_1, q'(s_1, f(t_2), t_1))]$, derivable from ϕ in one step. Thus ϕ and ψ are in the derivation relation $\Rightarrow$: $\phi \Rightarrow \psi$ (see Fig. 2b).

Since states may be nested we have the choice to which state we apply a rule first. There are two possibilities corresponding to the main methods in which parameters of a procedure in a programming language are handled: call by value and call by name. (Note that in the program schematological consideration of a macro tree transducer the first parameter, i.e., the input variable, is always a call by value parameter, because it should contain a tree in T_Σ .) Using the first strategy, the values of the parameters have to be calculated first and then transferred to the procedure body. Hence, if we want to have a call by value or inside-out (IO) derivation relation for the macro tree transducer, the trees in the positions of the formal parameters of a state have to be trees over the output alphabet. In the call by name strategy the parameters of a procedure call may themselves contain procedure calls and are transferred to the body in an unevaluated form. It is well known that in the call by name case we may use "normal order of evaluation," i.e., evaluate outermost occurrences of procedure calls first. Hence, in a call by name or outside-in (OI) derivation relation we apply a rule to one of the outermost states of the tree in question. An occurrence of a state is outermost in the tree t if on the path from this state to the root of t no other states occur. The IO- and OI-derivation relation are special cases of the unrestricted version in which to an arbitrary state of the tree a (suitable) rule may be applied.

In the following definition recall that $t \langle (s_1, \dots, s_m), (t_1, \dots, t_n) \rangle$ denotes $t[x_1/s_1, \dots, x_m/s_m, y_1/t_1, \dots, y_n/t_n]$, see Section 2.3.

3.4. DEFINITION. Let $M = (Q, \Sigma, A, q^{\text{in}}, R) \in MT_a$ and $\phi, \psi \in T_{Q \cup \Sigma \cup A}$. The binary relations $\Rightarrow_{M, \text{unr}}, \Rightarrow_{M, \text{IO}}, \Rightarrow_{M, \text{OI}} \subseteq T_{Q \cup \Sigma \cup A} \times T_{Q \cup \Sigma \cup A}$ are given by:

- (i) $\phi \Rightarrow_{M, \text{unr}} \psi$ if there is a rule in R of the form

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t,$$

and there is a $\beta \in T_{Q \cup \Sigma \cup A}(\{z_1\})$ in which z_1 occurs exactly once, and there are $s_1, \dots, s_m \in T_\Sigma$ and $t_1, \dots, t_n \in T_{Q \cup \Sigma \cup A}$, such that $\phi = \beta[z_1/q(\sigma(s_1, \dots, s_m), t_1, \dots, t_n)]$ and $\psi = \beta[z_1/t \langle (s_1, \dots, s_m), (t_1, \dots, t_n) \rangle]$.

- (ii) $\phi \Rightarrow_{M, \text{IO}} \psi$ if $\phi \Rightarrow_{M, \text{unr}} \psi$, but the $t_1, \dots, t_n$ in (i) are restricted to be elements of T_A .

- (iii) $\phi \Rightarrow_{M, \text{OI}} \psi$ if $\phi \Rightarrow_{M, \text{unr}} \psi$, but for the z_1 in (i) the following holds: on the path from z_1 to the root of β no state occurs.

If M, IO, OI , or unr is understood from the context it is dropped; $\Rightarrow^*$ and $\Rightarrow^+$ are defined as usual.

If we consider macro tree transducers using the derivation mode $\mu \in \{\text{OI}, \text{IO}, \text{unr}\}$ we refer to them as μ -macro tree transducers.

Note that $\Rightarrow_{unr}$ is the usual derivation relation for top-down tree transducers. If there are no nested states in right-hand sides of rules of a macro tree transducer, then the three derivation modes coincide: every actual parameter of q is in T_A (i.e., $\Rightarrow_{unr} = \Rightarrow_{IO}$) and every q is outermost (i.e., $\Rightarrow_{unr} = \Rightarrow_{OI}$).

Definition 3.4 is very similar to the definition of the derivation relation of context-free tree grammars in [20]. The connections between these grammars and macro tree transducers are discussed in Section 3.2.

3.5. DEFINITION. Let $\mu \in \{IO, OI, unr\}$ and $M \in MT_a$:

(i) The μ -translation realized by M , denoted by $\tau_\mu(M)$, is $\{(s, t) \in T_\Sigma \times T_A \mid q^{in}(s) \xrightarrow{*}_{M, \mu} t\}$.

(ii) Let $\xi \in T_{Q \cup \Sigma \cup A}$. The μ -language derivable by M from ξ , denoted by $L_\mu(M, \xi)$, is $\{t \in T_A \mid \xi \xrightarrow{*}_{M, \mu} t\}$.

(iii) The μ -output language of M is $out(\tau_\mu(M))$.

Note that $\tau_\mu(M) = \{(s, t) \mid s \in T_\Sigma, t \in L_\mu(M, q^{in}(s))\}$ for $M \in MT_a$. The classes of μ -translations realized by MT_a , DMT_a , ${}_iMT_a$, and LMT_a are denoted by MT_μ , DMT_μ , ${}_iMT_\mu$, and LMT_μ and similarly for other combinations. The class of translations realized by top-down tree transducers is denoted by T . We denote $\text{dom}(DT\text{-}FTA)$ by $DT\text{-}RECOG$. Note that $DT\text{-}RECOG \subsetneq RECOG = \text{dom}(T\text{-}FTA)$, see [37]. Since $B\text{-}FTA = T\text{-}FTA$, we will denote this class by FTA .

Since a top-down tree transducer can be obtained from a macro tree transducer by restricting the rank of the states to be one, nested states do not occur. Hence, the inclusion of the translation classes is obvious.

3.6. PROPOSITION. $T \subseteq MT_{IO}$ and $T \subseteq MT_{OI}$.

Similar results hold for the restricted transducer classes.

How do the trees derivable from an initial sentential form $q^{in}(s)$ look like? Of course, they are elements of $T_{Q \cup \Sigma \cup A}$. But they appear only in a special shape: the first argument of a state is always a tree over the input alphabet Σ and these input symbols occur in the first argument of a state only. As suggested by the definition of right-hand sides, the other arguments of a state may contain output symbols and again states which satisfy the above explained restrictions. We will prove that all derivable trees beginning with $q^{in}(s)$ for a tree $s \in T_\Sigma$ are elements of $SF(Q, \Sigma, A)$. This set is defined as follows:

3.7. DEFINITION. Let Q , Σ , and A be ranked alphabets. The set of *sentential forms* over Q , Σ , and A , denoted by $SF(Q, \Sigma, A)$, is the smallest set $sf \subseteq T_{Q \cup \Sigma \cup A}$ satisfying the following conditions: (i) For $\delta \in A_k$ with $k \geq 0$ and $t_1, \dots, t_k \in sf$, $\delta(t_1, \dots, t_k) \in sf$, and (ii) for $q \in Q_{k+1}$ with $k \geq 0$, $s \in T_\Sigma$, and $t_1, \dots, t_k \in sf$, $q(s, t_1, \dots, t_k) \in sf$. If Q , Σ , and A are understood from the context, they are dropped.

3.8. PROPOSITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. For all $t \in \text{SF}$ and $t' \in T_{Q \cup \Sigma \cup \Delta}$, if $t \xRightarrow{\text{unr}} t'$, then $t' \in \text{SF}$.

Proof. First, the substitution of input trees and sentential forms for variables and parameters into a right-hand side delivers a sentential form.

$$\text{For all } t \in \text{RHS}(Q, \Delta, m, n), \pi \in T_{\Sigma}^m \text{ and } \rho \in \text{SF}^n: t\langle\pi, \rho\rangle \in \text{SF}. \quad (*)$$

This can easily be shown by induction on the structure of t , cf. Definition 3.1. Hence, every tree which is derivable in one step from a sentential form is again a sentential form.

$$\text{For all } t \in \text{SF} \text{ and } t' \in T_{Q \cup \Sigma \cup \Delta}: t \Rightarrow_{\text{unr}} t' \text{ implies } t' \in \text{SF}. \quad (**)$$

This can be shown by straightforward induction on the structure of t , cf. Definition 3.7, using (*) for case (ii) in 3.7, if a rule is applied to q . Clearly, this proves the proposition. ■

Note in particular, since every $q^{\text{in}}(s)$ with $s \in T_{\Sigma}$ is a sentential form, only elements of SF occur in the derivation process. Hence, we could also have defined the derivation relations as subsets of $\text{SF} \times \text{SF}$.

To make all these notions clearer we present the usual example [29] in which production trees for binary numbers are translated into trees denoting their decimal value. (Production trees are a kind of derivation trees of a context-free grammar in which the nodes are labelled with the applied productions rather than nonterminals, see [2].)

3.9. EXAMPLE. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in DMT_a$, where $Q = \{q^{\text{in}}, q^{(2)}\}$, $\Sigma = \{\langle S \rightarrow N \rangle^{(1)}, \langle N \rightarrow N0 \rangle^{(1)}, \langle N \rightarrow N1 \rangle^{(1)}, \langle N \rightarrow 1 \rangle^{(0)}\}$, $\Delta = \{0^{(0)}, 1^{(0)}, \text{exp}^{(1)}, +^{(2)}\}$, and R contains the following rules:

- (1) $q^{\text{in}}(\langle S \rightarrow N \rangle(x)) \rightarrow q(x, 0)$,
- (2) $q(\langle N \rightarrow N0 \rangle(x), y) \rightarrow q(x, +(y, 1))$,
- (3) $q(\langle N \rightarrow N1 \rangle(x), y) \rightarrow +(\text{exp}(y), q(x, +(y, 1)))$, and
- (4) $q(\langle N \rightarrow 1 \rangle, y) \rightarrow \text{exp}(y)$.

Interpreting the output symbols in the obvious way (with $\text{exp}(t)$ denoting 2^t) the information in the parameter position indicates the length of the already processed suffix part of the binary number. With this length information rule (3) can calculate the decimal value of a 1 in the binary number. A complete derivation is written down in Fig. 3. The input tree is a production tree for the binary number 110, the decimal value of which is 6; if we interpret the output tree we obtain indeed 6.

Note that, interpreted, M specifies the denotational semantics of binary numbers, in a meta-notation close to the usual one [26]: q is the name of a semantic function and, e.g., rule (3) would be written as semantic clause $q[\![N1]\!](y) = +(\text{exp}(y), q[\![N]\!](+(y, 1)))$ (see also Sect. 1).

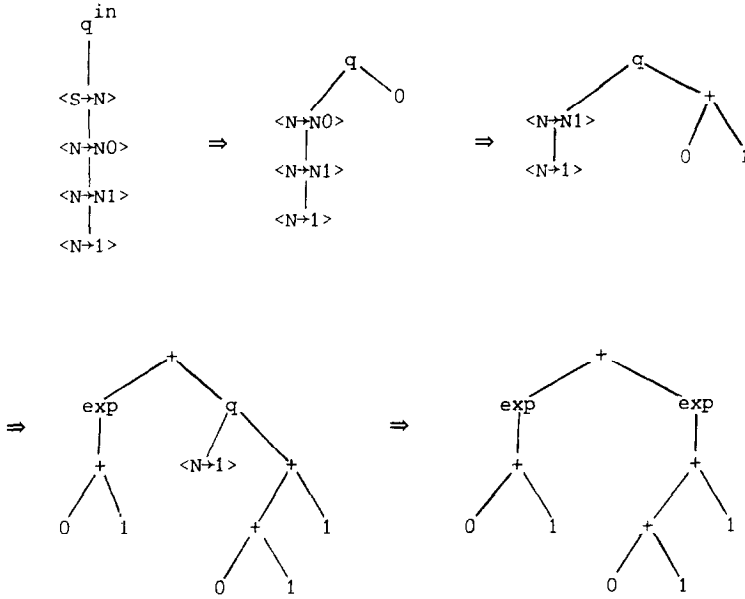


FIG. 3. A complete derivation of a macro tree transducer.

In this binary number example there is no difference between the three derivation relations, because there do not occur nested states in the rules of M . But of course, there are the usual examples [20] in which IO- and OI-derivation lead to different results.

3.10. EXAMPLE. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$, where $Q = \{q^{\text{in}}, p^{(2)}\}$, $\Sigma = \{f^{(1)}, a^{(0)}, b^{(0)}\}$, $\Delta = \{g^{(2)}, d^{(1)}, e^{(0)}, c^{(0)}\}$, and R contains the following rules:

- (1) $q^{\text{in}}(f(x)) \rightarrow p(x, q^{\text{in}}(x))$,
- (2), (3) $q^{\text{in}}(a) \rightarrow e|c$,
- (4) $p(a, y) \rightarrow g(y, y)$,
- (5) $p(b, y) \rightarrow e$, and
- (6) $p(f(x), y) \rightarrow p(x, d(y))$.

For the input tree $f(a)$ all possible IO-derivations are

$$q^{\text{in}}(f(a)) \Rightarrow p(a, q^{\text{in}}(a)) \Rightarrow p(a, e) \Rightarrow g(e, e)$$

and

$$q^{\text{in}}(f(a)) \Rightarrow p(a, q^{\text{in}}(a)) \Rightarrow p(a, c) \Rightarrow g(c, c)$$

and the OI-derivations are

$$\begin{aligned} q^{\text{in}}(f(a)) &\Rightarrow p(a, q^{\text{in}}(a)) \Rightarrow g(q^{\text{in}}(a), q^{\text{in}}(a)) \Rightarrow^2 g(e, e), \\ q^{\text{in}}(f(a)) &\Rightarrow \cdots \Rightarrow g(q^{\text{in}}(a), q^{\text{in}}(a)) \Rightarrow^2 g(c, e), \\ q^{\text{in}}(f(a)) &\Rightarrow \cdots \Rightarrow g(q^{\text{in}}(a), q^{\text{in}}(a)) \Rightarrow^2 g(e, c), \end{aligned}$$

and

$$q^{\text{in}}(f(a)) \Rightarrow \cdots \Rightarrow g(q^{\text{in}}(a), q^{\text{in}}(a)) \Rightarrow^2 g(c, c).$$

This difference between IO- and OI-derivation is caused by the order of copying and nondeterminism: whereas in the IO-mode nondeterminism (by application of rule (2) and (3)) is followed by copying (rule (4)), the OI-mode copies $q^{\text{in}}(a)$ first, which then can be derived nondeterministically.

There is another difference, which becomes obvious by considering $f(b)$ as input: the IO-derivation $q^{\text{in}}(f(b)) \Rightarrow p(b, q^{\text{in}}(b))$ stops, because there is no rule with left-hand side $q^{\text{in}}(b)$ in R . Hence $f(b) \notin \text{dom}(\tau_{\text{IO}}(M))$. But the OI-derivation yields a proper result: $q^{\text{in}}(f(b)) \Rightarrow p(b, q^{\text{in}}(b)) \Rightarrow e$; hence, $f(b) \in \text{dom}(\tau_{\text{OI}}(M))$.

This difference is caused by the order of “checking” followed by deletion: in the IO-derivation the “partial” state q^{in} has to be evaluated first and this cannot be done. In the OI-derivation the produced state p deletes its parameter in which a partial state occurs; the translation is successful.

The second difference in Example 3.10 reveals a nice property of an IO-macro tree transducer: it can check in the parameter positions of a state whether an input (sub-) tree is in a recognizable tree language; if it is in the required language, then the transducer can continue, otherwise it stops. This is a typical bottom-up property (checking followed by deletion, see [12] and in fact, later we will show that $B \subseteq MT_{\text{IO}}$, see Theorem 5.16).

We end this section by proving an elementary property of derivations. One could imagine that there are arbitrarily long derivations in a macro tree transducer, obtained by repeatedly nesting states. Whereas the height of one of the involved input subtrees always decreases whenever a rule is applied, a lot of copies of the subtrees may occur in the resulting sentential form. But the following shows that for a given sentential form t there is a bound on the length of the derivations starting with t . Even, for fixed t_0 , the bound for $t_0[y_1/t_1, \dots, y_n/t_n]$ depends only on the bounds for $t_1, \dots, t_n$. Later (in Theorem 3.25) we will provide a concrete bound on the length of the possible derivations of a macro tree transducer.

The following theorem is also given in [24].

3.11. THEOREM. *Let M be a macro tree transducer. For every sentential form t there is a bound on the length of derivations of M starting with t .*

Proof. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. Let $\Rightarrow$ denote $\Rightarrow_{M, \text{unr}}$. We prove the following two statements by simultaneous induction (see Sect. 2.5):

(1) For all $s \in T_\Sigma$ and all $q \in Q_{n+1}$ with $n \geq 0$ there is a function $g_{q,s}: \mathbb{N}^n \rightarrow \mathbb{N}$ such that for all $t_1, \dots, t_n \in \text{SF}$, if the derivations from t_i are not longer than c_i ($1 \leq i \leq n$), then the derivations from $q(s, t_1, \dots, t_n)$ are not longer than $g_{q,s}(c_1, \dots, c_n)$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$ and all $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$ there is a function $g_{\pi,t}: \mathbb{N}^n \rightarrow \mathbb{N}$ such that for all $\rho = (t_1, \dots, t_n) \in \text{SF}^n$, if the derivations from t_i are not longer than c_i ($1 \leq i \leq n$), then the derivations from $t\langle\pi, \rho\rangle$ are not longer than $g_{\pi,t}(c_1, \dots, c_n)$.

Following Section 2.5, we have a predicate P of $s \in T_\Sigma$, defined by (1) (dropping the quantification of s), and a predicate Q of $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$, defined by (2) (dropping the quantification of π). We have to prove (a) and (b), where (a) is the implication $Q((s_1, \dots, s_m)) \Rightarrow P(\sigma(s_1, \dots, s_m))$ and (b) is the implication $P(s_1)$ and $P(s_2)$ and ... and $P(s_m) \Rightarrow Q((s_1, \dots, s_m))$. We first prove (a) which is, roughly speaking, $(2) \Rightarrow (1)$.

(a) An arbitrary derivation from $q(\sigma(s_1, \dots, s_m), t_1, \dots, t_n)$ looks either like $q(\sigma(s_1, \dots, s_m), t_1, \dots, t_n) \xRightarrow{*} q(\sigma(s_1, \dots, s_m), t'_1, \dots, t'_n)$ with $t_i \xRightarrow{*} t'_i$ and then it is not longer than $\sum_{i=1}^n c_i$, or it looks like $q(\sigma(s_1, \dots, s_m), t_1, \dots, t_n) \xRightarrow{*} q(\sigma(s_1, \dots, s_m), t'_1, \dots, t'_n) \Rightarrow t\langle\pi, \rho'\rangle \xRightarrow{*} t'$ with $t_i \xRightarrow{*} t'_i$, t is a right-hand side of a (q, σ) -rule, $\pi = (s_1, \dots, s_m)$, and $\rho' = (t'_1, \dots, t'_n)$. Clearly, because $t_i \xRightarrow{*} t'_i$, the length of derivations from t'_i is also bounded by c_i . Hence, using (2), this derivation is not longer than $\sum_{i=1}^n c_i + 1 + g_{\pi,t}(c_1, \dots, c_n)$. Hence, as a rough bound for $s = \sigma(s_1, \dots, s_m)$ we can take $g_{q,s}(c_1, \dots, c_n) = c_1 + \dots + c_n + 1 + \sum_t g_{\pi,t}(c_1, \dots, c_n)$, where t ranges over the right-hand sides of rules with left-hand side $q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n)$. (We take $\sum_t$ rather than $\max_t$ for the sake of the proof of Theorem 3.25.)

Part (b) of the proof (the implication $(1) \Rightarrow (2)$) is proved by induction on the structure of t with respect to Definition 3.1.

(b) (i) If $t = y_j \in Y_n$, then let $g_{\pi,t}(c_1, \dots, c_n) = c_i$.

(ii) If $t = \delta(\xi_1, \dots, \xi_k)$ with $\delta \in \Delta_k$, $k \geq 1$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, then let $g_{\pi,t}(c_1, \dots, c_n) = \sum_{i=1}^k g_{\pi,\xi_i}(c_1, \dots, c_n)$; if $k = 0$, $g_{\pi,t}(c_1, \dots, c_n) = 1$.

(iii) If $t = q(x_i, \xi_1, \dots, \xi_k)$ with $q \in Q_{k+1}$, $k \geq 0$, $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, then using (1) for s_i , we can take $g_{\pi,t}(c_1, \dots, c_n) = g_{q,s_i}(g_{\pi,\xi_1}(c_1, \dots, c_n), \dots, g_{\pi,\xi_k}(c_1, \dots, c_n))$.

It is left to the reader to check that these g -functions satisfy the conditions. By the remarks in Section 2.5, (1) and (2) are now proved. Since every sentential form t can be written as $t'\langle\pi, (\)\rangle$ for some $t' \in \text{RHS}(Q, \Delta, m, 0)$, $m \geq 0$, $\pi \in T_\Sigma^m$, it follows immediately from (2) that the derivations starting from t have bounded length. ■

3.2. Context-free Tree Grammars

In Section 1 we have considered macro tree transducers as a combination of top-down tree transducers and context-free tree grammars (CFTG). Now we want to make obvious that the new model has at least the power of both of the latter ones. Furthermore, the behavior of a macro tree transducer can also be modelled

by a context-free tree grammar. For a precise definition of a context-free tree grammar (or macro grammar) and all other notations, see e.g., [20].

We have already seen (Proposition 3.6) that the class of top-down tree transductions is included in the class of macro tree transductions. On the other hand, the classes of tree languages generated by context-free tree grammars (*IOT*, *OIT*), are contained in the classes of output languages of macro tree transducers using the corresponding derivation mode. In fact, the languages generated by $G = (N, \Delta, S, R) \in CFTG$, where N and Δ denote the set of nonterminals and terminals, respectively, S is the initial nonterminal, and R denotes a finite set of rules, can be obtained from a macro tree transducer M , while supplying monadic trees $\sigma^n e$ (with σ of rank 1 and e of rank 0) as input and disallowing rules with e in the left-hand side. Thus, $\sigma^n e$ is a kind of dummy input, because in the choice of the applied rule there is no decision depending on the input symbol which is read. Thus, we can construct $M = (N, \{\sigma^{(1)}, e^{(0)}\}, \Delta, S, R') \in MT_a$, where R' is obtained from R as follows: if $A(y_1, \dots, y_n) \rightarrow t$ with $A \in N_n$ and $t \in T_{N \cup \Delta}(Y_n)$ is a rule in R then $A(\sigma(x_1), y_1, \dots, y_n) \rightarrow \psi(t)$ is a rule in R' , where $\psi: T_{N \cup \Delta}(Y_n) \rightarrow \text{RHS}(N, \Delta, 1, n)$ is defined by string manipulation: replace B by $B(x_1$ for every occurrence of a nonterminal B in t . For example, $\psi(g(B(a, A(g(b, c))), d)) = g(B(x_1, a, A(x_1, g(b, c))), d)$. Note that, if $A \in N_n$, then A is considered in M as a state of rank $n + 1$. There is no rule in R with left-hand side $A(e, y_1, \dots, y_n)$; reaching such construct, the derivation stops. Hence, to obtain all elements of the tree language μ -generated ($\mu \in \{\text{IO}, \text{OI}, \text{unr}\}$) by G , which is denoted by $L_\mu(G)$ [20], we need monadic input trees of arbitrary length. Since M provides the “same” derivations as G except that it “drags along” the $\sigma^n e$, it is intuitively clear that

$$L_\mu(G) = \bigcup \{L_\mu(M, S(\sigma^n e)) \mid n \in \mathbb{N}\}.$$

It should be obvious that this method can be used for every derivation mode and so we obtain the following result.

3.12. COROLLARY.

$$OIT \subseteq MT_{OI}(\{\sigma^n e \mid n \in \mathbb{N}\})$$

and

$$IOT \subseteq MT_{IO}(\{\sigma^n e \mid n \in \mathbb{N}\}).$$

Note that in the same way top-down tree transducers correspond to context-free tree grammars without parameters, i.e., to regular tree grammars. Vice versa, any macro tree transducer with $\Sigma = \{\sigma^{(1)}, e^{(0)}\}$ and no (q, e) -rule can be turned into a context-free tree grammar generating the output language of M .

The monadic input tree plays the role of a limiter (clock) for the length (“time”) of a derivation. Hence, it could be worthwhile to study macro tree transducers with monadic input trees $\sigma^n e$ for “time complexity” investigations on context-free tree grammars.

We have shown how to simulate a context-free tree grammar by a macro tree transducer. Actually, we can, conversely, simulate a macro tree transducer by a context-free tree grammar in the following sense: the μ -language ($\mu \in \{IO, OI, unr\}$) derivable from an initial sentential form $q^{\text{in}}(s)$ using an arbitrary macro tree transducer M can be obtained as the tree language μ -generated by a particular context-free tree grammar G_s , i.e., $L_\mu(M, q^{\text{in}}(s)) = L_\mu(G_s)$, as follows. For an $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$ and an input tree $s \in T_\Sigma$ we encode two pieces of information into the nonterminals of a context-free tree grammar $G_s = (N_s, \Delta, S_s, R_s)$: a state q in which an input tree s' is read. Hence, the set of nonterminals is defined by $N_s = \{\langle q, s' \rangle \mid q \in Q, s' \text{ is a subtree of } s\}$, where $\langle q, s' \rangle$ is of rank n if $q \in Q_{n+1}$. Let the initial non-terminal S_s be $\langle q^{\text{in}}, s \rangle$. In every rule

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \quad t \in \text{RHS}(Q, \Delta, m, n) \quad (*)$$

of R these two pieces of information are combined and (1) turns into a grammar rule as follows. For all subtrees s' of s , if s' is of the form $\sigma(s_1, \dots, s_m)$ with $m \geq 0$, then $\langle q, s' \rangle(y_1, \dots, y_n) \rightarrow (t)_\pi$ is in R_s , where π denotes $(s_1, \dots, s_m)$. The auxiliary function $(\)_\pi: \text{RHS}(Q, \Delta, m, n) \rightarrow T_{N_s \cup \Delta}(Y_n)$ is defined in an obvious way (using RHS to abbreviate $\text{RHS}(Q, \Delta, m, n)$): (i) for $j \in [n]$, $(y_j)_\pi = y_j$, (ii) for $\delta \in \Delta_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $(\delta(\xi_1, \dots, \xi_k))_\pi = \delta((\xi_1)_\pi, \dots, (\xi_k)_\pi)$, and (iii) for $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $(q(x_i, \xi_1, \dots, \xi_k))_\pi = \langle q, s_i \rangle((\xi_1)_\pi, \dots, (\xi_k)_\pi)$. Note that N_s and R_s are finite sets.

It should be obvious that the sentential forms appearing in the derivations of M with input s are in a 1-1 correspondence with the sentential forms in the derivations of the grammar G_s and that their notions of derivation step also correspond. Hence, both sets of derivations are the "same" and we obtain

$$L_\mu(M, q^{\text{in}}(s)) = L_\mu(G_s) \quad \text{with} \quad \mu \in \{IO, OI, unr\}.$$

Since $L_{IO}(G) \subseteq L_{OI}(G) = L_{unr}(G)$ for all $G \in CFTG$ [23], we obtain immediately the analogous result for the derivation relations of macro tree transducers.

3.13. COROLLARY. For all $M \in MT_a$,

$$\tau_{IO}(M) \subseteq \tau_{OI}(M) = \tau_{unr}(M).$$

Hence, $MT_{unr} = MT_{OI}$ and therefore we will not consider the unrestricted derivation mode any more.

3.3. Inductive Characterizations and Finiteness Properties

In this section we characterize the translation of a macro tree transducer in two different ways. Using these characterizations we can conclude a height restriction for the output trees in terms of the height of the input tree. Finally, we give an explicit bound on the length of derivations for a macro tree transducer.

For the proofs of composition and decomposition results in later sections an inductive characterization for the translations realized by macro tree transducers is

convenient. This characterization provides, for a sentential form t , the set of output trees to which the macro tree transducer translates t in an immediate fashion, i.e., without the use of derivations. Thus, we could write, for instance, in the binary number example, equations like

$$q^{\text{in}}(\overline{110})|_M = +(\exp(+ (0, 1)), \exp(+ (+(0, 1), 1))),$$

where $\overline{110}$ denotes the production tree of 110 and $q^{\text{in}}(\overline{110})|_M$ denotes the q^{in} -translation of $\overline{110}$. The rules in this example also turn into equations, similar to the semantic clauses of denotational semantics; e.g., rule (3): $q(\langle N \rightarrow N1 \rangle(x), y) \rightarrow +(\exp(y), q(x, +(y, 1)))$ turns into $q(\langle N \rightarrow N1 \rangle(s), t)|_M = +(\exp(t), q(s, +(t, 1))|_M)$ for all $s \in T_{\Sigma}$ and $t \in T_{\Delta}$. Now $q(\langle N \rightarrow N1 \rangle(s), t)|_M$ is the q -translation of $\langle N \rightarrow N1 \rangle(s)$ in the context t .

Since we have to distinguish between IO- and OI-derivation, we provide for both derivation modes inductive characterizations. Let us start with the IO-case. The definition is by simultaneous induction (see Sect. 2.5), with an inner induction on the structure of right-hand sides.

3.14. DEFINITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$:

(1) For all $s = \sigma(s_1, \dots, s_m) \in T_{\Sigma}$ with $m \geq 0$, $q \in Q_{n+1}$ with $n \geq 0$ and $\rho = (t_1, \dots, t_n) \in T_{\Delta}^n$, $q(s, t_1, \dots, t_n) \uparrow_M \subseteq T_{\Delta}$ is the set $\bigcup \{t \langle \uparrow, \pi, \rho \rangle_M | q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R\}$, where $\pi = (s_1, \dots, s_m)$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_{\Sigma}^m$, $t \in \text{RHS}(Q, \Delta, m, n)$, and $\rho = (t_1, \dots, t_n) \in T_{\Delta}^n$ with $m, n \geq 0$, $t \langle \uparrow, \pi, \rho \rangle_M \subseteq T_{\Delta}$ is defined inductively on the structure of t .

(i) For $y_j \in Y_n$, $y_j \langle \uparrow, \pi, \rho \rangle_M = \{t_j\}$.

(ii) For $\delta \in \Delta_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\delta(\xi_1, \dots, \xi_k) \langle \uparrow, \pi, \rho \rangle_M = \{\delta(\xi_1, \dots, \xi_k) | \xi_i \in \xi_i \langle \uparrow, \pi, \rho \rangle_M \text{ for all } i \in [k]\}$.

(iii) For $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $q(x_i, \xi_1, \dots, \xi_k) \langle \uparrow, \pi, \rho \rangle_M = \{t' \in T_{\Delta} | \text{there are } \zeta_1, \dots, \zeta_k \text{ such that } \zeta_i \in \xi_i \langle \uparrow, \pi, \rho \rangle_M \text{ for all } i \in [k] \text{ and } t' \in q(s_i, \zeta_1, \dots, \zeta_k) \uparrow_M\}$.

If M is understood from the context it is omitted.

Note that $q(s, t_1, \dots, t_n) \uparrow_M$ is always a finite set of trees: the set of possible q -translations of s in the context $t_1, \dots, t_n$. We prove that this characterization correctly models the IO-derivation of macro tree transducers.

3.15. THEOREM. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. For all $s \in T_{\Sigma}$, $q^{\text{in}}(s) \uparrow_M = L_{\text{IO}}(M, q^{\text{in}}(s))$.

Proof. To show the theorem we prove two statements by simultaneous induction. To illustrate this proof technique once more we work this out in detail:

(1) For all $s \in T_{\Sigma}$, $q \in Q_{n+1}$ with $n \geq 0$, and $t_1, \dots, t_n \in T_{\Delta}$, $q(s, t_1, \dots, t_n) \uparrow_M = L_{\text{IO}}(M, q(s, t_1, \dots, t_n))$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_{\Sigma}^m$ with $m \geq 0$, $t \in \text{RHS}(Q, \Delta, m, n) = \text{RHS}$ with $n \geq 0$, and $\rho = (t_1, \dots, t_n) \in T_{\Delta}^n$, $t \langle \uparrow, \pi, \rho \rangle_M = L_{\text{IO}}(M, t \langle \pi, \rho \rangle)$.

(a) We have to show that, if (2) holds for $\pi = (s_1, \dots, s_m)$, then (1) holds for $\sigma(s_1, \dots, s_m)$. Let $s = \sigma(s_1, \dots, s_m)$, $\pi = (s_1, \dots, s_m)$, and $\rho = (t_1, \dots, t_n)$. Then

$$\begin{aligned} q(s, t_1, \dots, t_n) \uparrow_M &= \bigcup \{ t \langle \uparrow, \pi, \rho \rangle_M \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R \} && \text{by Definition 3.14,} \\ &= \bigcup \{ L_{\text{IO}}(M, t \langle \pi, \rho \rangle) \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R \} && \text{by (2),} \\ &= L_{\text{IO}}(M, q(s, t_1, \dots, t_n)). && (*) \end{aligned}$$

(b) We have to show that, if (1) holds for $s_1, \dots, s_m$, then (2) holds for $(s_1, \dots, s_m)$. This is proved by induction on the structure of t . Let $\langle \uparrow \rangle$ abbreviate $\langle \uparrow, \pi, \rho \rangle_M$:

- (i) For $y_j \in Y_n$, $y_j \langle \uparrow \rangle = \{t_j\} = L_{\text{IO}}(M, t_j) = L_{\text{IO}}(M, y_j \langle \pi, \rho \rangle)$.
- (ii) For $\delta \in \Delta_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$,

$$\begin{aligned} \delta(\xi_1, \dots, \xi_k) \langle \uparrow \rangle &= \{ \delta(\zeta_1, \dots, \zeta_k) \mid \zeta_i \in \xi_i \langle \uparrow \rangle \text{ for all } i \in [k] \} && \text{by Definition 3.14,} \\ &= \{ \delta(\zeta_1, \dots, \zeta_k) \mid \zeta_i \in L_{\text{IO}}(M, \xi_i \langle \pi, \rho \rangle) \text{ for all } i \in [k] \} && \text{by I.H.,} \\ &= L_{\text{IO}}(M, \delta(\xi_1 \langle \pi, \rho \rangle, \dots, \xi_k \langle \pi, \rho \rangle)) \\ &= L_{\text{IO}}(M, \delta(\xi_1, \dots, \xi_k) \langle \pi, \rho \rangle) && \text{by an obvious fact for tree substitution. } (*) \end{aligned}$$

- (iii) For $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$,

$$\begin{aligned} q(x_i, \xi_1, \dots, \xi_k) \langle \uparrow \rangle &= \{ t' \in T_{\Delta} \mid \text{there are } \zeta_1, \dots, \zeta_k \text{ such that } \zeta_i \in \xi_i \langle \uparrow \rangle \text{ for all } i \in [k] \text{ and} \\ &\quad t' \in q(s_i, \zeta_1, \dots, \zeta_k) \uparrow_M \} && \text{by Definition 3.14,} \\ &= \{ t' \in T_{\Delta} \mid \text{there are } \zeta_1, \dots, \zeta_k \text{ such that } \zeta_i \in L_{\text{IO}}(M, \xi_i \langle \pi, \rho \rangle) \text{ for all} \\ &\quad i \in [k] \text{ and } t' \in L_{\text{IO}}(M, q(s_i, \zeta_1, \dots, \zeta_k)) \} && \text{by (1) and I.H.,} \\ &= L_{\text{IO}}(M, q(x_i, \xi_1, \dots, \xi_k) \langle \pi, \rho \rangle). && (*) \end{aligned}$$

At (*) we have used some obvious properties of L_{IO} , i.e., of IO-derivations. Recall from Section 2.5 that both statements are proved now, the latter of which, taking $q = q^{\text{in}}$ and $n = 0$, passes into the theorem. ■

We compute $q^{\text{in}}(f(a)) \uparrow_M$ for the transducer of Example 3.10 to make Definition 3.14 more vivid:

$$\begin{aligned} q^{\text{in}}(f(a)) \uparrow_M &= p(x, q^{\text{in}}(x)) \langle \uparrow, (a), () \rangle_M \\ &= \{ t \mid \text{there is a } \zeta \text{ in } q^{\text{in}}(x) \langle \uparrow, (a), () \rangle \text{ and } t \in p(a, \zeta) \uparrow_M \} \end{aligned}$$

$$\begin{aligned}
&= \{t \mid \text{there is a } \zeta \text{ in } \{e, c\} \text{ and } t \in p(a, \zeta) \uparrow_M\} \\
&= \{g(e, e), g(c, c)\},
\end{aligned}$$

because $p(a, e) \uparrow_M = g(y, y) \langle \uparrow, (), (e) \rangle_M = \{g(e, e)\}$ and $p(a, c) \uparrow_M = g(y, y) \langle \uparrow, (), (c) \rangle_M = \{g(c, c)\}$. Thus, $q^{\text{in}}(f(a)) \uparrow_M = L_{\text{IO}}(M, q^{\text{in}}(f(a)))$.

If the macro tree transducer M is deterministic (total deterministic), it is obvious that $q(s, t_1, \dots, t_n) \uparrow_M$ and $t \langle \uparrow, \pi, \rho \rangle_M$ are always sets with at most one symbol (singletons, respectively). Hence, we may consider $\uparrow_M$ and $\langle \uparrow, \pi, \rho \rangle_M$ as partial functions (total functions, respectively) in the customary way:

$$\uparrow_M: Q(T_\Sigma, T_A) \rightarrow T_A \quad \text{and} \quad \langle \uparrow, \pi, \rho \rangle_M: \text{RHS}(Q, A, m, n) \rightarrow T_A,$$

where $Q(T_\Sigma, T_A) = \{q(s, t_1, \dots, t_n) \mid q \in Q_{n+1} \text{ with } n \geq 0, s \in T_\Sigma, \text{ and } t_1, \dots, t_n \in T_A\}$.

Note that, since $T_a \subseteq MT_a$, Definition 3.14 also provides an inductive characterization for top-down tree transducers.

Now let us turn to the OI-characterization. It is defined in a way similar to the IO-case, but in an OI-derivation we must be aware of the fact that an “activated” state may contain sentential forms ξ in its arguments rather than only output trees. Since a whole tree language is represented by ξ , we define for $q \in Q_{n+1}$ with $n \geq 0$, $s \in T_\Sigma$, and tree languages $L_1, \dots, L_n$ over the output alphabet A , $q(s, L_1, \dots, L_n) \downarrow_M$ as the q -translation of s in the context $L_1, \dots, L_n$ using OI-derivation mode. In Example 3.10, we then compute, using rule (6): $p(f(a), L) \downarrow_M = p(x, d(y)) \langle \downarrow, (a), L \rangle_M$. If a formal parameter is projected, as in rule (4), a whole tree language is yielded, viz.,

$$p(a, L) \downarrow_M = g(y, y) \langle \downarrow, (), (L) \rangle_M = g(L, L),$$

where $g(L, L)$ is, by a standard definition, the set $\{g(t_1, t_2) \mid t_1, t_2 \in L\}$.

The definition is again done by simultaneous induction on s and $(s_1, \dots, s_m)$.

3.16. DEFINITION. Let $M = (Q, \Sigma, A, q^{\text{in}}, R) \in MT_a$:

- (1) For all $s = \sigma(s_1, \dots, s_m) \in T_\Sigma$ with $m \geq 0$, $q \in Q_{n+1}$ with $n \geq 0$, and $\rho = (L_1, \dots, L_n) \in \mathcal{P}(T_A)^n$, $q(s, L_1, \dots, L_n) \downarrow_M \subseteq T_A$ is the set $\bigcup \{t \langle \downarrow, \pi, \rho \rangle_M \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R\}$, where $\pi = (s_1, \dots, s_m)$.
- (2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$, $t \in \text{RHS}(Q, A, m, n) = \text{RHS}$ with $n \geq 0$, and $\rho = (L_1, \dots, L_n) \in \mathcal{P}(T_A)^n$, $t \langle \downarrow, \pi, \rho \rangle_M \subseteq T_A$ is defined by induction on the structure of t .

- (i) For $y_j \in Y_n$, $y_j \langle \downarrow, \pi, \rho \rangle_M = L_j$.
- (ii) For $\delta \in \Delta_k$ with $k \geq 0$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\delta(\xi_1, \dots, \xi_k) \langle \downarrow, \pi, \rho \rangle_M = \{\delta(\xi_1, \dots, \xi_k) \mid \xi_i \in \xi_i \langle \downarrow, \pi, \rho \rangle_M \text{ for all } i \in [k]\}$.
- (iii) For $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $q(x_i, \xi_1, \dots, \xi_k) \langle \downarrow, \pi, \rho \rangle_M = q(s_i, \xi_1 \langle \downarrow, \pi, \rho \rangle_M, \dots, \xi_k \langle \downarrow, \pi, \rho \rangle_M) \downarrow_M$.

If M is understood from the context it is omitted.

We can prove that this definition corresponds to the OI-derivation of a macro tree transducer.

3.17. THEOREM. *Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. For all $s \in T_\Sigma$, $q^{\text{in}}(s) \downarrow_M = L_{\text{OI}}(M, q^{\text{in}}(s))$.*

Proof. The proof is again done by two statements:

(1) For all $s \in T_\Sigma$, $q \in Q_{n+1}$ with $n \geq 0$, and $t_1, \dots, t_n \in \text{SF}$, $q(s, L_{\text{OI}}(M, t_1), \dots, L_{\text{OI}}(M, t_n)) \downarrow_M = L_{\text{OI}}(M, q(s, t_1, \dots, t_n))$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$, $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$, and $\rho = (t_1, \dots, t_n) \in \text{SF}^n$, $t \langle \downarrow, \pi, \rho' \rangle_M = L_{\text{OI}}(M, t \langle \pi, \rho \rangle)$, where $\rho' = (L_{\text{OI}}(M, t_1), \dots, L_{\text{OI}}(M, t_n))$.

(1) and (2) can easily be proved by simultaneous induction using obvious properties of L_{OI} , i.e., of OI-derivations. The proof of (1) $\Rightarrow$ (2) is again by induction on t and we only point out that part in which t has the form $q(x_i, \xi_1, \dots, \xi_k)$, i.e., case (iii) of Definition 3.1:

$$\begin{aligned}
 & q(x_i, \xi_1, \dots, \xi_k) \langle \downarrow, \pi, \rho' \rangle \\
 &= q(s_i, \xi_1 \langle \downarrow, \pi, \rho' \rangle_M, \dots, \xi_k \langle \downarrow, \pi, \rho' \rangle_M) \downarrow_M \\
 &= q(s_i, L_{\text{OI}}(M, \xi_1 \langle \pi, \rho \rangle), \dots, L_{\text{OI}}(M, \xi_k \langle \pi, \rho \rangle)) \downarrow_M \quad \text{by I.H.,} \\
 &= L_{\text{OI}}(M, q(s_i, \xi_1 \langle \pi, \rho \rangle, \dots, \xi_k \langle \pi, \rho \rangle)) \quad \text{by (1),} \\
 &= L_{\text{OI}}(M, q(x_i, \xi_1, \dots, \xi_k) \langle \pi, \rho \rangle).
 \end{aligned}$$

For $q = q^{\text{in}}$ and $n = 0$ (1) turns into the statement of the theorem. ■

We use again the transducer M of Example 3.10 to illustrate Definition 3.16 and compute $q^{\text{in}}(f(a)) \downarrow_M$. For the computation we need $q^{\text{in}}(x) \langle \downarrow, (a), () \rangle_M = q^{\text{in}}(a) \downarrow = e \langle \downarrow, (), () \rangle \cup c \langle \downarrow, (), () \rangle = \{e, c\}$. Then

$$\begin{aligned}
 q^{\text{in}}(f(a)) \downarrow &= p(x, q^{\text{in}}(x)) \langle \downarrow, (a), () \rangle \\
 &= p(a, q^{\text{in}}(x) \langle \downarrow, (a), () \rangle) \downarrow \\
 &= p(a, \{e, c\}) \downarrow \\
 &= g(y, y) \langle \downarrow, (), (\{e, c\}) \rangle \\
 &= \{g(\xi_1, \xi_2) \mid \xi_1, \xi_2 \in y \langle \downarrow, (), (\{e, c\}) \rangle\} \\
 &= \{g(\xi_1, \xi_2) \mid \xi_1, \xi_2 \in \{e, c\}\} \\
 &= \{g(e, e), g(e, c), g(c, e), g(c, c)\}.
 \end{aligned}$$

Thus, indeed, $q^{\text{in}}(f(a)) \downarrow_M = L_{\text{OI}}(M, q^{\text{in}}(f(a)))$.

For a total deterministic or deterministic macro tree transducer M it is obvious that $L_{OI}(M, q(s, t_1, \dots, t_n))$ (where $q \in Q_{n+1}$ with $n \geq 0$ and $t_1, \dots, t_n \in SF$) and $L_{OI}(M, t\langle\pi, \rho\rangle)$ where $t \in \text{RHS}(Q, \Delta, m, n)$ with $m, n \geq 0$, $\pi \in T_\Sigma^m$, and $\rho \in SF^n$) denote singletons or sets with at most one output tree, respectively. Hence, we may consider $\downarrow_M$ and $\langle\downarrow, \pi, \rho\rangle_M$ again as a total or partial function, respectively, where ρ is a sequence of tree languages that are singletons or sets with at most one element, respectively.

The previous inductive characterization, for the OI-case, is closely related to the least fixed-point characterization of OI context-free tree grammars as proved in [11], where the rules of the grammar, viewed as a set of equations, are solved in the algebra of functions $\mathcal{P}(T_\Delta)^n \rightarrow \mathcal{P}(T_\Delta)$. Note, in fact, that in Definition 3.16, for fixed q and s , $\lambda L_1 \dots L_n. q(s, L_1, \dots, L_n) \downarrow_M$ is such a function. In what follows we want to give another inductive characterization of the translation of a macro tree transducer, which is closely related to the least fixed-point characterization of the (IO and) OI context-free tree grammars as shown in [32, 20], where the equations are solved in the algebra of tree languages $\mathcal{P}(T_\Delta(Y))$, with substitution as basic operation. This inductive characterization will be called symbolic, because it treats the formal parameters $y_1, y_2, \dots$, as output symbols of rank 0.

The subsequent definition is meant to work for both IO and OI. Let μ be a derivation mode. Intuitively, the symbolic inductive characterization, denoted by $M_\mu(q(s, y_1, \dots, y_n))$ for $q \in Q_{n+1}$ and $s \in T_\Sigma$ and $M_{\mu, \pi}(t)$ for $t \in \text{RHS}$ and $\pi \in T_\Sigma^m$, should again model the μ -language derivable from $q(s, y_1, \dots, y_n)$ and $t\langle\pi\rangle$, respectively, where $t\langle\pi\rangle$ is $t[x_1/\pi(1), \dots, x_m/\pi(m)]$. For the definition of tree language substitution, see Section 2.3.

3.18. DEFINITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$ and $\mu \in \{\text{IO}, \text{OI}\}$:

(1) For all $s = \sigma(s_1, \dots, s_m) \in T_\Sigma$ with $m \geq 0$ and $q \in Q_{n+1}$ with $n \geq 0$, $M_\mu(q(s, y_1, \dots, y_n)) \subseteq T_\Delta(Y_n)$ is the set $\bigcup \{M_{\mu, \pi}(t) \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R\}$, where $\pi = (s_1, \dots, s_m)$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$ and $t \in \text{RHS}(Q, \Delta, m, n) = \text{RHS}$ with $n \geq 0$, $M_{\mu, \pi}(t) \subseteq T_\Delta(Y_n)$ is defined by induction on the structure of t .

(i) For $y_j \in Y_n$, $M_{\mu, \pi}(y_j) = \{y_j\}$.

(ii) For $\delta \in \Delta_k$, with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $M_{\mu, \pi}(\delta(\xi_1, \dots, \xi_k)) = \{\delta(y_1, \dots, y_k)\} \leftarrow_\mu (M_{\mu, \pi}(\xi_1), \dots, M_{\mu, \pi}(\xi_k))$.

(iii) For $q \in Q_{k+1}$ with $k \geq 0$, $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $M_{\mu, \pi}(q(x_i, \xi_1, \dots, \xi_k)) = M_\mu(q(s_i, y_1, \dots, y_k)) \leftarrow_\mu (M_{\mu, \pi}(\xi_1), \dots, M_{\mu, \pi}(\xi_k))$.

Whenever μ is understood from the context we write M and M_π rather than M_μ and $M_{\mu, \pi}$, respectively (thus, in particular, M denotes both the macro tree transducer and its effect on sentential forms $q(s, y_1, \dots, y_n)$).

The next two lemmata show that the symbolic characterization of the previous definition fulfills what we have in mind.

3.19. LEMMA. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$:

(1) For all $s \in T_\Sigma$, $q \in Q_{n+1}$ with $n \geq 0$, and $t_1, \dots, t_n \in T_\Delta$, $q(s, t_1, \dots, t_n) \uparrow_M = M_{\text{IO}}(q(s, y_1, \dots, y_n)) \xleftarrow{\text{IO}} (\{t_1\}, \dots, \{t_n\})$.

(2) For all $\pi \in T_\Sigma^m$ with $m \geq 0$, $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$, and $\rho = (t_1, \dots, t_n) \in T_\Delta^n$,

$$t \langle \uparrow, \pi, \rho \rangle_M = M_{\text{IO}, \pi}(t) \xleftarrow{\text{IO}} (\{t_1\}, \dots, \{t_n\}).$$

Proof. The two statements can be shown easily by simultaneous induction. The implication (1) $\Rightarrow$ (2) is proved, as usual, by induction on t ; in the cases (ii) $t = \delta(\xi_1, \dots, \xi_k)$ and (iii) $t = q(x_i, \xi_1, \dots, \xi_k)$ the associativity of IO-substitution of tree languages is needed, which is valid in this case (cf. Lemma 2.4.3 of [20]) but not true in general. ■

3.20. LEMMA. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$:

(1) For all $s \in T_\Sigma$, $q \in Q_{n+1}$ with $n \geq 0$, and $L_1, \dots, L_n \in \mathcal{P}(T_\Delta)$, $q(s, L_1, \dots, L_n) \downarrow_M = M_{\text{OI}}(q(s, y_1, \dots, y_n)) \xleftarrow{\text{OI}} (L_1, \dots, L_n)$.

(2) For all $\pi \in T_\Sigma^m$ with $m \geq 0$, $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$, and $\rho \in \mathcal{P}(T_\Delta)^n$, $t \langle \downarrow, \pi, \rho \rangle_M = M_{\text{OI}, \pi}(t) \xleftarrow{\text{OI}} \rho$.

Proof. The proof is straightforward using simultaneous induction and the fact that OI-substitution of tree languages is associative. ■

These lemmata and the remarks following Theorem 3.15 and Theorem 3.17 justify to consider M_μ and $M_{\mu, \pi}$ ($\mu \in \{\text{IO}, \text{OI}\}$) as total or partial functions for total deterministic or deterministic macro tree transducers, respectively. Note that in this case the tree language substitution in Definition 3.18(2) can be replaced by ordinary tree substitution.

As an immediate consequence of Lemma 3.19 and Lemma 3.20 we obtain the validness of the symbolic inductive characterization.

3.21. THEOREM. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. For all $s \in T_\Sigma$, $M_{\text{IO}}(q^{\text{in}}(s)) = L_{\text{IO}}(M, q^{\text{in}}(s))$ and $M_{\text{OI}}(q^{\text{in}}(s)) = L_{\text{OI}}(M, q^{\text{in}}(s))$.

Proof. Theorems 3.15 and 3.17, Lemmata 3.19 and 3.20. ■

For technical reasons it is convenient to have besides the usual μ -translation ($\mu \in \{\text{IO}, \text{OI}\}$) of a macro tree transducer M (see Definition 3.5) the symbolic μ -translation of M available, starting from an arbitrary state of M . Of course, this symbolic translation contains trees with formal parameters and is defined by means of the symbolic inductive characterization of M .

3.22. DEFINITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$, $q \in Q_{n+1}$ with $n \geq 0$, and $\mu \in \{\text{IO}, \text{OI}\}$. The symbolic μ -translation of M starting from q , denoted by $\tau_{q, \mu}(M)$, is the set of pairs $\{(s, t) \in T_\Sigma \times T_\Delta(Y_n) \mid s \in T_\Sigma, t \in M_\mu(q(s, y_1, \dots, y_n))\}$.

Note that $\tau_\mu(M) = \tau_{p,\mu}(M)$ for any derivation mode μ , where $p = q^{\text{in}}$. It is easy to see that the symbolic μ -translation of a macro tree transducer M starting from an arbitrary state can be obtained as the usual translation of a slight modification M' of M : M' should treat the formal parameters of M as output symbols of rank 0.

3.23. PROPOSITION. *Let $M_1 = (Q, \Sigma, \Delta, q_1^{\text{in}}, R) \in MT_a$, $q \in Q_{n+1}$ with $n \geq 0$, and $\mu \in \{\text{IO}, \text{OI}\}$. There is an $M_2 \in MT_a$ such that $\tau_{q,\mu}(M_1) = \tau_\mu(M_2)$.*

Proof. Construct $M_2 = (Q_1 \cup \{q_2^{\text{in}}\}, \Sigma, \Delta \cup Y_n, q_2^{\text{in}}, R_2)$, where R_2 consists of all productions of R_1 with bars on the y s and of $\{q_2^{\text{in}}(\sigma(x_1, \dots, x_m)) \rightarrow t \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ in } R_1\}$. M_2 uses the formal parameters $\bar{y}_1, \bar{y}_2, \dots$, rather than $y_1, y_2, \dots$. The formal proof is left to the reader. ■

From Definition 3.18 and Proposition 3.23 it can be seen that for macro tree transducers there is not much difference between the constants in Δ_0 and the formal parameters in Y . Thus, one may imagine a slight variation of the macro tree transducer in which this difference has completely disappeared, as follows. The rules of such an $M = (Q, \Sigma, \Delta, q^{\text{in}}, R)$ are of the form $q(\sigma(x_1, \dots, x_m)) \rightarrow t$, where t is in the smallest set rhs such that

- (i) $\Delta_0 \subseteq \text{rhs}$,
- (ii) if $q \in Q$, $a_1, \dots, a_k \in \Delta_0$, and $\xi_1, \dots, \xi_k \in \text{rhs}$, then $q(x_i, a_1 \leftarrow \xi_1, \dots, a_k \leftarrow \xi_k) \in \text{rhs}$ for every $i \in [m]$, and
- (iii) if $\delta \in \Delta_k$ with $k \geq 1$ and $\xi_1, \dots, \xi_k \in \text{rhs}$, then $\delta(\xi_1, \dots, \xi_k) \in \text{rhs}$.

Thus, all elements of Δ_0 can be used as formal parameters and in a call of a state q some of them are given actual values. A definition of the translation realized by such a transducer can be given analogously to Definition 3.18, with $M_\mu(q(s))$ and $M_{\mu,\pi}(t)$ subsets of T_A ; e.g., $M_{\mu,\pi}(q(x_i, a_1 \leftarrow \xi_1, \dots, a_k \leftarrow \xi_k)) = M_\mu(q(s_i)) \leftarrow_{\mu,A} (M_{\mu,\pi}(\xi_1), \dots, M_{\mu,\pi}(\xi_k))$, where $\leftarrow_{\mu,A}$ denotes μ -substitution of tree languages, using $a_1, \dots, a_k$ as parameters. It is left to the reader to show, using Proposition 3.23, that this variant of macro tree transducer has the same power as the ordinary one.

Actually, we have discussed the above variation of macro tree transducer in order to claim that the macro tree transducer was invented by Irons in his original paper on syntax-directed translation [28]. The meta-language defined, more or less formally, in [28] is very close to the 1-state total deterministic macro tree transducer in the above variation. The reader should compare the above definition of rhs to clauses 1, 2, and 3 in [28]. Thus, the original formalism of Irons already accounted for the use of context and is therefore closer to the macro tree transducer than to the top-down tree transducer or the generalized syntax-directed translation scheme [2, 3].

We end this section with two theorems. Using the symbolic inductive characterization of Definition 3.18, we prove two properties of translations of macro tree transducers: the height of an output tree grows at most exponentially with respect to the height of the input tree and there is a double exponential bound on the length of derivations for a given input tree.

3.24. THEOREM. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. There is a constant c such that for all $s \in T_\Sigma$ and $t \in T_\Delta$, if $(s, t) \in \tau_{\text{OI}}(M)$, then $\text{height}(t) \leq c^{\text{height}(s)}$.

Proof. Let c be the maximal height of the right-hand sides of rules in R . For this c the estimation of the theorem is fulfilled. This is shown by proving the following two statements by simultaneous induction on s and π .

(1) For all $s \in T_\Sigma$, $q \in Q_{n+1}$ with $n \geq 0$, and $t \in T_\Delta(Y_n)$, if $t \in M_{\text{OI}}(q(s, y_1, \dots, y_n))$, then $h(t) \leq c^{h(s)}$.

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$, $t' \in \text{RHS}(Q, \Delta, m, n) = \text{RHS}$ with $n \geq 0$, and $t \in T_\Delta(Y_n)$, if $t \in M_{\text{OI}, \pi}(t')$, then $h(t) \leq h(t') \cdot c^{mx}$, where $mx = \max\{0, h(s_1), \dots, h(s_m)\}$.

Note that the 0 has only a technical reason: for $m=0$, $\max\{0, h(s_1), \dots, h(s_m)\} = 0$.

(a) $t \in M_{\text{OI}}(q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n))$ implies $t \in M_{\text{OI}, \pi}(t')$ for some right-hand side t' (with $\pi = (s_1, \dots, s_m)$); using (2) we can compute

$$h(t) \leq h(t') \cdot c^{mx} \leq c \cdot c^{mx} = c^{mx+1} = c^{h(s)}.$$

(b) The implication (1) $\Rightarrow$ (2) is proved as usual by induction on the structure of t' ,

(i) For $t' = y_j \in Y_n$: $t = y_j$ and $h(t) = 1 \leq 1 \cdot 1 = 1$ and $1 = h(t') \cdot c^0 \leq h(t') \cdot c^{mx}$.

(ii) For $t' = \delta(\xi_1, \dots, \xi_k)$ with $\delta \in \Delta_k$, $k \geq 0$, $\xi_1, \dots, \xi_k \in \text{RHS}$, and $t \in M_{\text{OI}, \pi}(t')$; and $k=0$: $t = \delta$ and (2) is obtained by the same argumentation as in case (i), and $k \geq 1$:

$$\begin{aligned} h(t) &\leq 1 + \max_{1 \leq i \leq k} \max\{h(u) | u \in M_{\text{OI}, \pi}(\xi_i)\} \\ &\leq 1 + \max_{1 \leq i \leq k} (h(\xi_i) \cdot c^{mx}) && \text{by I.H.,} \\ &= 1 + \left(\max_{1 \leq i \leq k} h(\xi_i) \right) \cdot c^{mx} \\ &\leq \left(1 + \max_{1 \leq i \leq k} h(\xi_i) \right) \cdot c^{mx} \\ &= h(\delta(\xi_1, \dots, \xi_k)) \cdot c^{mx} = h(t') \cdot c^{mx}. \end{aligned}$$

(iii) For $t' = q(x_i, \xi_1, \dots, \xi_k)$ with $q \in Q_{k+1}$, $k \geq 0$, $x_i \in X_m$, $\xi_1, \dots, \xi_k \in \text{RHS}$, and $t \in M_{\text{OI}, \pi}(t')$ and $k=0$:

$$\begin{aligned} h(t) &\leq \max\{h(u) | u \in M_{\text{OI}}(q(s_i))\} \\ &\leq c^{h(s_i)} && \text{by (1),} \\ &\leq h(q(x_i)) \cdot c^{mx} = h(t') \cdot c^{mx}, \end{aligned}$$

and $k \geq 1$:

$$\begin{aligned}
 h(t) &\leq \max \{h(u) \mid u \in M_{\text{OI}}(q(s_i, y_1, \dots, y_n))\} + \max_{1 \leq r \leq k} h(\xi_r) \cdot c^{mx} && \text{by I.H.,} \\
 &\leq c^{h(s_i)} + \max_{1 \leq r \leq k} h(\xi_r) \cdot c^{mx} && \text{by (1),} \\
 &\leq \left(1 + \max_{1 \leq r \leq k} h(\xi_r)\right) \cdot c^{mx} \\
 &= h(q(x_i, \xi_1, \dots, \xi_k)) \cdot c^{mx} = h(t') \cdot c^{mx}.
 \end{aligned}$$

Since $\tau_{\text{IO}}(M) \subseteq \tau_{\text{OI}}(M)$ for every $M \in MT_a$ (by Corollary 3.13), the height of an output tree IO-derivable is restricted by the same bound given in Theorem 3.24 for OI-derivable trees. Note that for a top-down tree transducer the height of an output tree is linearly bounded with respect to the height of the input tree, i.e., for every $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in T_a$ there is a constant c such that, if $(s, t) \in \tau(M)$, then $\text{height}(t) \leq c \cdot \text{height}(s)$.

We have already shown that there are no arbitrarily long derivations for a macro tree transducer and a given input tree (Theorem 3.11). Now we provide a concrete upper bound for this length which naturally depends on the height of the input tree.

3.25. THEOREM. *Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. There are constant c and d such that for all $s \in T_\Sigma$ the derivations of M starting with $q^{\text{in}}(s)$ are not longer than $d^{c \cdot \text{height}(s)}$.*

Proof. The idea is to construct a total deterministic macro tree transducer M' such that an output tree interpreted in $\mathbb{N}$ provides an upper bound for the length of a derivation of M from an input tree. Construct $M' = (Q, \Sigma, \Omega, q^{\text{in}}, R') \in D_t MT_a$ with $\Omega_0 = \{1\}$, $\Omega_2 = \{+\}$, and $\Omega_k = \{+\}$ for every $k \geq 1$ such that $\Delta_k \neq \emptyset$. The rules are constructed as follows. If $t_1, \dots, t_r$ with $r \geq 1$ are the right-hand sides of the (q, σ) -rules in R , with $m, n \geq 0$, then the rule

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow y_1 + \dots + y_n + 1 + t'_1 + \dots + t'_r \quad (*)$$

is in R' , where t'_i is obtained from t_i by changing every element of Δ_0 into 1 and every element of Δ_k ($k \geq 1$) into $+$. (Note that, for convenience, the $+\in\Omega_2$ is written in infix notation in rule $(*)$, omitting parentheses.) Now recall that the proof of Theorem 3.11 provides an inductive definition of the g -functions that give upper bounds for the length of derivations. For $t \in T_\Omega(Y_n)$, let $t_{\mathbb{N}}: \mathbb{N}^n \rightarrow \mathbb{N}$ denote the derived function of t in the algebra $\langle \mathbb{N}; +, 1 \rangle$, as usual (see, e.g., [27]). Thus, for $t \in T_\Omega$, $t_{\mathbb{N}}$ is the value of t in $\mathbb{N}$. We now claim, that M' computes these g -functions; formally,

- (1) for all $s \in T_\Sigma$ and $q \in Q_{n+1}$ with $n \geq 0$, $M'_{\text{IO}}(q(s, y_1, \dots, y_n))_{\mathbb{N}} = g_{q,s}$ and
- (2) for all $\pi \in T_\Sigma^m$ with $m \geq 0$ and $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$ $M'_{\text{IO}, \pi}(t')_{\mathbb{N}} = g_{\pi, t}$, where t' is defined as above.

The proof of (1) and (2) is straightforward from the proof of Theorem 3.11, using simultaneous induction on s and π . Hence, for all $s \in T_{\Sigma}$ and for $q = q^{\text{in}}$, $M'_{\text{IO}}(g(s))_{\mathbb{N}} = g_{q,s} \in \mathbb{N}$ is an upper bound for the length of a derivation from $q(s)$ using M , for short $ld_M(q(s))$.

Now we consider the shape of the tree computed by M' . It is obvious that for all $t \in T_{\Omega}$, $t_{\mathbb{N}}$ is equal to the length of $\text{yield}(t)$. Thus, if d is the maximal rank of $+$, then $t_{\mathbb{N}} \leq d^{\text{height}(t)}$. By Theorem 3.24 there is a constant c such that, if M' translates s into t , then $\text{height}(t) \leq c^{\text{height}(s)}$. Thus, we obtain the statement of the theorem: for all $s \in T_{\Sigma}$, $ld_M(q(s)) \leq g_{q,s} = t_{\mathbb{N}} \leq d^{\text{height}(t)} \leq d^{c^{\text{height}(s)}}$, where $t = M'_{\text{IO}}(q(s))$ and $q = q^{\text{in}}$. ■

Hence, the length of derivations is double exponential in the height of the input tree. This cannot be improved. In fact, consider $M = (Q, \Sigma, \Sigma, q^{\text{in}}, R) \in D_t MT_a$ with $Q = \{q^{\text{in}}, q^{(2)}\}$, $\Sigma = \{\sigma^{(2)}, a^{(0)}\}$ and rules

$$\begin{aligned} q^{\text{in}}(\sigma(x_1, x_2)) &\rightarrow q(x_2, q(x_1, a)), \\ q^{\text{in}}(a) &\rightarrow \sigma(a, a), \\ q(\sigma(x_1, x_2), y_1) &\rightarrow q(x_2, q(x_1, y_1)) \\ q(a, y_1) &\rightarrow \sigma(y_1, y_1). \end{aligned}$$

Let $t(n)$ be the fully balanced binary tree of height $n + 1$, i.e., $t(0) = a$ and $t(n + 1) = \sigma(t(n), t(n))$. It is easy to show by induction on $n \geq 1$ that there is a derivation of length at least 2^{2^n} such that $q(t(n), y_1) \xRightarrow{*} t(2^n)[a/y_1]$. Hence, there is a derivation $q^{\text{in}}(t(n)) \xRightarrow{*} t(2^n)$ with at least 2^{2^n} steps.

4. TOTAL DETERMINISTIC MACRO TREE TRANSDUCERS

To comprehend the power of macro tree transducers we investigate its simplest subclass first, viz., the class of total deterministic macro tree transducers, and show that there is no difference between the translation classes obtained with the various derivation modes. The class YIELD is modelled by $LD_t MT_a$ (Theorem 4.6) and the closure under right-composition with total deterministic top-down tree transducers is shown (Theorem 4.12). Finally, it is proved that the addition of regular look-ahead does not increase the transformational power of $D_t MT_a$ (Theorem 4.21).

4.1. Derivation Relation and Examples

Considering the inductive IO- and OI-characterization of Definitions 3.14 and 3.16 for a total deterministic macro tree transducer $M = (Q, \Sigma, A, q^{\text{in}}, R)$, $\uparrow_M$ and $\downarrow_M$ can be understood as total functions each delivering one output tree (as observed in Sect. 3). In Theorems 3.15 and 3.17 we proved that $\tau_{\text{IO}}(M) = \{(s, q^{\text{in}}(s)\uparrow_M) \mid s \in T_{\Sigma}\}$ and $\tau_{\text{OI}}(M) = \{(s, q^{\text{in}}(s)\downarrow_M) \mid s \in T_{\Sigma}\}$, respectively. Using $\tau_{\text{IO}}(M) \subseteq \tau_{\text{OI}}(M)$ from Corollary 3.13, we can conclude $\tau_{\text{IO}}(M) = \tau_{\text{OI}}(M)$ immediately. This proves the following result.

4.1. THEOREM. $D_t MT_{IO} = D_t MT_{OI} = D_t MT_{unr}$.

Since there is no difference between the various translation classes of total deterministic macro tree transducers, we will denote this class by $D_t MT$ from now on.

We provide three examples to become acquainted with the translation class $D_t MT$.

4.2. EXAMPLE. A *forest* is a finite sequence $[t_1, \dots, t_n]$ of trees with $n \geq 0$. Thus, a tree $\sigma(s_1, \dots, s_m)$ may be viewed as a father σ with the forest of sons $[s_1, \dots, s_m]$. Every forest (with $n \geq 1$) has a unique decomposition $[\sigma(s_1, \dots, s_m), t_2, \dots, t_n]$ with $m \geq 0$. The *binary tree* $\text{bin}(f)$ encoding a forest f is defined, as usual:

$$\begin{aligned} \text{for } n \geq 1: \text{bin}([\sigma(s_1, \dots, s_m), t_2, \dots, t_n]) &= \sigma'(\text{bin}([s_1, \dots, s_m]), \text{bin}([t_2, \dots, t_n])) \\ \text{for } n = 0: \text{bin}([\]) &= \text{nil}, \end{aligned}$$

where σ' and nil have rank 2 and 0, respectively.

For a ranked alphabet Σ we construct a total deterministic macro tree transducer M which computes for every tree $s \in T_\Sigma$ the corresponding binary tree $\text{bin}([s])$. Let $M = (\{q^{\text{in}}, p^{(2)}\}, \Sigma, \Sigma', q^{\text{in}}, R) \in D_t MT_a$, where $\Sigma' = \{\sigma' \mid \sigma \in \Sigma\} \cup \{\text{nil}\}$ and R contains for every $\sigma \in \Sigma_m$ with $m \geq 0$ the rules:

$$\begin{aligned} q^{\text{in}}(\sigma(x_1, \dots, x_m)) &\rightarrow \sigma'(p(x_1, p(x_2, \dots, p(x_m, \text{nil}) \dots)), \text{nil}) \\ p(\sigma(x_1, \dots, x_m), y) &\rightarrow \sigma'(p(x_1, p(x_2, \dots, p(x_m, \text{nil}) \dots)), y), \end{aligned}$$

where for $m = 0$, $p(x_1, p(x_2, \dots, p(x_m, \text{nil}) \dots))$ denotes nil . (Note that, since Σ is finite, R is finite too.)

To prove $q^{\text{in}}(s) \uparrow_M = \text{bin}([s])$ for all $s \in T_\Sigma$ it suffices to prove the following two statements by simultaneous induction on s and π :

(1) For all $s \in T_\Sigma$, $t_1, \dots, t_n \in T_\Sigma$ with $n \geq 0$,

$$p(s, \text{bin}([t_1, \dots, t_n])) \uparrow_M = \text{bin}([s, t_1, \dots, t_n]).$$

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$ with $m \geq 0$,

$$p(x_1, \dots, p(x_m, \text{nil}) \dots) \langle \uparrow, \pi, () \rangle_M = \text{bin}([s_1, \dots, s_m]).$$

This easy proof is left to the reader.

4.3. EXAMPLE. Consider $M = (\{q^{\text{in}}, p^{(2)}\}, \{f^{(1)}, a^{(0)}\}, \{f^{(1)}, a^{(0)}\}, q^{\text{in}}, R) \in D_t MT_a$, where R contains the rules

$$\begin{aligned} q^{\text{in}}(f(x)) &\rightarrow p(x, p(x, a)), \\ q^{\text{in}}(a) &\rightarrow f(a), \\ p(f(x), y) &\rightarrow p(x, p(x, y)), \\ p(a, y) &\rightarrow f(y). \end{aligned}$$

Then a derivation with input f^3a may look like

$$\begin{aligned}
 q^{\text{in}}(f^3a) &\Rightarrow p(f^2a, q^{\text{in}}(f^2a)) \\
 &\stackrel{2}{\Rightarrow} p(f^2a, p(fa, p(a, q^{\text{in}}(a)))) \\
 &\Rightarrow p(f^2a, p(fa, p(a, fa))) \\
 &\Rightarrow p(f^2a, p(fa, f^2a)) \\
 &\stackrel{3}{\Rightarrow} p(f^2a, f^4a) \\
 &\stackrel{7}{\Rightarrow} f^8a.
 \end{aligned}$$

By straightforward induction on n we can prove:

for all $n, m \geq 0$, $p(f^n a, f^m a) \uparrow_M = f^r a$ with $r = 2^n + m$.

Hence, for all $n \geq 0$: $q^{\text{in}}(f^{n+1}a) \uparrow_M = f^{2^{n+1}}a$. The second production implies $q^{\text{in}}(a) \uparrow_M = fa$. Thus, the translation of M is $\{(f^n a, f^{2^n} a) | n \geq 0\}$.

Note that the translation in Example 4.3 is not contained in T , because the height of the output tree is not linearly bounded with respect to the height of the input tree (cf. remark following Theorem 3.24). Hence, total deterministic macro tree transducers are more powerful than total deterministic top-down transducers (cf. Proposition 3.6).

4.4. PROPOSITION. $D_i T \subsetneq D_i MT$.

Proof. For the M of Example 4.3, $\tau(M) \in D_i MT - T$. ■

4.5. EXAMPLE. We want to model an arbitrary element of YIELD (cf. Sect. 2.3) by a linear total deterministic macro tree transducer. Let $f: \Sigma_0 \rightarrow T_A(Y)$ and $\text{YIELD}_f \in \text{YIELD}$. Then there is an $n \geq 0$ such that $f(\Sigma_0) \subseteq T_A(Y_n)$ and hence $\text{YIELD}_f: T_\Sigma \rightarrow T_A(Y_n)$. Abbreviate YIELD_f by Y_f . To construct $M \in LD_i MT_a$ we first provide a linear deterministic macro tree transducer M' , which by application of the construction of Proposition 3.23, changes into the desired M .

Let $M' = (\{q^{\text{in}}, p\}, \Sigma, A, q^{\text{in}}, R') \in \text{LDMT}_a$, where p is a state of rank $n+1$, and R' contains the following rules:

For all $\sigma \in \Sigma_m$ with $m \geq 1$, if $m \leq n$, then

$$p(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow p(x_1, p(x_2, y_1, \dots, y_n), \dots, p(x_m, y_1, \dots, y_n), y_m, \dots, y_n)$$

and if $m > n$, then

$$p(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow p(x_1, p(x_2, y_1, \dots, y_n), \dots, p(x_{n+1}, y_1, \dots, y_n))$$

is in R' , and for all $a \in \Sigma_0$, $p(a, y_1, \dots, y_n) \rightarrow f(a)$ is in R' .

Then, for all $s \in T_{\Sigma}$, $Y_f(s) = M'_{IO}(p(s, y_1, \dots, y_n))$. This can be shown by straightforward induction on the structure of s . In the induction step we use, in the case $m \leq n$,

$$Y_f(s_1)[Y_f(s_2), \dots, Y_f(s_m)] = Y_f(s_1)[Y_f(s_2), \dots, Y_f(s_m), y_m, \dots, y_n]$$

and, in the case $m > n$,

$$Y_f(s_1)[Y_f(s_2), \dots, Y_f(s_m)] = Y_f(s_1)[Y_f(s_2), \dots, Y_f(s_{n+1})].$$

Hence, if we consider Y_f as a relation in the usual way, $Y_f = \tau_{p, IO}(M')$ (cf. Definition 3.22). Applying the construction of Proposition 3.23, we obtain an $M \in LD_i MT_a$ such that $\tau_{p, IO}(M') = \tau_{IO}(M)$.

The construction of Example 4.5 gives rise to the following result.

4.6. THEOREM. $YIELD \subsetneq LD_i MT$.

Proof. The validity of the inclusion is shown by the construction in Example 4.5. The inclusion is proper because, clearly, the identity mapping is not in $YIELD$. ■

We finally provide a technical lemma for total deterministic macro tree transducers, viz., that M_{π} is a homomorphism with respect to tree substitution (for M_{π} , cf. Definition 3.18).

4.7. LEMMA. Let $M = (Q, \Sigma, \Delta, q^{in}, R) \in D_i MT_a$ and $a_1, \dots, a_r \in A_0$. Let $m, n \geq 0$ and $r \geq 1$. For all $t, t_1, \dots, t_r \in RHS(Q, \Delta, m, n)$ and $\pi \in T_{\Sigma}^n$

$$M_{\pi}(t[a_1/t_1, \dots, a_r/t_r]) = M_{\pi}(t)[a_1/M_{\pi}(t_1), \dots, a_r/M_{\pi}(t_r)].$$

Proof. First note that $t[a_1/t_1, \dots, a_r/t_r]$ is again in $RHS(Q, \Delta, m, n)$. The statement of the lemma is proved by straightforward induction on the structure of t with respect to Definition 3.1. ■

4.2. Decomposition and Composition of $D_i MT$

In this section we mention the main decomposition result for $D_i MT$, which is already known from the literature. A more general result is presented in Lemma 5.5. As an immediate consequence of this decomposition we obtain the closure of $D_i MT$ under left-composition with $D_i T$. But macro tree transducers are not closed under composition with themselves: iterated composition leads to an infinite hierarchy. Nevertheless, the closure under right-composition with $D_i T$ is valid.

The main decomposition result for macro tree transducers is the following [15, 8].

4.8. THEOREM. $D_i MT = D_i T \circ YIELD$.

From the point of view of automata theory, this means that the substitution mechanism for parameters, which is involved in the derivation of a total deter-

ministic macro tree transducer M and which is the main extra feature with respect to top-down tree transducers, can be characterized by YIELD. The equality justifies to first collect all substitutions of actual parameters into formal parameters, done during a derivation of M , and then treat them by one YIELD-function after a symbolic translation. The symbolic translation, done by a total deterministic top-down tree transducer, only executes the “textual” substitution and memorizes the parameter substitution by special symbols, e.g., sub, in the output tree. The sub-symbols are then interpreted by YIELD as substitution of subtrees.

Interpreting macro tree transducers as a model for denotational semantics, Theorem 4.8 points out that denotational semantics with context can be simulated by the composition of a denotational semantics without context (realized by D, T) and a YIELD-function.

As an example we decompose the macro tree transducer of Example 4.2.

4.9. EXAMPLE. Construct $M' = (\{q^{\text{in}}, p\}, \Sigma, \Delta, q^{\text{in}}, R') \in D_i T_a$ with $\Delta = \{\delta_3^{(3)}, \delta_2^{(2)}, \bar{y}^{(0)}, \text{nil}^{(0)}\} \cup \{\sigma'' \mid \sigma \in \Sigma\}$, where the σ'' are of rank 0, and R' contains for all $\sigma \in \Sigma_m$ with $m \geq 0$ the rules:

$$\begin{aligned} q^{\text{in}}(\sigma(x_1, \dots, x_m)) &\rightarrow \delta_3(\sigma'', \delta_2(p(x_1), \delta_2(p(x_2), \dots, \delta_2(p(x_m), \text{nil}) \dots)), \text{nil}), \\ p(\sigma(x_1, \dots, x_m)) &\rightarrow \delta_3(\sigma'', \delta_2(p(x_1), \delta_2(p(x_2), \dots, \delta_2(p(x_m), \text{nil}) \dots)), \bar{y}). \end{aligned}$$

Let $f: \Delta_0 \rightarrow T_{\Sigma}(Y_2)$ be defined by $f(\text{nil}) = \text{nil}$, $f(\bar{y}) = y_1$ and $f(\sigma'') = \sigma'(y_1, y_2)$. Then $\tau(M) = \tau(M') \circ \text{YIELD}_f$. Instead of giving a formal proof we compute $\text{YIELD}_f(M'(q^{\text{in}}(h(g(a), b))))$:

$$\begin{aligned} q^{\text{in}}(h(g(a), b)) &\Rightarrow \delta_3(h'', \delta_2(p(g(a)), \delta_2(p(b), \text{nil})), \text{nil}) \\ &\stackrel{2}{\Rightarrow} \delta_3(h'', \delta_2(\delta_3(g'', \delta_2(p(a), \text{nil}), \bar{y}), \delta_2(\delta_3(b'', \text{nil}, \bar{y}), \text{nil})), \text{nil}) \\ &\Rightarrow \delta_3(h'', \delta_2(\delta_3(g'', \delta_2(\delta_3(a'', \text{nil}), \bar{y}), \text{nil}), \bar{y}), \bar{y}), \delta_2(\delta_3(b'', \text{nil}, \bar{y}), \text{nil})), \text{nil}). \end{aligned}$$

Thus $q^{\text{in}}(h(g(a), b)) \stackrel{*}{\Rightarrow} t$, where $t = \delta_3(h'', t_3, \text{nil})$, $t_3 = \delta_2(\delta_3(g'', t_1, \bar{y}), t_2, \text{nil})$, $t_1 = \delta_2(\delta_3(a'', \text{nil}, \bar{y}), \text{nil})$, and $t_2 = \delta_2(\delta_3(b'', \text{nil}, \bar{y}), \text{nil})$.

We now compute $Y_f(t)$ as follows:

$$\begin{aligned} Y_f(t_1) &= Y_f(\delta_3(a'', \text{nil}, \bar{y})) [Y_f(\text{nil})] \\ &= (Y_f(a'') [Y_f(\text{nil}), Y_f(\bar{y})]) [Y_f(\text{nil})] = a'(\text{nil}, \text{nil}), \\ Y_f(t_2) &= b'(\text{nil}, \text{nil}), \\ Y_f(t_3) &= Y_f(\delta_3(g'', t_1, \bar{y})) [Y_f(t_2), Y_f(\text{nil})] \\ &= (Y_f(g'') [Y_f(t_1), Y_f(\bar{y})]) [Y_f(t_2), \text{nil}] \\ &= g'(a'(\text{nil}, \text{nil}), y_1) [b'(\text{nil}, \text{nil})] \\ &= g'(a'(\text{nil}, \text{nil}), b'(\text{nil}, \text{nil})). \end{aligned}$$

Finally, $Y_f(t) = Y_f(h'')[Y_f(t_3), \text{nil}] = h'(g'(a'(\text{nil}, \text{nil}), b'(\text{nil}, \text{nil})), \text{nil})$, the binary representation of $h(g(a), b)$.

Since it is well known that $D_i T$ is closed under composition [34] we obtain the closure of $D_i MT$ under left-composition with $D_i T$.

4.10. COROLLARY. $D_i T \circ D_i MT \subseteq D_i MT$.

Proof.

$$\begin{aligned} D_i T \circ D_i MT &= D_i T \circ D_i T \circ \text{YIELD} && \text{by Theorem 4.8,} \\ &= D_i T \circ \text{YIELD} && \text{by Theorem 2 of [34],} \\ &= D_i MT && \text{by Theorem 4.8. } \blacksquare \end{aligned}$$

We now show that $D_i MT$ is also closed under right-composition with $D_i T$. In the sequel we informally describe the construction used in the proof of this result.

Let $M_1 = (Q, \Sigma, \Delta, q^{\text{in}}, R_1) \in D_i MT_a$ and $M_2 = (P, \Delta, \Omega, p^{\text{in}}, R_2) \in D_i T_a$ and construct $M_3 = (Q_3, \Sigma, \Omega, q_3^{\text{in}}, R_3) \in D_i MT_a$ such that $\tau(M_1) \circ \tau(M_2) = \tau(M_3)$. The first idea one has in mind is the usual product construction for transducers (cf. Proof of Theorem 2 of [34]). Take as set of states Q_3 the cartesian product of Q and P and translate the right-hand sides of the rules in R_1 starting in every state of P using the rules in R_2 .

If, for instance, R_1 contains the rule $q_1(f(x), y) \rightarrow g(a, q_2(x))$ and R_2 the rules $p_1(g(x_1, x_2)) \rightarrow h(p_2(x_1), p_3(x_2))$ and $p_2(a) \rightarrow b$, the right-hand side $g(a, q_2(x))$ is translated in the state p_1 as follows: $p_1(g(a, q_2(x))) \Rightarrow h(p_2(a), p_3(q_2(x))) \Rightarrow h(b, p_3(q_2, x))$. Note that, using this extended derivation relation, a state $p \in P$ reads a tree in $T_A(Q(X))$ rather than only an element of T_A . The states p_3 and q_2 are combined into $\langle q_2, p_3 \rangle$ and this tuple contains the information that the q_2 -translation of an input subtree (substituted for x in the argument of $\langle q_2, p_3 \rangle$) has to be translated in state p_3 of M_2 . Hence, we obtain the rule $\langle q_1, p_1 \rangle(f(x), y) \rightarrow h(b, \langle q_2, p_3 \rangle(x))$ for R_3 .

But how shall we treat the parameters involved in the rules of R_1 ? Consider, for instance, the rule $q_1(f(x), y_1) \rightarrow g(y_1, q_2(x))$ of R_1 and let P contain the same rules as above. The above described method would deliver the rule $\langle q_1, p_1 \rangle(f(x), y_1) \rightarrow h(p_2(y_1), \langle q_2, p_3 \rangle(x))$ for R_3 . Of course, since a formal parameter never may occur in the first argument position of a state, this is an inadmissible rule. To solve this problem, we will compute for each parameter every translation by M_2 immediately when applying a rule of R_1 . For this purpose we replace each parameter y_j by new parameters $y_{j,i}$ representing the p_i -translation of y_j . Then we may replace constructs like $p_i(y_j)$ by $y_{j,i}$. Hence, if $P = \{p_1, p_2, p_3\}$, we obtain the following rule: $\langle q_1, p_1 \rangle(f(x), y_{1,1}, y_{1,2}, y_{1,3}) \rightarrow h(y_{1,2}, \langle q_2, p_3 \rangle(x))$. Now, what about a rule like $q_1(f(x), y_1) \rightarrow g(y_1, q_3(x, g(a, y_1)))$? By computing for $g(a, y_1)$ every translation by M_2 we obtain the rule $\langle q_1, p_1 \rangle(f(x), y_{1,1}, y_{1,2}, y_{1,3}) \rightarrow h(y_{1,2}, \langle q_3, p_3 \rangle(x, h(b, y_{1,3}), t_2, t_3))$, where t_2 is the p_2 -translation of $g(a, y_1)$ and t_3 its p_3 -translation. Similar

constructions (to deal with parameters) are given in [35, 11]. (In fact, in [23] a similar technique is used to prove that indexed languages are contained in OI-macro languages.)

Before presenting the formal construction, we give a simple example, which should give an idea of the possible use of the result.

4.11. EXAMPLE. Consider the transducer M of Example 3.9. To obtain a total deterministic macro tree transducer, we complete M by adding a new output symbol $\text{error}^{(0)}$ to $\mathcal{A}$ and the rules $q^{\text{in}}(\langle N \rightarrow N0 \rangle(x)) \rightarrow \text{error}$, $q^{\text{in}}(\langle N \rightarrow N1 \rangle(x)) \rightarrow \text{error}$, $q^{\text{in}}(\langle N \rightarrow 1 \rangle) \rightarrow \text{error}$, and $q(\langle S \rightarrow N \rangle(x), y) \rightarrow \text{error}$. The error in an output tree of the translation of M indicates that an inadmissible tree has been supplied to the input.

A binary number n , presented by a production tree pt_n , is translated into a tree t_n , which can be interpreted by a virtual machine VM as the decimal value $d(n)$ of n . Now suppose that a usual stack machine SM is at our disposal, which is able to push 0 or 1 onto the top of the stack (PUSH0, PUSH1), to replace the top element n by 2^n (EXP) and to replace the two topmost elements by their sum (ADD). An implementation of the virtual machine on the stack machine can be described by a total deterministic top-down tree transducer. The implementation $N = (\{v\}, \mathcal{A}, \Omega, v, R_N) \in D_t T_a$ is given by $\Omega_0 = \{\text{PUSH0}, \text{PUSH1}, \text{ADD}, \text{EXP}, \text{ERROR}, ;\}$ and $\Omega_2 = \{c\}$. We write $t_1 t_2$ rather than $c(t_1, t_2)$ because in this example we intend c to be interpreted as concatenation. R_N contains the rules

$$\begin{aligned} v(+ (x_1, x_2)) &\rightarrow v(x_1); v(x_2); \text{ADD}, \\ v(\text{exp}(x_1)) &\rightarrow v(x_1); \text{EXP}, \\ v(0) &\rightarrow \text{PUSH0}, \\ v(1) &\rightarrow \text{PUSH1}, \\ v(\text{error}) &\rightarrow \text{ERROR}. \end{aligned}$$

Now we obtain an assembler for the stack machine that uses trees like pt_n as input only and produces an assembler program ap_n , by composing the transducer M (which can be considered as an assembler for the virtual machine) and the implementation N , cf. Fig. 4. Applying the above described method we obtain the following total deterministic macro tree transducer $M_3 = (Q_3, \Sigma, \Omega, \langle q^{\text{in}}, v \rangle, R_3)$, where $Q_3 = \{\langle q^{\text{in}}, v \rangle^{(1)}, \langle q, v \rangle^{(2)}\}$ and R_3 contains the rules:

$$\langle q^{\text{in}}, v \rangle(\langle S \rightarrow N \rangle(x)) \rightarrow \langle q, v \rangle(x, \text{PUSH0}),$$

where PUSH0 is the v -translation of 0,

$$\langle q, v \rangle(\langle N \rightarrow N0 \rangle(x), y_v) \rightarrow \langle q, v \rangle(x, y_v; \text{PUSH1}; \text{ADD}),$$

$$\langle q, v \rangle(\langle N \rightarrow N1 \rangle(x), y_v) \rightarrow y_v; \text{EXP}; \langle q, v \rangle(x, y_v; \text{PUSH1}; \text{ADD}); \text{ADD},$$

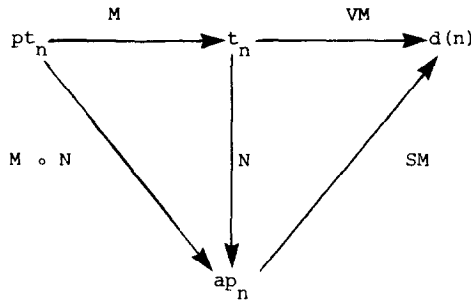


FIG. 4. Implementation of a virtual machine.

$$\langle q, v \rangle (\langle N \rightarrow 1 \rangle (x), y_v) \rightarrow y_v; \text{EXP},$$

$$\langle q^{\text{in}}, v \rangle (\sigma(x)) \rightarrow \text{ERROR} \quad \text{for } \sigma \in \{ \langle N \rightarrow N0 \rangle, \langle N \rightarrow N1 \rangle, \langle N \rightarrow 1 \rangle \},$$

$$\langle q, v \rangle (\langle S \rightarrow N \rangle (x), y_v) \rightarrow \text{ERROR}.$$

Note that, during a derivation of M_3 , y_v contains the v -translation of the value of y during the corresponding derivation of M . For $\langle S \rightarrow N \rangle (\langle N \rightarrow N0 \rangle \langle N \rightarrow N1 \rangle (\langle N \rightarrow 1 \rangle))$ the assembler program is computed in Fig. 5.

If the assembler program ap_n is executed on the stack machine, starting with the empty stack, the decimal value $d(n)$ of the input tree is computed in the top position of the stack.

4.12. THEOREM. $D_i MT \circ D_i T = D_i MT$.

Proof. Since $D_i MT \subseteq D_i MT \circ D_i T$ is obvious we have to show that $D_i MT \circ D_i T \subseteq D_i MT$. Let $M_1 = (Q, \Sigma, \Delta, q^{\text{in}}, R_1) \in D_i MT_a$ and $M_2 = (P, \Delta, \Omega, p^{\text{in}}, R_2) \in$

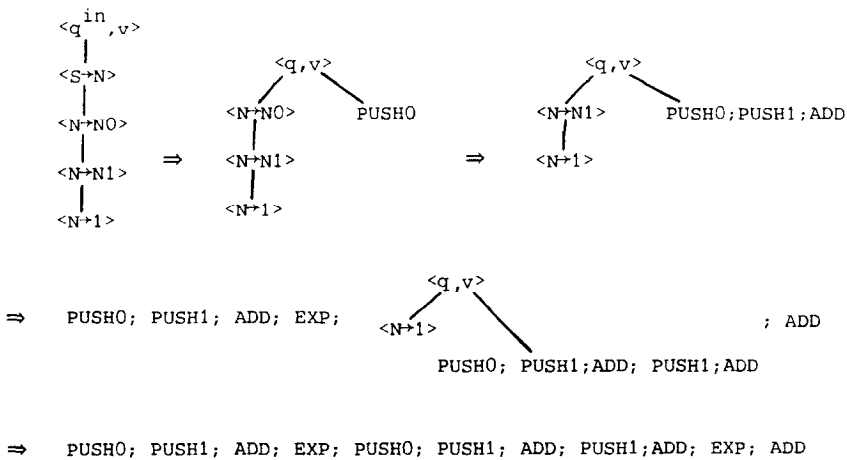


FIG. 5. Assembling of a production tree.

$D_t T_a$. Assume $P = \{p_1, \dots, p_r\}$. Construct $M_3 = (Q_3, \Sigma, \Omega, \langle q^{\text{in}}, p^{\text{in}} \rangle, R_3) \in D_t MT_a$ with $Q_3 = \{\langle q, p \rangle \mid q \in Q_{n+1} \text{ and } p \in P\}$ and R contains the following rules:

If $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t$ is in R_1 , then for all i ($1 \leq i \leq r$) the rule $\langle q, p_i \rangle (\sigma(x_1, \dots, x_m), y_{\langle 1,1 \rangle}, \dots, y_{\langle 1,r \rangle}, \dots, y_{\langle n,1 \rangle}, \dots, y_{\langle n,r \rangle}) \rightarrow \text{tr}(m, n, i)(t)$ is in R_3 . (Note that, formally, $\langle i, j \rangle$ denotes $r(i-1) + j$ for all $i, j \geq 1$.)

For all $m, n \geq 0$ and $1 \leq i \leq r$, $\text{tr}(m, n, i)$ is a total function of type $\text{RHS}(Q, \Delta, m, n) \rightarrow \text{RHS}(Q_3, \Omega, m, r \cdot n)$, which provides the p_i -translation of a right-hand side using the rules in R_2 . It is defined by induction on the tree in $\text{RHS}(Q, \Delta, m, n)$. We abbreviate $\text{tr}(m, n, i)$ and $\text{RHS}(Q, \Delta, m, n)$ by tr_i and RHS , respectively:

- (i) For $j \in [n]$, $\text{tr}_i(y_j) = y_{\langle j, i \rangle}$.
- (ii) For $\delta \in \Delta_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$ and the rule $p_i(\delta(x_1, \dots, x_k)) \rightarrow s$ in R_2 , $\text{tr}_i(\delta(\xi_1, \dots, \xi_k)) = s'$, where s' is obtained from s by replacing all constructs $p_u(x_v)$ by $\text{tr}_u(\xi_v)$, more precisely,

$$s' = s[p_1(x_1)/\text{tr}_1(\xi_1), \dots, p_r(x_1)/\text{tr}_r(\xi_1), \dots, p_1(x_k)/\text{tr}_1(\xi_k), \dots, p_r(x_k)/\text{tr}_r(\xi_k)].$$

- (iii) For $q \in Q$ of rank $k+1$ ($k \geq 0$), $\xi_1, \dots, \xi_k \in \text{RHS}$, and $x_j \in X_m$,

$$\text{tr}_i(q(x_j, \xi_1, \dots, \xi_k)) = \langle q, p_i \rangle(x_j, \text{tr}_1(\xi_1), \dots, \text{tr}_r(\xi_1), \dots, \text{tr}_1(\xi_k), \dots, \text{tr}_r(\xi_k)).$$

Since for every pair $q \in Q$ and $\sigma \in \Sigma$ exactly one right-hand side t is provided by R_1 and $\text{tr}(m, n, i)$ is a total function, M_3 is total deterministic.

To prove $\tau(M_1) \circ \tau(M_2) = \tau(M_3)$ it suffices to show

$$\text{for all } s \in T_\Sigma, \quad p^{\text{in}}(q^{\text{in}}(s) \uparrow_{M_1}) \uparrow_{M_2} = \langle q^{\text{in}}, p^{\text{in}} \rangle(s) \uparrow_{M_3}. \quad (*)$$

Note that $\uparrow_M$ (and $\langle \uparrow, \pi, \rho \rangle_M$) are viewed as total functions as discussed in Section 3. We use the characterization of Definition 3.14 to prove the composition result, because it turns out to be more intuitive and easier to handle than the symbolic characterization of Definition 3.18. First we need a technical statement which extends case (2)(ii) of Definition 3.14:

$$\begin{aligned} &\text{For all } t \in T_\Delta(Y_k) \text{ with } k \geq 0, \xi_1, \dots, \xi_k \in \text{RHS}(Q, \Delta, m, n) \text{ with } m, n \geq 0, \text{ and} \\ &\text{all } \pi \in T_\Sigma^m, \rho \in T_\Delta^n, \\ &(t[\xi_1, \dots, \xi_k]) \langle \uparrow, \pi, \rho \rangle_{M_1} = t[\xi_1 \langle \uparrow, \pi, \rho \rangle_{M_1}, \dots, \xi_k \langle \uparrow, \pi, \rho \rangle_{M_1}]. \end{aligned} \quad (**)$$

The proof of (**) is done by straightforward induction on the structure of t .

As usual, we prove (*) by proving the following two statements by simultaneous induction on s and π :

- (1) For all $s \in T_\Sigma$, $q \in Q$ with rank $n+1$ ($n \geq 0$), $p_i \in P$ and $t_1, \dots, t_n \in T_\Delta$, $p_i(q(s, t_1, \dots, t_n) \uparrow_{M_1}) \uparrow_{M_2} = \langle q, p_i \rangle(s, p_1(t_1) \uparrow_{M_2}, \dots, p_r(t_1) \uparrow_{M_2}, \dots, p_1(t_n) \uparrow_{M_2}, \dots, p_r(t_n) \uparrow_{M_2}) \uparrow_{M_3}$.
- (2) For all $\pi \in T_\Sigma^m$ with $m \geq 0$, $t \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$, all $i \in [r]$, and $\rho = (t_1, \dots, t_n) \in T_\Delta^n$, $p_i(t \langle \uparrow, \pi, \rho \rangle_{M_1}) \uparrow_{M_2} = (\text{tr}_i(t)) \langle \uparrow, \pi, \rho' \rangle_{M_3}$, where $\rho' = (p_1(t_1) \uparrow_{M_2}, \dots, p_r(t_1) \uparrow_{M_2}, \dots, p_1(t_n) \uparrow_{M_2}, \dots, p_r(t_n) \uparrow_{M_2})$ and tr_i abbreviates $\text{tr}(m, n, i)$.

Using (**) and Definition 3.14 repeatedly, the implications (a) and (b) can be proved easily. We only work out that case of (b) in which t is of the form $\delta(\xi_1, \dots, \xi_k)$ for $\delta \in \Delta_k$, $k \geq 0$, and $\xi_1, \dots, \xi_k \in \text{RHS}(Q, \Delta, m, n)$. For $s \in T_Q(P(X_k))$ and trees $t_{u,v}$ with $1 \leq u \leq r$ and $1 \leq v \leq k$ we abbreviate $s[p_1(x_1)/t_{1,1}, \dots, p_r(x_r)/t_{r,1}, \dots, p_1(x_k)/t_{1,k}, \dots, p_r(x_k)/t_{r,k}]$ by $s[p_u(x_v)/t_{u,v}]$. Let $p_i(\delta(x_1, \dots, x_k)) \rightarrow s$ be the (p_i, δ) -rule of R_2 . For $\pi \in T_{\Sigma}^m$, $\rho \in T_{\Delta}^n$, $\rho' \in T_{\Omega}^n$ (given as in (2)), and $\langle M_1 \rangle$ as abbreviation for $\langle \uparrow, \pi, \rho \rangle_{M_1}$ we obtain

$$\begin{aligned}
 & p_i(\delta(\xi_1, \dots, \xi_k) \langle M_1 \rangle) \uparrow_{M_2} \\
 &= p_i(\delta(\xi_1 \langle M_1 \rangle, \dots, \xi_k \langle M_1 \rangle)) \uparrow_{M_2} && \text{by Definition 3.14(2),} \\
 &= s \langle \uparrow, \bar{\pi}, () \rangle_{M_2}, \text{ where } \bar{\pi} = (\xi_1 \langle M_1 \rangle, \dots, \xi_k \langle M_1 \rangle) && \text{by Definition 3.14(1),} \\
 &= s[p_u(x_v)/p_u(x_v) \langle \uparrow, \bar{\pi}, () \rangle_{M_2}] && \text{by (9),} \\
 &= s[p_u(x_v)/p_u(\xi_v \langle M_1 \rangle) \uparrow_{M_2}] && \text{by Definition 3.14(2),} \\
 &= s[p_u(x_v)/(\text{tr}_u(\xi_v)) \langle \uparrow, \pi, \rho' \rangle_{M_3}] && \text{by I.H.,} \\
 &= (s[p_u(x_v)/\text{tr}_u(\xi_v)]) \langle \uparrow, \pi, \rho' \rangle_{M_3} && \text{by (9),} \\
 &= (\text{tr}_i(\delta(\xi_1, \dots, \xi_k))) \langle \uparrow, \pi, \rho' \rangle_{M_3} && \text{by definition of tr}_i.
 \end{aligned}$$

For $q = q^{\text{in}}$, (1) turns into (*). ■

Now we obtain a characterization of the n -fold composition of $D_i MT$.

4.13. COROLLARY. For all $n \geq 1$, $D_i MT^n = D_i T \circ \text{YIELD}^n$.

Proof. The proof is straightforward by induction on n , using Theorems 4.8 and 4.12. ■

In [10] (total deterministic) n -level top-down tree transducers, denoted by n -DT, are introduced. They correspond to the n - λ schemes in [9] in the same way as macro tree transducers correspond to context-free tree grammars. In fact, macro tree transducers and context-free tree grammars are 1-DT and 1- λ schemes, respectively. Furthermore, in [10] it is proved that n -DT = $D_i T \circ \text{YIELD}^n$. Hence, the n -fold composition of total deterministic macro tree transducers is characterized by the class of n -level top-down tree transducers.

In [16] it is shown that, by Corollary 4.13, $D_i MT^n \subseteq AG^{n+1}$ where AG is the class of attribute grammars viewed as tree transducers (note that $AG \subseteq D_i MT$, see [8, 24, 16] and Sect. 1). In fact, both $D_i T$ and YIELD are included in AG .

The natural question whether $D_i MT$ is closed under composition can be answered negatively. Moreover, composition leads to an infinite, strict hierarchy. To show this we give an upper bound on the height of output trees derivable by the n -fold composition of MT_a and provide for every $n \geq 1$ a translation R_{n+1} realizable by an $(n+1)$ -fold composition of $D_i MT_a$ but not by an n -fold composition of MT_a .

4.14. LEMMA. Let $n \geq 1$ and $R \in (MT_{IO} \cup MT_{OI})^n$. There is a constant c such that for all $(s, t) \in R$,

$$\begin{array}{c} \nearrow^n 2^{c \cdot \text{height}(s)} \\ \searrow^n \dots \\ \text{height}(t) \leq 2 \end{array}$$

Proof. Since $R \in (MT_{IO} \cup MT_{OI})^n$, there are $M_1, \dots, M_n \in MT_a$ such that, if $(s, t) \in R$, then $(s, t) \in \tau_{OI}(M_1) \circ \dots \circ \tau_{OI}(M_n)$ (cf. Corollary 3.13). Let c_i ($1 \leq i \leq n$) be the constant associated with M_i by Theorem 3.24. We may assume that $c_i \geq 2$. Applying Theorem 3.24 repeatedly, we obtain,

$$\begin{array}{c} c_n^{h(s)} \\ \dots \\ h(t) \leq c_1 \end{array}$$

We can prove by induction on n that for all $n \geq 1$ and $c_1, \dots, c_n \geq 2$ there is a $c \geq 1$ such that for all $k \geq 1$:

$$\begin{array}{c} c_n^k \\ \dots \leq \nearrow^n 2^{c \cdot k} \\ \searrow^n \dots \\ c_1 \quad 2 \end{array} \quad \blacksquare$$

Define the translation R_n for $n \geq 1$ in the obvious way as follows: Let $\Sigma = \{f^{(1)}, a^{(0)}\}$ be a ranked alphabet and abbreviate

$$\begin{array}{c} \nearrow^n 2^k \\ \searrow^n \dots \\ 2 \end{array}$$

by $\exp(n, k)$. Then $R_n = \{(f^k a, f^m a) | k \geq 0, m = \exp(n, k)\}$.

4.15. LEMMA. For all $n \geq 1$, $R_{n+1} \in D_i MT^{n+1} - (MT_{IO} \cup MT_{OI})^n$.

Proof. Compose the transducer M of Example 4.3 $n+1$ times with itself. It is obvious that $R_{n+1} = \tau(M)^{n+1}$, hence $R_{n+1} \in D_i MT^{n+1}$. Suppose $R_{n+1} \in (MT_{IO} \cup MT_{OI})^n$. By Lemma 4.14 there is a constant c such that for all $(s, t) = (f^k a, f^{\exp(n+1, k)} a) \in R_{n+1}$, $\text{height}(t) \leq \exp(n, c \cdot \text{height}(s))$, i.e., for all $k \geq 0$, $\exp(n+1, k) + 1 \leq \exp(n, c \cdot (k+1))$. This is a contradiction for big k , thus, $R_{n+1} \notin (MT_{IO} \cup MT_{OI})^n$. $\blacksquare$

We obtain the composition hierarchy for macro tree transducers.

4.16. THEOREM. For all $n \geq 1$, $D_i MT^n \subsetneq D_i MT^{n+1}$ and the same holds for the composition of the classes DMT_μ , ${}_i MT_\mu$, and MT_μ with $\mu \in \{IO, OI\}$.

Proof. Since $D_i MT$ is contained in DMT_μ and ${}_i MT_\mu$ and the latter classes are contained in MT_μ , the theorem follows immediately from Lemma 4.15. ■

In [9] it is shown that even $\text{out}(D_i MT^n) \subsetneq \text{out}(D_i MT^{n+1})$. In fact, he shows this for $D_i T \circ \text{YIELD}^n$, see Corollary 4.13.

4.3. Total Deterministic Macro Tree Transducers with Regular Look-ahead

To be able to compare macro tree transducers with other tree transducers we add the usual look-ahead ability to the transducer model [14]: a rule in an ordinary macro tree transducer can be applied only if each of the substituted input trees is an element of a recognizable tree language that is specified by a total deterministic bottom-up finite tree automaton (cf. Sect. 2.4). In this section we show that this feature of "regular look-ahead" does not enlarge the class of translations realized by total deterministic macro tree transducers. Note that for top-down tree transducers a larger class is obtained by this addition [14].

4.17. DEFINITION. A macro tree transducer with regular look-ahead $M = (Q, P, \Sigma, \Delta, q^{\text{in}}, R, \delta)$ consists of

1. a total deterministic bottom-up finite tree automaton (P, Σ, δ) ; P is the set of look-ahead states; and
2. a macro tree transducer $(Q, \Sigma, \Delta, q^{\text{in}}, R)$, where the rules are now of the form

$$(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle), \quad (*)$$

with $m, n \geq 0$, $q \in Q_{n+1}$, $\sigma \in \Sigma_m$, $t \in \text{RHS}(Q, \Delta, m, n)$, and $p_1, \dots, p_m \in P$.

A macro tree transducer with regular look-ahead is *deterministic (total)* if for every $q \in Q$, $\sigma \in \Sigma$, and $p_1, \dots, p_m \in P$ with $m \geq 0$ there is at most (at least) one rule of the form $(*)$ in R .

If X is a class of macro tree transducers then X^R denotes the corresponding class of macro tree transducers with regular look-ahead and similarly for their classes of translations.

4.18. DEFINITION. Let $M = (Q, P, \Sigma, \Delta, q^{\text{in}}, R, \delta) \in MT_a^R$. The derivation relations for M are defined in the same way as for ordinary macro tree transducers, except that a rule $(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle)$ in R can be applied only if for all substituted input trees $s_1, \dots, s_m \in T_\Sigma$, $\delta(s_i) = p_i$ ($1 \leq i \leq m$) holds (cf. Definition 3.4).

It can easily be shown that for the particular case of a top-down tree transducer with regular look-ahead, our definition is equivalent to the one in [14].

4.19. REMARK. Similarly as for the usual macro tree transducer we can define an inductive and a symbolic inductive characterization of the translation realized by an

$M \in MT_a^R$. We have to redefine in Definitions 3.14, 3.16, and 3.18 only the first case; thus, 3.14(1) should be $q(\sigma(s_1, \dots, s_m), t_1, \dots, t_n) \uparrow_M = \bigcup \{ t \langle \uparrow, \pi, \rho \rangle_M \mid (q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle) \text{ is in } R \text{ and } \delta(s_i) = p_i \text{ for all } i (1 \leq i \leq m) \}$. Analogously for 3.16 and 3.18. The proof of the validity of these characterizations are the same as those of Theorems 3.15, 3.17 and Lemmata 3.19 and 3.20.

To elucidate the capability of a macro tree transducer with regular look-ahead we give an example (cf. [16]).

4.20. EXAMPLE. Consider a tree s in T_Σ , where Σ is the ranked alphabet $\{\sigma^{(2)}, \beta^{(1)}, a^{(0)}\}$, and insert an E above the root of a subtree s' of s if the yield of s' has an even number of a s, and insert an O otherwise. This tree translation can be done by the following macro tree transducer with regular look-ahead. Let $M = (Q, P, \Sigma, \Delta, q, R, \delta) \in D_t MT_a^R$, where $Q = \{q\}$, $P = \{p_O, p_E\}$, $\Delta = \{\sigma^{(2)}, \beta^{(1)}, a^{(0)}, O^{(1)}, E^{(1)}\}$, and R contains the rules

$$\begin{aligned} (q(\sigma(x_1, x_2)) &\rightarrow E(\sigma(q(x_1), q(x_2))), \langle p_E, p_E \rangle), \\ (q(\sigma(x_1, x_2)) &\rightarrow E(\sigma(q(x_1), q(x_2))), \langle p_O, p_O \rangle), \\ (q(\sigma(x_1, x_2)) &\rightarrow O(\sigma(q(x_1), q(x_2))), \langle p_E, p_O \rangle), \\ (q(\sigma(x_1, x_2)) &\rightarrow O(\sigma(q(x_1), q(x_2))), \langle p_O, p_E \rangle), \\ (q(\beta(x_1)) &\rightarrow E(\beta(q(x_1))), \langle p_E \rangle), \\ (q(\beta(x_1)) &\rightarrow O(\beta(q(x_1))), \langle p_O \rangle), \\ (q(a) &\rightarrow O(a), \langle \rangle), \end{aligned}$$

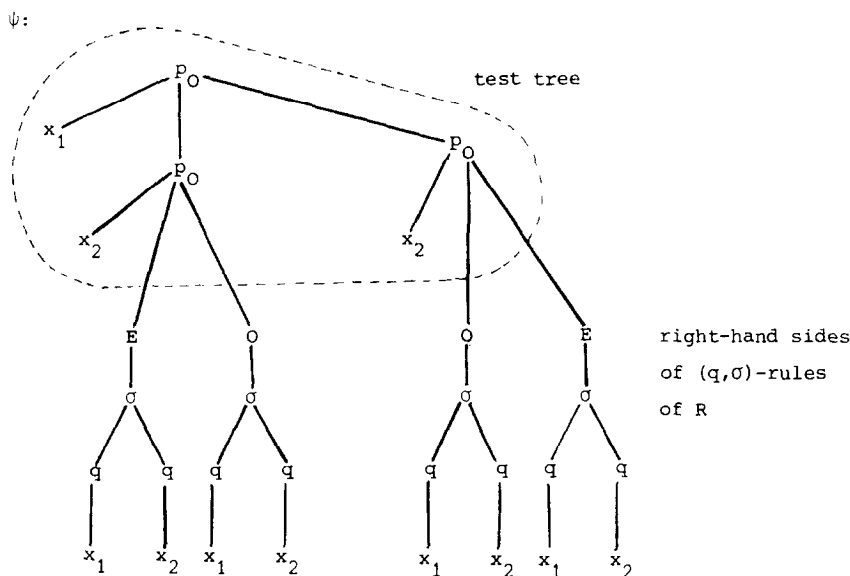
and δ is given by

$$\begin{aligned} \delta_a(\) &= p_O, & \delta_\sigma(p_O, p_O) &= \delta_\sigma(p_E, p_E) = p_E, \\ & & \delta_\sigma(p_E, p_O) &= \delta_\sigma(p_O, p_E) = p_O, \\ \delta_\beta(p_E) &= p_E, & \delta_\beta(p_O) &= p_O. \end{aligned}$$

Although M , in Example 4.20, is "only" a total deterministic top-down tree transducer with regular look-ahead, it can be used to illustrate the technique of simulating a transducer in $D_t MT_a^R$ by one without regular look-ahead.

In the sequel we use only the OI-derivation mode. This is no restriction, because we are dealing with total deterministic macro tree transducers (cf. Theorem 4.1).

4.20. EXAMPLE (continued). We want to construct $M' = (Q', \Sigma, \Delta, q^{\text{in}}, R') \in D_t MT_a$ such that $\tau(M) = \tau(M')$. For this purpose we combine all right-hand sides of (q, σ) -rules of R into one right-hand side, that starts with a test tree, see Fig. 6. The test tree is built up like a nested if-then-else statement: if $p_O(x_1)$ then (if $p_O(x_2)$ then t_1 else t_2) else (if $p_O(x_2)$ then t_2 else t_1), where $t_1 = E(\sigma(q(x_1), q(x_2)))$ and $t_2 = O(\sigma(q(x_1), q(x_2)))$. If we know that s_1 has an odd number of a s in its yield and the

FIG. 6. Combination of right-hand sides into one tree ψ .

same holds for s_2 , then we know that $\sigma(s_1, s_2)$ has an even number of as in its yield and we want to continue with the right-hand side ι_1 . The rules of M' for the state p_O (which has now rank 3) are built up on the "regular behavior on the input tree," i.e., according to the function family δ of M . These rules ensure that for trees $u, v \in SF$ and $s \in T_\Sigma$, $p_O(s, u, v) \xrightarrow{*}_{OI} u$ in M' if the yield of s has an odd number of as and $p_O(s, u, v) \xrightarrow{*}_{OI} v$ otherwise. Thus these derivations "implement" the if-then-else construction. The construction of M' is now obvious: $Q' = Q \cup \{p_O^{(3)}\}$ and R' contains the rules

$$q(\sigma(x_1, x_2)) \rightarrow \psi, \quad \text{where } \psi \text{ is the whole tree of Fig. 6,}$$

$$q(\beta(x_1)) \rightarrow p_O(x_1, O(\beta(q(x_1))), E(\beta(q(x_1)))),$$

$$q(a) \rightarrow O(a),$$

$$p_O(\sigma(x_1, x_2), y_1, y_2) \rightarrow p_O(x_1, p_O(x_2, y_1, y_2), p_O(x_2, y_2, y_1)),$$

$$p_O(\beta(x_1), y_1, y_2) \rightarrow p_O(x_1, y_1, y_2),$$

$$p_O(a, y_1, y_2) \rightarrow y_2.$$

It is easy to see that, if s has an odd number of as , then $M'(p_O(s, y_1, y_2)) = y_1$, and otherwise $M'(p_O(s, y_1, y_2)) = y_2$. Note that the look-ahead state p_E of M is not used in the construction. This is due to the fact that a total deterministic bottom-up tree automaton B provides for every tree s a unique look-ahead state p such that $\delta(s) = p$. Hence, if we suppose that B has r states and we have already checked that s is not "translated" into one of the first $r-1$ states, then we are sure that $\delta(s)$ is equal to the last state.

The technique of Example 4.20 can be generalized in an obvious way. However, the formalization is a bit tedious.

4.21. THEOREM. $D_i MT^R = D_i MT$.

Proof. Let $M = (Q, P, \Sigma, \Delta, q^{\text{in}}, R, \delta) \in D_i MT_a^R$ and let for convenience $P = \{1, \dots, r\}$ with $r \geq 1$. To construct an $M' = (Q', \Sigma, \Delta, q^{\text{in}}, R') \in D_i MT_a$ such that $\tau(M) = \tau(M')$ we need some auxiliary tools, in particular the notion of test tree.

Let $m \geq 0$. The set of test variables Z_m is the set $\bigcup \{z_v | v \in [r]^n, 0 \leq n \leq m\}$. Intuitively, for $v \in Z_m$, $v(i)$ indicates the state of the bottom-up finite tree automaton for x_i (note that $v(i)$ denotes the i th component of v).

In what follows we assume that the elements of P have rank 3; actually, for M' we will define $Q' = Q \cup \{1^{(3)}, \dots, (r-1)^{(3)}\}$, the state r of P is not needed as explained in Example 4.20.

Let $m \geq 1$, $i \in [m]$, and $v \in [r]^{i-1}$. The m -optimal test tree of x_i at point v , $\text{opt}_m(x_i, v) \in T_P(\{x_i\} \cup Z_m)$, is the tree

$$1(x_i, z_{v_1}, 2(x_i, z_{v_2}, \dots, r-1(x_i, z_{v_{r-1}}, z_{v_r}) \dots)),$$

where $v_j \in [r]^i$ is defined by $v_j(k) = v(k)$ for all k , $1 \leq k < i$, and $v_j(i) = j$. (Note that for $r = 1$, $\text{opt}_m(x_i, v) = z_{v_1}$.) If for example, $P = \{1, 2, 3\}$, $r = 3$, $m = 3$, $i = 2$, $v = (2)$, then

$$\text{opt}_m(x_i, v) = 1(x_2, z_{(2,1)}, 2(x_2, z_{(2,2)}, z_{(2,3)})).$$

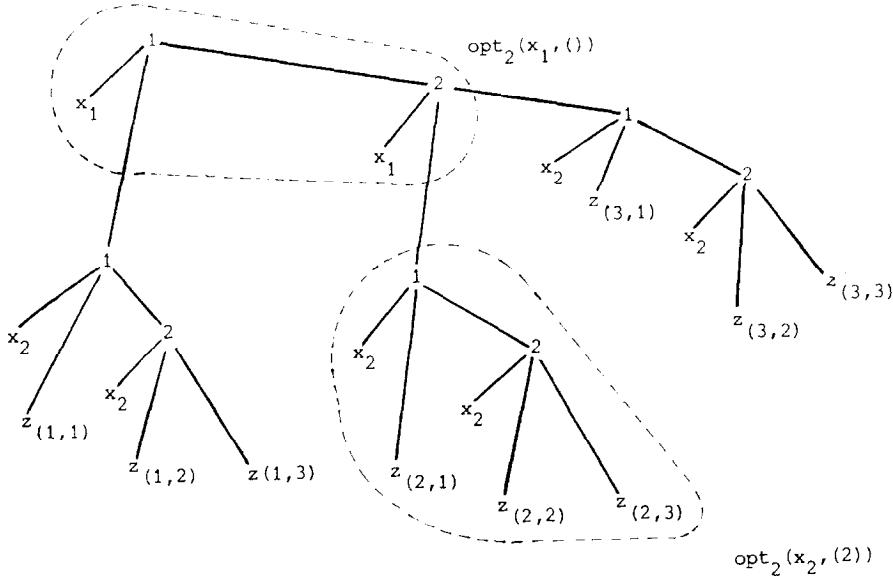
Let $m \geq 0$. The m -full test tree, $ft_m \in T_P(X_m \cup Z_m)$, is the tree $\text{test}_m(m)$, where $\text{test}_m(n)$ is defined inductively for n which $0 \leq n \leq m$, as follows:

$$\begin{aligned} \text{test}_m(0) &= z_{()} \\ \text{test}_m(n+1) &= \text{test}_m(n) [z_{(1, \dots, 1)} / \text{opt}_m(x_{n+1}, (1, \dots, 1)), \\ &\quad z_{(2, 1, \dots, 1)} / \text{opt}_m(x_{n+1}, (2, 1, \dots, 1)), \\ &\quad \vdots \\ &\quad z_{(r, \dots, r)} / \text{opt}_m(x_{n+1}, (r, \dots, r))], \end{aligned}$$

where $(1, \dots, 1)$, $(2, 1, \dots, 1), \dots, (r, \dots, r)$ have length n .

Note that all test variables in $\text{test}_m(n)$ are elements of $\{z_v | v \in [r]^n\}$. In the sequel we abbreviate the above substitution of optimal test trees by $[z_v / \text{opt}_m(x_{n+1}, v)]$, for all $v \in [r]^n$ and similarly in other cases. For an example of a full test tree, see Fig. 7. The right-hand side of the (q, σ) -rule $(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle \alpha_1, \dots, \alpha_m \rangle)$ is denoted by ψ_v , where $v = \langle \alpha_1, \dots, \alpha_m \rangle$.

Now we are able to construct $M' \in D_i MT_a$. Let $Q' = Q \cup \{1^{(3)}, \dots, (r-1)^{(3)}\}$ and let R' contain the following rules:

FIG. 7. Full test tree ft_m ; $P = \{1, 2, 3\}$, $r = 3$, $m = 2$.

(i) If $m, n \geq 0$, $q \in Q_{n+1}$, $\sigma \in \Sigma_m$, and $\psi_{v_1}, \dots, \psi_{v_k}$ with $k = r^m$ are the right-hand sides of the (q, σ) -rules in R , then

$$q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow ft_m[z_v/\psi_v \text{ for all } v \in [r]^m]$$

is in R' .

(ii) For all $i \in P$, $\sigma \in \Sigma_m$ with $m \geq 0$, $i(\sigma(x_1, \dots, x_m), y_1, y_2) \rightarrow ft_m[z_v/d_v]$, for all $v \in [r]^m$ is in R' , where

$$\begin{aligned} d_v &= y_1 & \text{if } \delta_\sigma(v) = i, \\ &= y_2 & \text{otherwise.} \end{aligned}$$

Before we prove $M_{OI}(q^{in}(s)) = M'_{OI}(q^{in}(s))$ for all $s \in T_\Sigma$, we need an auxiliary statement. Considering the test variables as ordinary output symbols of rank 0, we can prove the following two statements by simultaneous induction on s and π :

(1) For all $s \in T_\Sigma$ and $\alpha \in [r]$,

$$\begin{aligned} M'_{OI}(\alpha(s, y_1, y_2)) &= y_1 & \text{if } \delta(s) = \alpha, \\ &= y_2 & \text{otherwise.} \end{aligned}$$

(2) For all $\pi = (s_1, \dots, s_m) \in T_\Sigma^m$,

$$M'_{OI, \pi}(ft_m) = z_v \quad \text{where } v = (\delta(s_1), \dots, \delta(s_m)).$$

Note that, formally, for $M'_{OI,\pi}$ to be defined on ft_m , we have to add the test variables to Δ_0 . The proof of (2) $\Rightarrow$ (1) is straightforward using Lemma 4.7. In the proof of (1) $\Rightarrow$ (2) we need the following statement:

$$\begin{aligned} &\text{For all } i \in [m], v \in [r]^{i-1}: \text{ if } \delta(s_i) = \alpha, \text{ then for all } j, 1 \leq j \leq \alpha, \\ &M'_{OI,\pi}(\text{opt}_m(x_i, v)) = M'_{OI,\pi}(j(x_i, z_{v_j}, \dots, r-1(x_i, z_{v_{r-1}}, z_{v_r}, \dots))). \end{aligned} \quad (*)$$

This can be proved by induction on $j \in [\alpha]$ using (1). (Note that, if $\alpha = r = 1$, $j(x_i, z_{v_j}, \dots, r-1(x_i, z_{v_{r-1}}, z_{v_r}, \dots))$ denotes $z_{v_r} = z_{v_1}$.) Now, using (*) for $j = \alpha$ and again (1) we have: if $\delta(s_i) = \alpha$, then

$$\begin{aligned} M'_{OI,\pi}(\text{opt}_m(x_i, v)) &= M'_{OI,\pi}(\alpha(x_i, z_{v_\alpha}, \dots, r-1(x_i, z_{v_{r-1}}, z_{v_r}, \dots))) \quad \text{by } (*), \\ &= M'_{OI,\pi}(z_{v_\alpha}) \quad \text{by (1),} \\ &= z_{v_\alpha} \quad \text{by Definition 3.18.} \end{aligned}$$

Now we can show (1) $\Rightarrow$ (2) by showing that, for all $n \in [m]$, $M'_{OI,\pi}(\text{test}_m(n)) = z_{(\delta(s_1), \dots, \delta(s_n))}$. This is again a straightforward induction on n using Lemma 4.7 and the above conclusion of (*). This finishes the proof of (1) and (2).

Now we are prepared to give the main part of the proof. For $q = q^{\text{in}}$, the first of the next two statements turns into $M'_{OI}(q^{\text{in}}(s)) = M_{OI}(q^{\text{in}}(s))$ for all $s \in T_\Sigma$. Hence, it is sufficient to show

$$(3) \quad \text{for all } s \in T_\Sigma \text{ and } q \in Q_{n+1} \text{ with } n \geq 0, \quad M'_{OI}(q(s, y_1, \dots, y_n)) = M_{OI}(q(s, y_1, \dots, y_n)) \text{ and}$$

$$(4) \quad \text{for all } \pi = (s_1, \dots, s_m) \in T_\Sigma^m \text{ and } t \in \text{RHS}(Q, \Delta, m, n), \quad M'_{OI,\pi}(t) = M_{OI,\pi}(t)$$

by simultaneous induction on s and π (see Remark 4.19 for the fact that M_{OI} is defined). Since the behavior of M' on test trees is described by (1) and (2), it is sufficient for the proof of (3) to cut the derivation of M' at those sentential forms which do not contain "test" states (states of P), i.e., to consider in (4) right-hand sides of M only.

(4) $\Rightarrow$ (3). $M'_{OI}(q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n)) = M'_{OI,\pi}(ft_m[z_v/\psi_v \text{ for all } v \in [r]^m])$ (where the ψ_v are the right-hand sides of the (q, σ) -rules and $\pi = (s_1, \dots, s_m)$)

$$\begin{aligned} &= M'_{OI,\pi}(ft_m)[z_v/M'_{OI,\pi}(\psi_v)] \quad \text{by Lemma 4.7,} \\ &= z_{(\delta(s_1), \dots, \delta(s_m))}[z_v/M'_{OI,\pi}(\psi_v)] \quad \text{by (2),} \\ &= M'_{OI,\pi}(\psi_{(\delta(s_1), \dots, \delta(s_m))}) \\ &= M_{OI,\pi}(\psi_{(\delta(s_1), \dots, \delta(s_m))}) \quad \text{by (4),} \\ &= M_{OI}(q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n)) \quad \text{by Definition 3.18.} \end{aligned}$$

(3) $\Rightarrow$ (4). Straightforward induction on the structure of t . Note that t does not contain a state in P .

This shows $D_t MT^R \subseteq D_t MT$. The other direction is easy to see. Let $M = (Q, \Sigma, \Delta, q^{in}, R) \in D_t MT_a$. Construct $M' = (Q, P, \Sigma, \Delta, q^{in}, R', \delta) \in D_t MT_a^R$ by $P = \{p\}$ and for all $m \geq 0$, $\sigma \in \Sigma_m$: $\delta_\sigma(p, \dots, p) = p$. If R contains the rule $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t$, where $m, n \geq 0$, $q \in Q_{n+1}$, $\sigma \in \Sigma_m$, and $t \in \text{RHS}(Q, \Delta, m, n)$, then

$$(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p, \dots, p \rangle) \text{ is in } R'.$$

Obviously, for all $s \in T_\Sigma$: $\delta(s) = p$. Hence, the application of rules in R' is not subjected to any restrictions. It immediately follows that $M_{OI}(q^{in}(s)) = M'_{OI}(q^{in}(s))$. ■

As a consequence we obtain the next corollary.

4.22. COROLLARY. $D_t B \subsetneq D_t MT$.

Proof.

$$\begin{aligned} D_t B &\subsetneq D_t T^R && \text{by the "total" analog of Theorem 3.2 in [14]} \\ &\subseteq D_t MT^R \\ &= D_t MT && \text{by Theorem 4.21. } \blacksquare \end{aligned}$$

Thus, although the macro tree transducer is a typical top-down formalism, $D_t MT$ contains both the (total deterministic) bottom-up and top-down finite state tree transductions. Note that the latter two classes are incomparable (Sect. 2 of [12]). Theorem 4.21 can also be used to show that $AG \subseteq D_t MT$, see [16]. Another proof of this is given in [24].

5. IO-MACRO TREE TRANSDUCERS

For MT_{IO} a decomposition result is provided that generalizes the result in Theorem 4.8. We model bottom-up tree transducers by IO-macro tree transducers and investigate the enlargement with regular look-ahead.

5.1. Decomposition of MT_{IO}

In this section we work out the decomposition of MT_{IO} into translations realized by top-down tree transducers followed by relations in YIELD (Lemma 5.5). A partial converse of this decomposition is proved (Lemma 5.8) and by means of an example it is shown that the composition of T and YIELD is more powerful than MT_{IO} (Example 5.10). However, by restricting the class of top-down tree transducers (res- T , see Definition 5.11) we obtain a characterization of MT_{IO} in the form of the composition of res- T and YIELD (Theorem 5.12). On the other hand, we show that the class $T \circ \text{YIELD}$ is not too different from MT_{IO} . In fact, $T \circ \text{YIELD}$ is characterized by left-composition of linear tree homomorphisms to MT_{IO}

(Theorem 5.15). To obtain these results we need some auxiliary definitions and lemmata.

5.1. DEFINITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$. A state $q \in Q$ is *deterministic* if for every $\sigma \in \Sigma$ there is at most one (q, σ) -rule in R and (if there is one) all states in the right-hand side of that rule are deterministic. Similarly, a state $q \in Q$ is *total* if for every $\sigma \in \Sigma$ there is at least one (q, σ) -rule and, for at least one of these rules, all states in the right-hand side are total.

It is easy to show that, if all states in a right-hand side t are deterministic, then $M_\pi(t)$ is a singleton or empty, and if all states in t are total, then $M_\pi(t)$ is non-empty. In the sequel we prove a substitution lemma of which Lemma 4.7 is a special case.

5.2. LEMMA. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in MT_a$ and $\pi \in T_\Sigma^m$ with $m \geq 0$. Let $t_0 \in \text{RHS}(Q, \Delta, m, p)$ with $p \geq 1$ and $t_1, \dots, t_p \in \text{RHS}(Q, \Delta, m, n)$ with $n \geq 0$ satisfy the following conditions: for every i ($1 \leq i \leq p$),

- (1) all states in t_i are deterministic or y_i occurs at most once in t_0 , and
- (2) all states in t_i are total or y_i occurs at least once in t_0 .

Then

$$M_{\text{IO},\pi}(t_0[t_1, \dots, t_p]) = M_{\text{IO},\pi}(t_0) \leftarrow_{\text{IO}} (M_{\text{IO},\pi}(t_1), \dots, M_{\text{IO},\pi}(t_p)). \quad (*)$$

Proof. First, it can be shown by induction on t_0 that for all $t_1, \dots, t_p$, if for some i ($1 \leq i \leq p$) $M_{\text{IO},\pi}(t_i) = \emptyset$ and y_i occurs in t_0 , then $M_{\text{IO},\pi}(t_0[t_1, \dots, t_p]) = \emptyset$. This is left to the reader. Next we show by induction on the structure of t_0 that, if $t_1, \dots, t_p$ satisfy condition (1) and $M_{\text{IO},\pi}(t_i) \neq \emptyset$ for all i ($1 \leq i \leq p$), then $(*)$ holds. (This proves the result: if $M_{\text{IO},\pi}(t_i) = \emptyset$ for some i , then by condition (2), y_i has to occur in t_0 , and hence $(*)$ holds by the above.)

(i) For $t_0 = y_j \in Y_p$, we obtain $M_{\text{IO},\pi}(t_j)$ on both sides of $(*)$, because $M_{\text{IO},\pi}(t_i) \neq \emptyset$ for all i ($1 \leq i \leq p$).

(ii) For $t_0 = \delta(\xi_1, \dots, \xi_k)$, $k \geq 0$, $\delta \in \Delta_k$, and $\xi_1, \dots, \xi_k \in \text{RHS}(Q, \Delta, m, p)$, and the abbreviation $\langle M_{\text{IO},\pi}(t_i) \rangle$ for $(M_{\text{IO},\pi}(t_i), \dots, M_{\text{IO},\pi}(t_p))$,

$$\begin{aligned} & M_{\text{IO},\pi}(t_0[t_1, \dots, t_p]) \\ &= M_{\text{IO},\pi}(\delta(\xi_1[t_1, \dots, t_p], \dots, \xi_k[t_1, \dots, t_p])) \\ &= \{\delta(y_1, \dots, y_k)\} \leftarrow_{\text{IO}} (M_{\text{IO},\pi}(\xi_1[t_1, \dots, t_p]), \dots, M_{\text{IO},\pi}(\xi_k[t_1, \dots, t_p])) \\ &\quad \text{by Definition 3.18,} \\ &= \{\delta(y_1, \dots, y_k)\} \leftarrow_{\text{IO}} (M_{\text{IO},\pi}(\xi_1) \leftarrow_{\text{IO}} \langle M_{\text{IO},\pi}(t_i) \rangle, \dots, M_{\text{IO},\pi}(\xi_k) \leftarrow_{\text{IO}} \langle M_{\text{IO},\pi}(t_i) \rangle) \\ &\quad \text{by I.H.,} \end{aligned}$$

$$\begin{aligned}
&= (\{\delta(y_1, \dots, y_k)\} \xleftarrow{\text{IO}} (M_{\text{IO}, \pi}(\xi_1), \dots, M_{\text{IO}, \pi}(\xi_k))) \xleftarrow{\text{IO}} \langle M_{\text{IO}, \pi}(t_i) \rangle \quad (**) \\
&= M_{\text{IO}, \pi}(\delta(\xi_1, \dots, \xi_k)) \xleftarrow{\text{IO}} \langle M_{\text{IO}, \pi}(t_i) \rangle.
\end{aligned}$$

Note that the induction hypothesis is applicable because if $\delta(\xi_1, \dots, \xi_k)$ satisfies condition (1) then so does ξ_j . Equation (**) follows from Lemma 2.4.3 of [20] which is applicable for the following reasons (see [20]): for every i , either t_i contains deterministic states only and then $M_{\text{IO}, \pi}(t_i)$ is a singleton, or t_0 is linear in $\{y_i\}$ and so y_i occurs in at most one of the ξ_j and hence in the trees of at most one of the $M_{\text{IO}, \pi}(\xi_j)$ (for any t , the trees in $M_{\text{IO}, \pi}(t)$ contain at most the y 's of t).

(iii) The case that t has the form $q(x_j, \xi_1, \dots, \xi_k)$ is analogous to case (ii). ■

Note that the conditions of Lemma 5.2 are the negations of the two bottom-up properties (B1) and (B2) discussed in Section 2 of [12]: condition (1) prohibits nondeterminism followed by copying (that is, property (B1)), condition (2) prohibits partialness followed by deletion (that is, property (B2)).

To understand the necessity of the conditions (1) and (2) for the validity of the previous lemma, we refer to Example 3.10. Choosing $t_0 = g(y, y)$, $t_1 = q^{\text{in}}(x)$, and $\pi = (a)$ the first condition is violated; for $t_0 = e$, $t_1 = q^{\text{in}}(x)$, and $\pi = (b)$ the second condition does not hold. The reader can easily check that in both cases (*) is not true.

To support the following central lemma technically we define a connection between top-down tree transducers and macro tree transducers via a YIELD-mapping.

5.3. DEFINITION. Let $N = (Q, \Sigma, \Omega, q^{\text{in}}, R_N) \in T_a$, $M = (Q, \Sigma, \Delta, q^{\text{in}}, R_M) \in MT_a$, and let $f: \Omega_0 \rightarrow T_\Delta(Y)$ be a mapping. The rank of the elements of Q may be different for N and M . Then, N and M are related by f if $R_M = \{q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow \text{YIELD}_f(t) \mid q(\sigma(x_1, \dots, x_m)) \rightarrow t \text{ in } R_N\}$, where YIELD_f is extended in the obvious way to right-hand sides of rules of R_N by defining $\text{YIELD}_f(q(x_j)) = q(x_j, y_1, \dots, y_n)$ if $q \in Q_{n+1}$ in M (thus, now $\text{YIELD}_f: \text{RHS}(Q, \Omega, m, 0) \rightarrow \bigcup \{\text{RHS}(Q, \Delta, m, k) \mid k \geq 0\}$ for every $m \geq 0$).

Note that, if N and M are related, then N is deterministic (total) if and only if M is deterministic (total).

The next lemma expresses the connection of the translations of a top-down tree transducer and a macro tree transducer that are related by a mapping.

5.4. LEMMA. Let N , M , and f be as in Definition 5.3, and let N and M be related by f . Let N satisfy the following property: for every subtree $\delta(t_0, t_1, \dots, t_k)$ of a right-hand side of a rule of N ($\delta \in \Omega_{k+1}$, $k \geq 0$) and for every i ($1 \leq i \leq k$):

- (1) all states in t_i are deterministic or y_i occurs at most once in $\text{YIELD}_f(t_0)$, and
- (2) all states in t_i are total or y_i occurs at least once in $\text{YIELD}_f(t_0)$. Then, for

every $q \in Q$, $\tau_{q, \text{IO}}(M) = \tau_q(N) \circ \text{YIELD}_f$. (For the definition of τ_q we refer to Definition 3.22.)

Proof. We show the following two statements by simultaneous induction on s and π :

- (1) For all $s \in T_\Sigma$ and $q \in Q_{n+1}$ with $n \geq 0$,

$$M_{\text{IO}}(q(s, y_1, \dots, y_n)) = \text{YIELD}_f(N(q(s))).$$

- (2) For all $\pi \in T_\Sigma^m$ with $m \geq 0$ and all $t \in \text{RHS}(Q, \Omega, m, 0)$ that are subtrees of a right-hand side of a rule in R_N ,

$$M_{\text{IO}, \pi}(\text{YIELD}_f(t)) = \text{YIELD}_f(N_\pi(t)).$$

In the rest of this proof IO is omitted and YIELD_f is abbreviated by Y_f . The proof of (2) $\Rightarrow$ (1) is left to the reader. By induction on the structure of t (1) $\Rightarrow$ (2) is proved. We want to work out only the case in which $t = \delta(\xi_0, \xi_1, \dots, \xi_k)$, where $k \geq 0$, $\delta \in \Omega_{k+1}$, and $\xi_0, \dots, \xi_k \in \text{RHS}(Q, \Omega, m, 0)$. Then

$$\begin{aligned} M_\pi(Y_f(\delta(\xi_0, \xi_1, \dots, \xi_k))) \\ &= M_\pi(Y_f(\xi_0)[Y_f(\xi_1), \dots, Y_f(\xi_k)]) \quad \text{by definition of } Y_f, \\ &= M_\pi(Y_f(\xi_0)[Y_f(\xi_1), \dots, Y_f(\xi_k), y_{k+1}, \dots, y_p]) \end{aligned}$$

for some p such that $Y_f(\xi_0) \in \text{RHS}(Q, \Delta, m, p)$,

$$\begin{aligned} &= M_\pi(Y_f(\xi_0)) \xleftarrow{\text{IO}} (M_\pi(Y_f(\xi_1)), \dots, M_\pi(Y_f(\xi_k)), M_\pi(y_{k+1}), \dots, M_\pi(y_p)) \quad (*) \\ &= Y_f(N_\pi(\xi_0)) \xleftarrow{\text{IO}} (Y_f(N_\pi(\xi_1)), \dots, Y_f(N_\pi(\xi_k)), \{y_{k+1}\}, \dots, \{y_p\}) \quad \text{by I.H.,} \\ &= Y_f(\{\delta(t_0, t_1, \dots, t_k) \mid t_i \in N_\pi(\xi_i) \text{ for all } i, 0 \leq i \leq k\}) \quad \text{by definition of } Y_f, \\ &= Y_f(N_\pi(\delta(\xi_0, \xi_1, \dots, \xi_k))). \end{aligned}$$

At (*) Lemma 5.2 is applied which is possible by our assumption on N : if q is deterministic (or total) in N , then it is also deterministic (or total, respectively) in M (but not necessarily vice versa if f is not injective). Note that $Y_f(\xi)$ contains at most the same states as ξ and note furthermore that, trivially, all states in y_i are deterministic and total ($k+1 \leq i \leq p$). ■

Now we are in the position to prove the decomposition result.

5.5. LEMMA. $MT_{\text{IO}} \subseteq T \circ \text{YIELD}$, and the properties “total,” “deterministic,” and “linear” are preserved going from MT_{IO} to T .

Proof. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R_M)$ be a macro tree transducer. Define Ω to be the ranked alphabet such that $\Omega_0 = \{\pi_1, \dots, \pi_n\} \cup \{\delta' \mid \delta \in \Delta\}$, where $n+1$ is the maximal rank of elements of Q , and $\Omega_k = \{c_k\}$ for every k such that $Q_k \neq \emptyset$ or $\Delta_{k-1} \neq \emptyset$. Define $f: \Omega_0 \rightarrow T_\Delta(Y)$ by $f(\pi_i) = y_i$, $1 \leq i \leq n$, and $f(\delta') = \delta(y_1, \dots, y_k)$ if

$\delta \in \Delta_k$. For the construction of the top-down tree transducer we need a family $\text{COMB} = \{\text{COMB}_{m,n}\}_{m,n \geq 0}$ of mappings $\text{COMB}_{m,n}: \text{RHS}(Q, \Delta, m, n) \rightarrow \text{RHS}(Q, \Omega, m, 0)$, that are defined inductively on the structure of a right-hand side (cf. Definition 4.4 in [20]):

- (i) For $j \in [n]$, $\text{COMB}_{m,n}(y_j) = \pi_j$.
- (ii) For $\delta \in \Delta_k$, $k \geq 0$, and $\xi_1, \dots, \xi_k \in \text{RHS}(Q, \Delta, m, n)$,

$$\text{COMB}_{m,n}(\delta(\xi_1, \dots, \xi_k)) = c_{k+1}(\delta', \text{COMB}_{m,n}(\xi_1), \dots, \text{COMB}_{m,n}(\xi_k)).$$
- (iii) For $q \in Q_{k+1}$, $k \geq 0$, $x_j \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}(Q, \Delta, m, n)$,

$$\text{COMB}_{m,n}(q(x_j, \xi_1, \dots, \xi_k)) = c_{k+1}(q(x_j), \text{COMB}_{m,n}(\xi_1), \dots, \text{COMB}_{m,n}(\xi_k)).$$

Now we define the top-down tree transducer $N = (Q, \Sigma, \Omega, q^{\text{in}}, R_N)$ with

$$R_N = \{q(\sigma(x_1, \dots, x_m)) \rightarrow \text{COMB}_{m,n}(t) \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ is in } R_M\}.$$

It is easy to see that N and M are related by f ; in fact, for

$$t \in \text{RHS}(Q, \Delta, m, n), \quad t = \text{YIELD}_f(\text{COMB}_{m,n}(t)).$$

It should also be clear from the definition of COMB that for every subtree $\delta(t_0, t_1, \dots, t_k)$ of the right-hand side of a rule of R_N (i.e., $\delta = c_{k+1}$), $\text{YIELD}_f(t_0)$ contains each y_i ($1 \leq i \leq k$) exactly once. Hence, N and M satisfy the conditions of Lemma 5.4 and so (taking $q = q^{\text{in}}$), $\tau_{\text{IO}}(M) = \tau(N) \circ \text{YIELD}_f$. Thus, $\tau_{\text{IO}}(M) \in T \circ \text{YIELD}$. Clearly, since N and M are related, if M is total or deterministic, then so is N . Also, $\text{COMB}_{m,n}$ preserves the number of occurrences of x_j in the right-hand side; hence, if M is linear then so is N . ■

From this decomposition we obtain a result concerning the domain of IO-macro tree transducers. Recall that $DT\text{-RECOG} = \text{dom}(DT\text{-FTA})$.

5.6. COROLLARY. $\text{dom}(MT_{\text{IO}}) = \text{RECOG}$ and $\text{dom}(DMT_{\text{IO}}) = DT\text{-RECOG} \subsetneq \text{RECOG}$.

Proof.

$$\begin{aligned} \text{dom}(MT_{\text{IO}}) &= \text{dom}(T) \quad (\text{YIELD contains total functions only}), \\ &= \text{RECOG} \quad [34], \end{aligned}$$

$$\text{dom}(DMT_{\text{IO}}) = \text{dom}(DT) = \text{dom}(DT\text{-FTA}) \subsetneq \text{RECOG}. \quad \blacksquare$$

In the next corollary we characterize the IO context-free tree languages as the images of the recognizable tree languages under linear (IO) macro tree transductions (cf. Theorem 4.6).

5.7. COROLLARY. $IOT = LMT_{IO}(\text{RECOG}) = LD_i MT(\text{RECOG}) \subsetneq D_i MT(\text{RECOG})$.

Proof. It is known from [20] that $IOT = \text{YIELD}(\text{RECOG})$. Hence, by Theorem 4.6, $IOT \subseteq LD_i MT(\text{RECOG}) \subseteq LMT_{IO}(\text{RECOG})$. By Lemma 5.5, $LMT_{IO} \subseteq LT \circ \text{YIELD}$ and hence, since RECOG is closed under LT , $LMT_{IO}(\text{RECOG}) \subseteq \text{YIELD}(LT(\text{RECOG})) = \text{YIELD}(\text{RECOG}) = IOT$. The properness of the inclusion $IOT \subseteq D_i MT(\text{RECOG})$ is guaranteed by the following fact. The path languages of IOT are context-free languages [20], but $D_i MT(\text{RECOG})$ contains non-context-free path languages; cf. Example 4.3 which says that $\{f^{2^n}a | n \geq 0\} \in D_i MT(\text{RECOG})$. ■

In the next lemma we give a partial converse of Lemma 5.5.

5.8. LEMMA. Let $N = (Q, \Sigma, \Omega, q^{\text{in}}, R_N) \in T_a$ and $f: \Omega_0 \rightarrow T_A(Y_n)$ for some $n \geq 0$. Extend YIELD_f to $\text{RHS}(Q, \Omega, m, 0)$ by $\text{YIELD}_f(q(x_j)) = q(x_j, y_1, \dots, y_n)$ for all $q \in Q$ and $x_j \in X_m$. If (with respect to this YIELD_f) N satisfies the property of Lemma 5.4, then $\tau(N) \circ \text{YIELD}_f \in MT_{IO}$.

Proof. Add a new (dummy) initial state q_1 to N , without rules: $N' = (Q \cup \{q_1\}, \Sigma, \Omega, q_1, R_N)$. Then $\tau_p(N') = \tau(N)$, where $p = q^{\text{in}}$. Define $M = (Q \cup \{q_1\}, \Sigma, \Delta, q_1, R_M) \in MT_a$, such that every state $q \in Q$ has rank $n+1$, q_1 has rank 1, and $R_M = \{q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow \text{YIELD}_f(t) | q(\sigma(x_1, \dots, x_m)) \rightarrow t \text{ is in } R_N\}$. Note that, if $t \in \text{RHS}(Q, \Omega, m, 0)$, then $\text{YIELD}_f(t) \in \text{RHS}(Q, \Delta, m, n)$, as required. Clearly, N' and M are related by f . Hence, by Lemma 5.4, $\tau_{p, IO}(M) = \tau_p(N') \circ \text{YIELD}_f = \tau(N) \circ \text{YIELD}_f$ and by Proposition 3.23 it follows that $\tau(N) \circ \text{YIELD}_f \in MT_{IO}$. ■

As a special case of the Lemmata 5.5 and 5.8 we can prove the already mentioned decomposition result for total deterministic macro tree transducers (cf. Theorem 4.8).

5.9. COROLLARY. $D_i MT = D_i T \circ \text{YIELD}$.

Proof. $D_i MT \subseteq D_i T \circ \text{YIELD}$ by Lemma 5.5. Since every $N \in D_i T_a$ satisfies the condition of Lemma 5.4 trivially (every state in N is total deterministic), the inclusion $D_i T \circ \text{YIELD} \subseteq D_i MT$ holds by Lemma 5.8. ■

However, the composition $T \circ \text{YIELD}$ is not contained in MT_{IO} . The next example justifies the necessity of the restriction on the top-down tree transducer required in Lemma 5.8.

5.10. EXAMPLE. Let Σ and Δ be two ranked alphabets given by $\Sigma = \Sigma' \cup \{\sigma^{(3)}\}$ with $\Sigma' = \{a^{(1)}, e^{(0)}\}$ and $\Delta = \Delta' \cup \{\delta^{(2)}\}$ with $\Delta' = \{a^{(1)}, b^{(1)}, \alpha^{(1)}, \beta^{(1)}, e^{(0)}\}$. Consider the relation $R \subseteq T_{\Sigma} \times T_{\Delta}$ given as the set of pairs $\{(\sigma(ue, ve, we), \delta(v\alpha u_1 e, w\beta u_1 e)) | ue, ve, we \in T_{\Sigma'}, v\alpha u_1 e, w\beta u_1 e \in T_{\Delta'}, h(u) = h(u_1) \text{ and } u_1 \text{ contains } as \text{ and } bs \text{ only}\}$. Roughly speaking u_1 is a nondeterministic relabeling of u .

This relation can be obtained by composing a top-down tree transducer N with a mapping YIELD_f ; informally spoken

$$\sigma(ue, ve, we) \xrightarrow{N} c_2(c_3(\delta', t_v, t_w), t_{u_1}) \xrightarrow{\text{YIELD}_f} \delta(v\alpha u_1 e, w\beta u_1 e),$$

where, for example, ue , ve , and we denote the monadic trees aae , ae , and $aaae$, respectively; t_{u_1} , t_v , and t_w are the trees $c_2(a', c_2(b', e'))$, $c_2(a', \alpha')$, and $c_2(a', c_2(a', \beta'))$ (cf. Fig. 8).

Formally, define the top-down tree transducer $N = (\{q_0, q_1, q_2, q_3\}, \Sigma, \Omega, q_0, R) \in T_a$ with $\Omega = \{d' \mid d \in \mathcal{A}\} \cup \{c_2^{(2)}, c_3^{(3)}\}$, where the elements originated from $\mathcal{A}$ have rank 0. Let R contain the following rules:

$$\begin{aligned} q_0(\sigma(x_1, x_2, x_3)) &\rightarrow c_2(c_3(\delta', q_2(x_2), q_3(x_3)), q_1(x_1)), \\ q_1(a(x)) &\rightarrow c_2(a', q_1(x)) | c_2(b', q_1(x)), \\ q_1(e) &\rightarrow e', \\ q_i(a(x)) &\rightarrow c_2(a', q_i(x)) \quad \text{for } i=2,3, \\ q_2(e) &\rightarrow \alpha' \quad \text{and} \quad q_3(e) \rightarrow \beta'. \end{aligned}$$

YIELD_f is determined by the mapping $f: \Omega_0 \rightarrow T_{\mathcal{A}}(Y_2)$, where $f(\delta') = \delta(y_1, y_2)$, $f(d') = d(y_1)$ for all $d \in \mathcal{A}$ that are of rank 1, and $f(e') = e$.

It is easy to see that $\tau(N) \circ \text{YIELD}_f = R$. Note that N does not satisfy part (1) of the condition of Lemma 5.4: in the right-hand side of the first rule q_1 is not deterministic and y_1 occurs twice in $\text{YIELD}_f(c_3(\delta', q_2(x_2), q_3(x_3))) = \delta(q_2(x_1, y_1), q_3(x_3, y_1))$. If we define a related macro tree transducer M by taking YIELD_f of the right-hand sides, M would contain the rule $q_1(\sigma(x_1, x_2, x_3)) \rightarrow \delta(q_2(x_2, q_1(x_1)), q_3(x_3, q_1(x_1)))$ and $\tau_{10}(M) = \{(\sigma(ue, ve, we), \delta(v\alpha u_1 e, w\beta u_1 e)) \mid v\alpha u_1 e, w\beta u_1 e \in T_{\mathcal{A}}\}$.

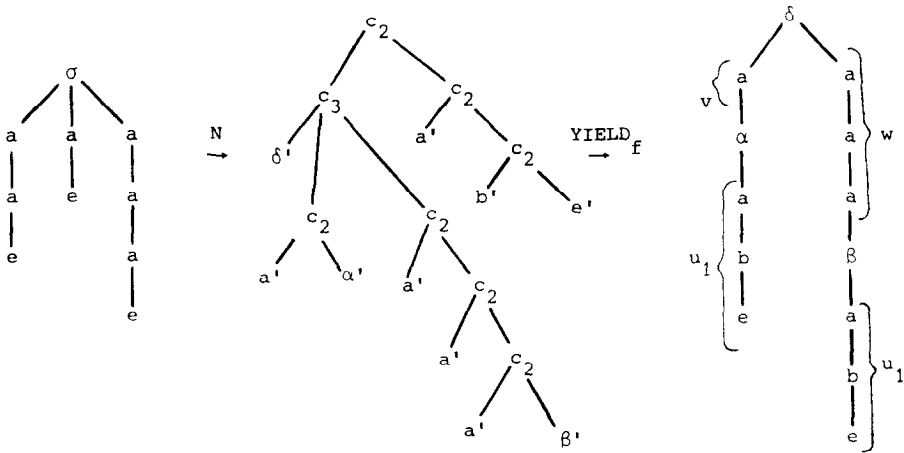


FIG. 8. Decomposition of the relation R .

and $h(u_1) = h(u_2) = h(u)\} \neq R$. In fact, it can be shown by a long combinatorial proof that $R \notin MT_{IO}$.

Now, a natural question is, how a restricted top-down tree transducer ($\text{res-}T_a$) would look like such that the composition with YIELD characterizes MT_{IO} . The following definition provides an answer.

5.11. DEFINITION. Let $N = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in T_a$. N is called *restricted* if for every rule $q(\sigma(x_1, \dots, x_m)) \rightarrow t$ in R with $q \in Q$, $\sigma \in \Sigma_m$, and $m \geq 0$, either $t \in T_\Delta$ or for every subtree $\delta(t_0, t_1, \dots, t_k)$ of t , with $\delta \in \Delta_{k+1}$ ($k \geq 0$), $t_0 \in Q(X_m)$.

For example, the following rules are admissible for a restricted top-down tree transducer:

$$\begin{aligned} q(\sigma(x_1, x_2)) &\rightarrow \delta(a), \\ q(\sigma(x_1, x_2)) &\rightarrow q(x_1), \end{aligned}$$

because there are no subtrees of $q(x_1)$ starting with an output symbol,

$$q(\sigma(x_1, x_2)) \rightarrow f(q_1(x_1), a, g(q_2(x_2), \delta(q_3(x_1))))).$$

The class of restricted top-down tree transducers and their translations are denoted by $\text{res-}T_a$ and $\text{res-}T$, respectively. The notions of restricted deterministic and total top-down tree transducers are defined in an obvious way.

5.12. THEOREM. $MT_{IO} = \text{res-}T \circ \text{YIELD}$, and similarly for DMT_{IO} and MT_{IO} .

Proof. “ $\subseteq$.” Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R_M) \in MT_a$. Now apply the construction of Lemma 5.5 to obtain a top-down tree transducer $N = (Q, \Sigma, \Omega, q^{\text{in}}, R_N)$ and a mapping YIELD_f . From N we can easily derive a restricted top-down tree transducer $N' = (Q', \Sigma, \Omega, q^{\text{in}}, R'_N)$ such that $\tau(N) = \tau(N')$.

Let $Q' = Q \cup \{q_\delta | \delta \in \Delta\}$. It is easy to see from the definition of $\text{COMB}_{m,n}$ that the set of right-hand sides of rules in R_N is contained in the smallest set $\text{rhs}(N)$, that fulfills (i)–(iii):

- (i) For $j \in [n]$, $\pi_j \in \text{rhs}(N)$.
- (ii) For $\delta \in \Delta_k$, $k \geq 0$, $\xi_1, \dots, \xi_k \in \text{rhs}(N)$, and $c_{k+1} \in \Omega_{k+1}$, $c_{k+1}(\delta', \xi_1, \dots, \xi_k) \in \text{rhs}(N)$.
- (iii) For $q \in Q_{k+1}$, $k \geq 0$, $x_j \in X_m$ for any $m \geq 1$, $\xi_1, \dots, \xi_k \in \text{rhs}(N)$, and $c_{k+1} \in \Omega_{k+1}$, $c_{k+1}(q(x_j), \xi_1, \dots, \xi_k) \in \text{rhs}(N)$.

Following this inductive description, we define the right-hand sides for the rules in R'_N by means of a function $\psi: \text{rhs}(N) \rightarrow \bigcup \{\text{RHS}(Q', \Omega, m, 0) | m \geq 0\}$. In case (i), $\psi(\pi_j) = \pi_j$. In case (ii), $\psi(c_{k+1}(\delta', \xi_1, \dots, \xi_k)) = c_{k+1}(q_\delta(x_1), \psi(\xi_1), \dots, \psi(\xi_k))$, and in

case (iii), $\psi(c_{k+1}(q(x_j), \xi_1, \dots, \xi_k)) = c_{k+1}(q(x_j), \psi(\xi_1), \dots, \psi(\xi_k))$. Then the rule set for N' is

$$\begin{aligned} R'_N = & \{q(\sigma(x_1, \dots, x_m)) \rightarrow t \mid q(\sigma(x_1, \dots, x_m)) \rightarrow t \text{ is in } R_N \text{ and } t \in T_\Omega\} \\ & \cup \{q(\sigma(x_1, \dots, x_m)) \rightarrow \psi(t) \mid q(\sigma(x_1, \dots, x_m)) \rightarrow t \text{ is in } R_N \\ & \text{and } t \text{ contains at least one } x\} \\ & \cup \{q_\delta(\alpha(x_1, \dots, x_n)) \rightarrow \delta' \mid \text{for all } \delta \in \Delta, \alpha \in \Sigma_n \text{ with } n \geq 0\}. \end{aligned}$$

Clearly, $\tau(N) = \tau(N')$. Furthermore, it is obvious that N' is restricted. Hence, $MT_{IO} \subseteq \text{res-}T \circ \text{YIELD}$.

" $\supseteq$." We can immediately apply Lemma 5.8, because a restricted transducer fulfills the condition of Lemma 5.4. In fact, if a right-hand side t is in T_Ω , then, trivially, every state in t is total deterministic; if for every subtree $\delta(t_0, t_1, \dots, t_k)$ of a right-hand side t , $t_0 \in Q(X_m)$, then in $\text{YIELD}_f(t_0)$ y_i occurs exactly once for all i , $1 \leq i \leq k$. ■

We note here that we could obtain a proof of $IOT = \text{YIELD}(\text{RECOG})$ (see [20]) as a special case of Theorem 5.12. In fact, if M and N are related (Definition 5.3), then M corresponds to a context-free grammar (in the sense of Sect. 3.2) if and only if N corresponds to a regular tree grammar. Every recognizable tree language can be generated by a regular tree grammar in normal form (cf. Lemma 4.8 of [20]), which corresponds to (even a particular case of) a restricted top-down tree transducer. Hence, by Theorem 5.12, $IOT = \text{out}(MT_{IO} - g) = \text{out}(\text{res-}T - g \circ \text{YIELD}) = \text{YIELD}(\text{out}(\text{res-}T - g)) = \text{YIELD}(\text{RECOG})$, where the g indicates the restriction to grammars. (For the connections of the above mentioned tree grammar and transducer concepts cf. Sects. 1 and 3.2.)

Now we characterize the composition class $T \circ \text{YIELD}$ by means of MT_{IO} . In fact, we obtain this class by left-composing MT_{IO} with linear tree homomorphisms (LHOM), of which the only purpose is to expand the input of the macro tree transducer. Theorem 5.15 ($T \circ \text{YIELD} = \text{LHOM} \circ MT_{IO}$) is prepared by the formal definition of LHOM and a decomposition for top-down tree transducers.

5.13. DEFINITION. Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in D_i T_a$ such that $Q = \{q^{\text{in}}\}$. The translation $\tau(M)$ is called a *tree homomorphism*. If M is linear, then the translation $\tau(M)$ is called a *linear tree homomorphism*.

The class of (linear) tree homomorphisms is denoted by (LHOM) HOM, cf. [12]. Thus, $\text{LHOM} \subseteq \text{HOM} \subseteq D_i T$.

Now we decompose a top-down tree transducer into a linear tree homomorphism and a restricted top-down tree transducer (cf. Definition 5.11). The idea underlying the decomposition of a top-down tree transducer M consists of inserting above every symbol σ of an input tree t a copy of σ , say, σ' with rank 1, and between σ and σ' a certain amount of markers, say c of rank 1, by means of a

linear tree homomorphism. Thus, for example, an input tree $\sigma(t_1, t_2)$ changes into $\sigma'(c^k(\sigma(t'_1, t'_2)))$. Then, every right-hand side of a rule in M is cut into "pieces" and a new rule set is introduced that uses these "pieces" as right-hand sides. Applying the new rules successively by reading the inserted markers, the original right-hand side is built up again. In order to control this rebuilding, the subtrees of the right-hand side that still have to be built up are taken into the finite control. If, for instance, $q(\sigma(x_1, x_2)) \rightarrow t$ with $t = f(s_1)$, $s_1 = g(s_2, s_3, s_4)$, and $s_2 = q_1(x_1)$, $s_3 = a$, $s_4 = h(s_5)$, $s_5 = q_2(x_2)$ is a rule in R , then the following rules are in the new transducer M' , where p abbreviates the rule $q(\sigma(x_1, x_2)) \rightarrow t$:

$$\begin{aligned}
 q(\sigma'(x)) &\rightarrow \langle p, t \rangle(x), \\
 \langle p, t \rangle(c(x)) &\rightarrow f(\langle p, s_1 \rangle(x)), \\
 \langle p, s_1 \rangle(c(x)) &\rightarrow g(\langle p, s_2 \rangle(x), \langle p, s_3 \rangle(x), \langle p, s_4 \rangle(x)), \\
 \langle p, s_2 \rangle(c(x)) &\rightarrow \langle p, s_2 \rangle(x), \langle p, s_2 \rangle(\sigma(x_1, x_2)) \rightarrow q_1(x_1), \\
 \langle p, s_3 \rangle(c(x)) &\rightarrow \langle p, s_3 \rangle(x), \langle p, s_3 \rangle(\sigma(x_1, x_2)) \rightarrow a, \\
 \langle p, s_4 \rangle(c(x)) &\rightarrow h(\langle p, s_5 \rangle(x)), \\
 \langle p, s_5 \rangle(c(x)) &\rightarrow \langle p, s_5 \rangle(x), \langle p, s_5 \rangle(\sigma(x_1, x_2)) \rightarrow q_2(x_2).
 \end{aligned}$$

It is obvious that M' can compute with the input $\sigma'(c^3(t_0))$, where t_0 abbreviates $\sigma(t_1, t_2)$,

$$\begin{aligned}
 q(\sigma'(c^3(t_0))) &\xRightarrow{M'} \langle p, 1 \rangle(c^3(t_0)) \\
 &\xRightarrow{M'} f(g(\langle p, s_2 \rangle(t_0), \langle p, s_3 \rangle(t_0), h(\langle p, s_5 \rangle(t_0)))) \\
 &\xRightarrow{M'} f(g(q_1(t_1), a, h(q_2(t_2)))).
 \end{aligned}$$

This derivation simulates the application of the rule $q(\sigma(x_1, x_2)) \rightarrow t$ of the transducer M .

Of course, the purpose of cutting up the right-hand sides is to obtain a restricted top-down tree transducer. The construction is formalized in the following lemma.

5.14. LEMMA. $T = \text{LHOM} \circ \text{res-}T$ and determinism and totality are preserved.

Proof. " $\supseteq$." $\text{LHOM} \circ \text{res-}T \subseteq D_t T \circ T \subseteq T$ (Lemma 8.3 in [37]).

" $\subseteq$." Let $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in T_a$. A rule $q(\sigma(x_1, \dots, x_m)) \rightarrow t$ in R is denoted by $\langle q, \sigma, t \rangle$. Let mx be $\max\{\text{height}(t) \mid \langle q, \sigma, t \rangle \in R\} - 1$. Let $\Sigma' = \Sigma \cup \Sigma''$, where $\Sigma'' = \{\sigma' \mid \sigma \in \Sigma\} \cup \{c\}$ and all elements in Σ'' have rank 1. Construct $N = (\{q\}, \Sigma, \Sigma', q, R_N) \in D_t T_a$ such that R_N contains for all $\sigma \in \Sigma_m$ with $m \geq 0$ the rule $q(\sigma(x_1, \dots, x_m)) \rightarrow \sigma'(c^{mx}(\sigma(q(x_1), \dots, q(x_m))))$. Clearly, $\tau(N)$ is a linear tree homomorphism. The total mapping from T_Σ to $T_{\Sigma'}$ induced by N is denoted by h_N . Note that h_N is yield-preserving, i.e., for every $s \in T_\Sigma$, $\text{yield}(s) = \text{yield}(h_N(s))$.

Construct $M' = (Q', \Sigma', \Delta, q^{\text{in}}, R') \in \text{res-}T_a$ with $Q' = Q \cup \{\langle p, s \rangle \mid p = \langle q, \sigma, t \rangle \in R, s \text{ subtree of } t\}$. If $p = \langle q, \sigma, t \rangle$ is a rule in R ($q \in Q, \sigma \in \Sigma_m, m \geq 0$, and $t \in T_A(Q(X_m))$), then $q(\sigma'(x)) \rightarrow \langle p, t \rangle(x)$ is in R' . Furthermore, for every subtree s of t the following rules are in R' :

If $s \in \Delta_0 \cup Q(X_m)$, then $\langle p, s \rangle(c(x)) \rightarrow \langle p, s \rangle(x)$ and $\langle p, s \rangle(\sigma(x_1, \dots, x_m)) \rightarrow s$ are in R' , and

if $s = \delta(s_1, \dots, s_n)$ with $\delta \in \Delta_n, n \geq 1$, and $s_1, \dots, s_n \in T_A(Q(X_m))$, then $\langle p, s \rangle(c(x)) \rightarrow \delta(\langle p, s_1 \rangle(x), \dots, \langle p, s_n \rangle(x))$ is in R' .

Clearly, M' is restricted, even in a stronger sense.

First we have to show that the computation on the inserted markers really rebuilds the original right-hand side. For this purpose, the following statement can be proved by induction on the structure of s :

For all $p = \langle q, \sigma, t \rangle \in R$, all subtrees s of t , and all $r_1, \dots, r_m \in T_{\Sigma'}$, if $\text{height}(s) - 1 \leq k$, then $M'(\langle p, s \rangle(c^k(\sigma(r_1, \dots, r_m)))) = M'_\pi(s)$, with $\pi = (r_1, \dots, r_m)$. (*)

Now it is straightforward to show, using (*), that for all $s \in T_{\Sigma'}$ and $q \in Q$, $M'(q(h_N(s))) = M(q(s))$, which proves for $q = q^{\text{in}}$ that $\tau(M) = \tau(N) \circ \tau(M')$.

If M is total, the construction would not immediately provide a total deterministic transducer M' . But it is easy to see that M' can be completed arbitrarily in the part in which the markers are read, because there the computation can never block. ■

The characterization of the composition $T \circ \text{YIELD}$ is now obtained as an immediate consequence.

5.15. THEOREM. $T \circ \text{YIELD} = \text{LHOM} \circ MT_{\text{IO}}$, and determinism and totality are preserved.

Proof.

$$\begin{aligned} T \circ \text{YIELD} &= \text{LHOM} \circ \text{res-}T \circ \text{YIELD} && \text{by Lemma 5.14,} \\ &= \text{LHOM} \circ MT_{\text{IO}}. && \text{by Theorem 5.12. } \blacksquare \end{aligned}$$

5.2. Bottom-Up Tree Transducers

In this section we will show that the bottom-up tree transductions are contained in MT_{IO} . This containment formalizes a property of MT_{IO} that was already mentioned after Example 3.10. IO-macro tree transducers can check in the parameter of their states whether an input (sub)tree is contained in a recognizable tree language (in [12] this property is called (B2)). Furthermore, IO-macro tree transducers have the capability of nondeterminism followed by copying due to their derivation mode (in [12] this is the property (B1) of bottom-up tree transducers). For the definition of bottom-up tree transducer see Section 2.4.

5.16. THEOREM. $B \subseteq MT_{IO}$.

Proof. Let $M = (Q, \Sigma, \Delta, Q_f, R) \in B_a$. Construct $M' = (Q', \Sigma, \Delta, q^{in}, R') \in MT_a$, where $Q' = Q \cup R \cup \{q^{in}\}$ and the states of Q have rank 1, a state $p \in R$ has rank $k + 1$, if the left-hand side of the rule p is headed by a symbol $\sigma \in \Sigma_k$. R' contains the following rules:

(i) If $a \rightarrow q(t)$ is in R , where $a \in \Sigma_0$, $q \in Q$, and $t \in T_\Delta$, then $q(a) \rightarrow t$ is in R' . If, moreover, $q \in Q_f$, then $q^{in}(a) \rightarrow t$ is in R' , too.

(ii) If p is the rule $\sigma(q_1(x_1), \dots, q_k(x_k)) \rightarrow q(t)$ in R , where $\sigma \in \Sigma_k$, $k \geq 1$, $q, q_1, \dots, q_k \in Q$, and $t \in T_\Delta(X_k)$, then $q(\sigma(x_1, \dots, x_k)) \rightarrow p(x_1, q_1(x_1), \dots, q_k(x_k))$ is in R' and for all $\delta \in \Sigma_n$ with $n \geq 0$, $p(\delta(x_1, \dots, x_n), y_1, \dots, y_k) \rightarrow t[x_1/y_1, \dots, x_k/y_k]$ are rules in R' . If, moreover, $q \in Q_f$, then $q^{in}(\sigma(x_1, \dots, x_k) \rightarrow p(x_1, q_1(x_1), \dots, q_k(x_k)))$ is in R' , too.

Now we can prove that

$$\text{for all } s \in T_\Sigma, t \in T_\Delta, \text{ and } q \in Q, \quad s \xrightarrow[M]{*} q(t) \quad \text{iff} \quad q(s) \xrightarrow[M', IO]{*} t \quad (*)$$

by straightforward induction on the structure of s , using obvious properties of derivations of bottom-up tree transducers (expressed in Lemma 1.1 of [12]) and of IO-derivations of macro tree transducers. Considering the first step in the derivation the following holds by construction of M' .

$$\text{For all } s \in T_\Sigma \text{ and } t \in T_\Delta, \quad q^{in}(s) \xrightarrow[M', IO]{*} t \quad \text{iff} \quad (**)$$

$$\text{there is a } q \in Q_f \text{ such that } q(s) \xrightarrow[M', IO]{*} t.$$

Using (*) and (**), we can conclude that $\tau(M) = \tau_{IO}(M')$. ■

To illustrate the construction of Theorem 5.16 we model one of the usual bottom-up tree transducers [12] by an IO-macro tree transducer.

5.17. EXAMPLE. Let $M = (Q, \Sigma, \Delta, Q_f, R) \in B_a$, where $Q = \{q, q_f\}$, $\Sigma = \{f^{(1)}, g^{(1)}, a^{(0)}\}$, $\Delta = \{\phi^{(2)}, \gamma^{(1)}, \delta^{(1)}, \alpha^{(0)}, \beta^{(0)}\}$, $Q_f = \{q_f\}$, and R contains the rules

$$\begin{aligned} p_1: f(q(x)) &\rightarrow q(\gamma(x)); \quad p_2: f(q(x)) \rightarrow q(\delta(x)); \\ p_3: g(q(x)) &\rightarrow q_f(\phi(x, x)); \\ p_4: a &\rightarrow q(\alpha); \quad \text{and } p_5: a \rightarrow q(\beta). \end{aligned}$$

Then, for instance, the monadic input tree $gffa$ can be translated by M in the following way:

$$gffa \Rightarrow gffq(\beta) \Rightarrow gfq(\gamma\beta) \Rightarrow gq(\delta\gamma\beta) \Rightarrow q_f(\phi(\delta\gamma\beta, \delta\gamma\beta))$$

using rules p_5, p_1, p_2 , and p_3 successively.

According to the construction in Theorem 5.16, we obtain a macro tree transducer $M' = (Q', \Sigma, \Delta, q^{\text{in}}, R')$, where $Q' = \{q^{(1)}, q_f^{(1)}, p_1^{(2)}, p_2^{(2)}, p_3^{(2)}, p_4^{(1)}, p_5^{(1)}, q^{\text{in}}\}$ and R' contains for all $\sigma \in \Sigma_m$ with $m \geq 0$ the rules

$$\begin{aligned} q(a) &\rightarrow \alpha|\beta, \\ q(f(x)) &\rightarrow p_1(x, q(x)), \quad p_1(\sigma(x_1, \dots, x_m), y_1) \rightarrow \gamma(y_1), \\ q(f(x)) &\rightarrow p_2(x, q(x)), \quad p_2(\sigma(x_1, \dots, x_m), y_1) \rightarrow \delta(y_1), \\ q(g(x)) &\rightarrow p_3(x, q(x)), \quad p_3(\sigma(x_1, \dots, x_m), y_1) \rightarrow \phi(y_1, y_1), \\ q^{\text{in}}(g(x)) &\rightarrow p_3(x, q(x)). \end{aligned}$$

Now, M' can translate the input tree $gffa$ using IO-derivation in the following way:

$$\begin{aligned} q^{\text{in}}(gffa) &\Rightarrow p_3(ffa, q(ffa)) \\ &\Rightarrow p_3(ffa, p_2(fa, q(fa))) \\ &\Rightarrow p_3(ffa, p_2(fa, p_1(a, q(a)))) \\ &\Rightarrow p_3(ffa, p_2(fa, p_1(a, \beta))) \\ &\Rightarrow p_3(ffa, p_2(fa, \gamma\beta)) \\ &\Rightarrow p_3(ffa, \delta\gamma\beta) \\ &\Rightarrow \phi(\delta\gamma\beta, \delta\gamma\beta). \end{aligned}$$

Proposition 3.6 and Theorem 5.16 show that the translations induced by top-down and bottom-up tree transducers are IO-macro tree transductions i.e., $T \cup B \subseteq MT_{\text{IO}}$. In fact, we can even show that a larger class of translation, the generalized finite state transformations (GFST), introduced in [12], are contained in MT_{IO} . This can be shown by a construction very similar to that of Theorem 5.16. Hence, we obtain $T \cup B \subseteq GFST \subseteq MT_{\text{IO}}$ using Theorem 5.7 of [12].

We have already seen that $D_i B \subseteq D_i MT$ (cf. Corollary 4.22). Note that there is no similar result for deterministic transducers.

5.18. COROLLARY. $DB \not\subseteq DMT_{\text{IO}}$.

Proof. Immediate from Corollary 5.6 and the fact that $\text{dom}(DB) = \text{RECOG}$ [12]. ■

5.3. IO-Macro Tree Transducers with Regular Look-ahead

In this section we consider the translation class MT_{IO}^R (see Definition 4.17), and investigate also the deterministic case. Moreover for DMT_{IO} decomposition results are provided.

First we want to show that adding regular look-ahead to IO-macro tree transducers does not increase their transformational power. To model the regular look-

ahead we use the property (B2) of IO-macro tree transducers that was discussed in Section 5.2 and illustrated in Example 3.10. The “checking-mechanism,” which is formalized in Definition 4.17 by means of a deterministic bottom-up finite tree automaton, is now implemented by a nondeterministic top-down tree transducer (computing the partial identity), embedded in an IO-macro tree transducer. This is possible because $\text{dom}(DB\text{-FTA}) = \text{dom}(T\text{-FTA})$, cf. [13].

5.19. THEOREM. $MT_{IO}^R = MT_{IO}$.

Proof. It clearly suffices to show $MT_{IO}^R \subseteq MT_{IO}$. Let $M = (Q, P, \Sigma, \Delta, q^{\text{in}}, R, \delta) \in MT_a^R$. Construct $M' = (Q', \Sigma, \Delta, q^{\text{in}}, R') \in MT_a$ by $Q' = Q \cup P \cup R$, where every state in P has rank 1, a state $r \in R$ has rank $m + 2$ if the rule r has a left-hand side of the form $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n)$ for some $n \geq 0$. R' contains the following rules:

- (i) If $(q(a, y_1, \dots, y_n) \rightarrow t, \langle \rangle)$ is in R with $a \in \Sigma_0$, then $q(a, y_1, \dots, y_n) \rightarrow t$ is in R' .
- (ii) If $r = (q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle)$ is in R with $\sigma \in \Sigma_m$, $m \geq 1$, and $p_1, \dots, p_m \in P$, then $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow r(x_1, t, p_1(x_1), \dots, p_m(x_m))$ is in R' , and for every $\sigma \in \Sigma_m$ with $m \geq 0$, $r(\sigma(x_1, \dots, x_m), y_0, y_1, \dots, y_m) \rightarrow y_0$ is in R' .
- (iii) If $\delta_\sigma(p_1, \dots, p_m) = p$ for some $\sigma \in \Sigma_m$ with $m \geq 0$ and $p, p_1, \dots, p_m \in P$, then $p(\sigma(x_1, \dots, x_m)) \rightarrow \sigma(p_1(x_1), \dots, p_m(x_m))$ is in R' .

First we claim that, for all $s \in T_\Sigma$ and $p \in P$, $p(s) \Rightarrow_{M', IO}^* s$ iff $\delta(s) = p$. This is proved by straightforward induction on the structure of s . It implies that

$$\text{for all } s \in T_\Sigma \text{ and } p \in P, \quad M'_{IO}(p(s)) \neq \emptyset \quad \text{iff} \quad \delta(s) = p. \quad (*)$$

Now we prove the following two statements by simultaneous induction on s and π (in which the derivation mode is omitted):

- (1) For all $s \in T_\Sigma$ and $q \in Q_{n+1}$ with $n \geq 0$,

$$M(q(s, y_1, \dots, y_n)) = M'(q(s, y_1, \dots, y_n)).$$

- (2) For all $\pi \in T_\Sigma^m$ with $m \geq 0$ and all $t \in \text{RHS}(Q, \Delta, m, n)$, $n \geq 0$,

$$M_\pi(t) = M'_\pi(t).$$

If $q = q^{\text{in}}$, (1) implies that $\tau(M) = \tau(M')$ and hence $MT_{IO}^R \subseteq MT_{IO}$:

- (a) $M(q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n))$
 $= \bigcup \{ M_\pi(t) \mid r \in R \text{ and for all } i \in [m], \delta(s_i) = p_i \}, \quad \text{where}$
 $r = (q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle) \text{ and } \pi = (s_1, \dots, s_m),$
 $= \bigcup \{ M_\pi(t) \mid r \in R \text{ and for all } i \in [m], M'(p_i(s_i)) \neq \emptyset \} \quad \text{by } (*),$
 $= \bigcup \{ \{ y_0 \} \xleftarrow{IO} (M_\pi(t), M'_\pi(p_1(x_1)), \dots, M'_\pi(p_m(x_m))) \mid r \in R \} \quad \text{by}$
 $\text{definition of IO-substitution,}$

$$\begin{aligned}
&= \bigcup \{ M'_\pi(y_0) \xleftarrow{\text{IO}} (M'_\pi(t), M'_\pi(p_1(x_1)), \dots, M'_\pi(p_m(x_m))) \mid r \in R \} \\
&\quad \text{by (2),} \\
&= \bigcup \{ M'(r(s_1, y_0, \dots, y_m)) \xleftarrow{\text{IO}} (M'_\pi(t), M'_\pi(p_1(x_1)), \dots, M'_\pi(p_m(x_m))) \mid \\
&\quad r \in R \} \quad \text{by Definition 3.18,} \\
&= \bigcup \{ M'_\pi(r(x_1, t, p_1(x_1), \dots, p_m(x_m))) \mid r \in R \} \\
&= M'(q(\sigma(s_1, \dots, s_m), y_1, \dots, y_n)).
\end{aligned}$$

(b) The proof of the implication (1) $\Rightarrow$ (2) is an easy induction on the structure of the right-hand side t and is left to the reader. ■

There is no result corresponding to Theorem 5.19 for deterministic IO-macro tree transducers.

5.20. COROLLARY. $DMT_{\text{IO}} \not\subseteq DMT_{\text{IO}}^R$.

Proof. Immediate from Corollary 5.6 and the obvious fact that $\text{dom}(DMT_{\text{IO}}^R) = \text{RECOG}$. ■

Indeed, in the construction of Theorem 5.19 we need nondeterminism to recognize (in a top-down fashion) the language that is expressed by a bottom-up finite tree automaton. Thus, deterministic IO-macro tree transducers have only a “deterministic” (top-down) look-ahead. This is partially formalized in the following lemma. For the definition of top-down finite tree automaton look at the remark after Definition 3.2.

5.21. LEMMA. $DT\text{-FTA} \circ DMT_{\text{IO}} \subseteq DMT_{\text{IO}}$.

Proof. Consider $M_1 \in DT\text{-FTA}_a$ and $M_2 \in DMT_a$. Now we can apply the same construction idea as in Theorem 5.19 to obtain $M_3 \in DMT_a$ such that $\tau(M_1) \circ \tau_{\text{IO}}(M_2) = \tau_{\text{IO}}(M_3)$. However, it is sufficient to apply the construction only to the initial rules of M_1 (these are the rules in which the left-hand side is headed by the initial state), because we only have to “check” the domain of M_1 rather than to control by means of a regular look-ahead on subtrees the application of rules throughout the computation (which is the behavior of an IO-macro tree transducer with regular look-ahead). The formal construction is obvious and, besides, the proof is left to the reader. ■

Using a decomposition result for deterministic top-down tree transducers, we provide characterizations of deterministic IO-macro tree transducers.

5.22. LEMMA. $DT \subseteq DT\text{-FTA} \circ D, T$.

Proof. For every $M = (Q, \Sigma, A, q^{\text{in}}, R) \in DT_a$ there is an $N = (Q_N, \Sigma, \Sigma, q_N, R_N) \in DT\text{-FTA}_a$ such that $\text{dom}(M) = \text{dom}(N)$, or, equivalently,

$$\tau(N) = \text{dom}(M) \times \text{dom}(M) \quad (18)$$

(cf. Theorem 3.1 in [14]). Construct $M' = (Q, \Sigma, \Delta, q^{\text{in}}, R') \in D_i T_a$ by completion of the rule set R , i.e., let R' contain all rules in R and if $q(\sigma(x_1, \dots, x_k))$ for $q \in Q$ and $\sigma \in \Sigma_k$ with $k \geq 0$ is no left-hand side of a rule in R , then the rule $q(\sigma(x_1, \dots, x_k)) \rightarrow a$ is in R' for an arbitrary symbol $a \in \Delta_0$.

Now let $(s, t) \in \tau(M)$. Then, by (18), $(s, s) \in \tau(N)$, and since M' is an extension of M , $(s, t) \in \tau(M')$. If, on the other hand, $(s, s) \in \tau(N)$ and $(s, t) \in \tau(M')$, then there is a t_1 such that $(s, t_1) \in \tau(M)$.

According to the construction of $M' (\Rightarrow_M \subseteq \Rightarrow_{M'})$, we have $(s, t_1) \in \tau(M')$. Hence, since M' is deterministic, $t_1 = t$ and so $(s, t) \in \tau(M)$. Thus, $\tau(M) = \tau(N) \circ \tau(M')$. ■

As immediate consequences we obtain two decompositions of deterministic IO-macro tree transducers.

5.23. THEOREM. $DMT_{\text{IO}} = DT\text{-}FTA \circ D_i MT$.

Proof.

$$\begin{aligned}
 DMT_{\text{IO}} &\subseteq DT \circ \text{YIELD} && \text{by Lemma 5.5,} \\
 &\subseteq DT\text{-}FTA \circ D_i T \circ \text{YIELD} && \text{by Lemma 5.22,} \\
 &= DT\text{-}FTA \circ D_i MT && \text{by Corollary 5.9,} \\
 &\subseteq DT\text{-}FTA \circ DMT_{\text{IO}} \\
 &\subseteq DMT_{\text{IO}} && \text{by Lemma 5.21.} \quad \blacksquare
 \end{aligned}$$

5.24. THEOREM. $DMT_{\text{IO}} = DT \circ \text{YIELD}$.

Proof. The proof is the same as for Theorem 5.23. ■

The reader should compare this result with Theorems 4.8 and 5.12.

6. OI-MACRO TREE TRANSDUCERS

We characterize the class MT_{OI} by means of a composition of $D_i MT$ and a simple relation class SET modelling nondeterminism. Extending this characterization to OI-macro tree transducers with regular look-ahead, we can show that the look-ahead feature does not increase the transformational power.

6.1. Decomposition of MT_{OI}

Here we model the transformation of an OI-macro tree transducer M (with output alphabet Ω) by a total deterministic macro tree transducer N whose output trees are interpreted by a function, called set_Ω . The construction involved is quite similar to the simulation of “nondeterministic” context-free tree grammars (using OI derivation mode) by “deterministic” ones, which have the property that for each

nonterminal there is exactly one right-hand side (cf. Sect. 5 in [20] or Theorem 7.2 in [9]). The nondeterminism and partialness of M are encoded in the right-hand sides of the rules of N using two new symbols, e.g., $+$ and θ , respectively. In an output tree of N these symbols are interpreted by application of set_Ω as nondeterminism and partialness, i.e., union and the empty set, respectively. Thus, for every output tree t of N , $\text{set}_\Omega(t)$ is a finite set of trees over Ω (Definition 6.1).

Since all results of this section can be transferred to total and deterministic macro tree transducers, we have to find for these cases the functions corresponding to set_Ω . For the total case we define the function plus_Ω that interprets only the $+$ symbol as nondeterminism and for the deterministic case we define the function empty_Ω that interprets only the θ symbol as partialness (Definition 6.2). Often it will be obvious how to modify constructions and proofs in this section when restricting to the deterministic or the total case. Hence, sometimes the results concerning these restricted classes are only stated.

Before proving the characterization of MT_{OI} (Theorem 6.10), we have to start with the definition of the function set_Ω . Then Theorem 6.10 is prepared by the central Lemma 6.8 which compares the translations of an $N \in D_t MT_a$ and its “interpretation” (Definition 6.5), i.e., an $M \in MT_a$.

6.1. DEFINITION. Let Ω be a ranked alphabet and let $+$ and θ be new symbols ($+$, $\theta \notin \Omega$) of rank 2 and 0, respectively. Let $\mathcal{A} = \Omega \cup \{+, \theta\}$. Then define the mapping $\text{set}_{\Omega,+, \theta}: T_{\mathcal{A}} \rightarrow \mathcal{P}(T_\Omega)$ inductively. Let set abbreviate $\text{set}_{\Omega,+, \theta}$. Then (i) $\text{set}(\theta) = \emptyset$, (ii) for $\sigma \in \Omega_k$ with $k \geq 0$ and $t_1, \dots, t_k \in T_{\mathcal{A}}$, $\text{set}(\sigma(t_1, \dots, t_k)) = \{\sigma(s_1, \dots, s_k) \mid s_i \in \text{set}(t_i) \text{ for all } i \in [k]\}$, and (iii) for $t_1, t_2 \in T_{\mathcal{A}}$, $\text{set}(+(t_1, t_2)) = \text{set}(t_1) \cup \text{set}(t_2)$.

The translation induced by $\text{set}_{\Omega,+, \theta}$ is defined, as usual, by $\tau(\text{set}_{\Omega,+, \theta}) = \{(s, t) \in T_{\Omega \cup \{+, \theta\}} \times T_\Omega \mid t \in \text{set}_{\Omega,+, \theta}(s)\}$.

The class of translations induced by set-mappings, denoted by SET, is the set $\{\tau(\text{set}_{\Omega,+, \theta}) \mid \Omega \text{ ranked alphabet, } +, \theta \notin \Omega\}$.

Similarly we can define the functions plus_Ω and empty_Ω that interpret only non-determinism and partialness, respectively.

6.2. DEFINITION. Let Ω , $+$, and θ be as in Definition 6.1. Let $\Pi = \Omega \cup \{+\}$ and $\Gamma = \Omega \cup \{\theta\}$. Then the mappings $\text{plus}_{\Omega,+}: T_\Pi \rightarrow \mathcal{P}(T_\Omega)$ and $\text{empty}_{\Omega,\theta}: T_\Gamma \rightarrow \mathcal{P}(T_\Omega)$ are determined by $\text{plus}_{\Omega,+} = \text{set}_{\Omega,+, \theta} \upharpoonright T_\Pi$ and $\text{empty}_{\Omega,\theta} = \text{set}_{\Omega,+, \theta} \upharpoonright T_\Gamma$, respectively (where for a function $f: A \rightarrow B$ and a subset $C \subseteq A$, $f \upharpoonright C$ denotes the restriction of f to C in the usual sense).

The translation classes corresponding to plus_Ω and empty_Ω are denoted by PLUS and EMPTY, respectively, and are defined similarly to SET. The class SET is very simple; it can be obtained by a (1-state) linear top-down tree transducer.

6.3. LEMMA. $\text{SET} \subseteq LT$.

Proof. Let $\tau(\text{set}_{\Omega,+, \theta}) \in \text{SET}$ with $+, \theta \notin \Omega$. Let $\mathcal{A} = \Omega \cup \{+, \theta\}$. Construct $M = (\{q\}, \mathcal{A}, \Omega, q, R) \in LT_a$ such that R contains for all $\sigma \in \Omega_k$ with $k \geq 0$ the rule

$q(\sigma(x_1, \dots, x_k)) \rightarrow \sigma(q(x_1), \dots, q(x_k))$, and furthermore the rules $q(+ (x_1, x_2)) \rightarrow q(x_1) | q(x_2)$. (Note that there is no (q, θ) -rule in R .)

It is an easy proof by induction on s that for all $s \in T_A$,

$$\text{set}_{\Omega, +, \theta}(s) = M(q(s)). \quad \blacksquare$$

By obvious modifications of the construction in Lemma 6.3 we obtain similar results for PLUS and EMPTY.

6.4. LEMMA. $\text{PLUS} \subseteq L_i T$ and $\text{EMPTY} \subseteq LDT$.

The central lemma is based on the following definition, which connects a total deterministic macro tree transducer N and a macro tree transducer M . M is called the $(+, \theta)$ -interpretation of N , if the symbols $+$ and θ in the right-hand sides of rules in N are treated as nondeterminism and partialness in M , respectively. Thus, it is to be expected that $\tau_{OI}(M) = \tau(N) \circ \tau(\text{set}_{\Omega, +, \theta})$.

6.5. DEFINITION. Let $N = (Q_N, \Sigma, A, q_N^{\text{in}}, R_N) \in D_i MT_a$ and $M = (Q_M, \Sigma, \Omega, q_M^{\text{in}}, R_M) \in MT_a$. Let $+$ and θ be symbols of rank 2 and 0, respectively. Then, M is the $(+, \theta)$ -interpretation of N , if

- (i) $Q_M = Q_N \cup \{q_+^{(3)}, q_\theta^{(1)}\}$ and $q_M^{\text{in}} = q_N^{\text{in}}$,
- (ii) $\Omega = A - \{+, \theta\}$, and
- (iii) $R_M = \{q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t' \mid q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t \text{ is in } R_N \text{ and } t' \in \text{tr}(+, \theta, m, n)(t)\} \cup \{q_+(\sigma(x_1, \dots, x_m), y_1, y_2) \rightarrow y_1, q_\theta(\sigma(x_1, \dots, x_m), y_1, y_2) \rightarrow y_2 \mid \sigma \in \Sigma_m \text{ with } m \geq 0\}$.

For every $m, n \geq 0$ the mapping $\text{tr}(+, \theta, m, n): \text{RHS}(Q_N, A, m, n) \rightarrow \mathcal{P}(\text{RHS}(Q_M, \Omega, m, n))$ is defined inductively. Abbreviate $\text{tr}(+, \theta, m, n)$ and $\text{RHS}(Q_N, A, m, n)$ by tr and RHS , respectively:

- (i) $\text{tr}(\theta) = \emptyset$.
- (ii) For $j \in [n]$, $\text{tr}(y_j) = \{y_j\}$.
- (iii) For $\delta \in \Omega_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\text{tr}(\delta(\xi_1, \dots, \xi_k)) = \{\delta(\xi_1, \dots, \xi_k) \mid \xi_i \in \text{tr}(\xi_i) \text{ for all } i \in [k]\}$.
- (iv) For $\xi_1, \xi_2 \in \text{RHS}$, $\text{tr}(+(\xi_1, \xi_2)) = \text{tr}(\xi_1) \cup \text{tr}(\xi_2)$.
- (v) For $q \in Q_N$ of rank $k+1$, $k \geq 0$, $x_j \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\text{tr}(q(x_j, \xi_1, \dots, \xi_k)) = \{q(x_j, \text{par-tr}(\xi_1), \dots, \text{par-tr}(\xi_k))\}$.

For every $m \geq 1$ and $n \geq 0$ the mapping $\text{par-tr}(+, \theta, m, n): \text{RHS}(Q_N, A, m, n) \rightarrow \text{RHS}(Q_M, \Omega, m, n)$ is defined inductively (using two obvious abbreviations):

- (i) $\text{par-tr}(\theta) = q_\theta(x_1)$.
- (ii) For $j \in [n]$, $\text{par-tr}(y_j) = y_j$.
- (iii) For $\delta \in \Omega_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\text{par-tr}(\delta(\xi_1, \dots, \xi_k)) = \delta(\text{par-tr}(\xi_1), \dots, \text{par-tr}(\xi_k))$.

(iv) For $\xi_1, \xi_2 \in \text{RHS}$, $\text{par-tr}(+(\xi_1, \xi_2)) = q_+(x_1, \text{par-tr}(\xi_1), \text{par-tr}(\xi_2))$.

(v) For $q \in \mathcal{Q}_N$ of rank $k+1$ ($k \geq 0$), $x_j \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$, $\text{par-tr}(q(x_j, \xi_1, \dots, \xi_k)) = q(x_j, \text{par-tr}(\xi_1), \dots, \text{par-tr}(\xi_k))$.

A small example illustrates this definition.

6.6. EXAMPLE. Let $N = (\{q^{\text{in}}, p^{(3)}\}, \Sigma, \Delta, q^{\text{in}}, R_N) \in D_t MT_a$ with $\Sigma = \{\sigma^{(1)}, a^{(0)}\}$ and $\Delta = \{\delta^{(2)}, b^{(0)}, c^{(0)}, +^{(2)}, \theta^{(0)}\}$ and let R_N contain the rules

$$q^{\text{in}}(\sigma(x)) \rightarrow +(p(x, +(c, b), \theta), c),$$

$$q^{\text{in}}(a) \rightarrow +(c, +(b, \theta)),$$

$$p(\sigma(x), y_1, y_2) \rightarrow \delta(y_1, y_1),$$

$$p(a, y_1, y_2) \rightarrow \theta.$$

Let $M = (\{q^{\text{in}}, p^{(3)}, q_+^{(3)}, q_\theta^{(1)}\}, \Sigma, \Omega, q^{\text{in}}, R_M) \in MT_a$, where $\Omega = \{\delta^{(2)}, b^{(0)}, c^{(0)}\}$ and R_M contains the rules

$$q^{\text{in}}(\sigma(x)) \rightarrow p(x, q_+(x, c, b), q_\theta(x))|c,$$

$$q^{\text{in}}(a) \rightarrow c|b,$$

$$p(\sigma(x), y_1, y_2) \rightarrow \delta(y_1, y_1)$$

$$q_+(\sigma(x), y_1, y_2) \rightarrow y_1|y_2$$

$$q_+(a, y_1, y_2) \rightarrow y_1|y_2.$$

It is easy to check that M is the $(+, \theta)$ -interpretation of N . For the input-tree $\sigma^2 a$, N computes the output tree $+(\delta(+(c, b), +(c, b)), c)$. Applying $\text{set}_{\Omega, +, \theta}$ to this tree we obtain $S = \{\delta(c, c), \delta(c, b), \delta(b, c), \delta(b, b), c\}$. M computes for the input tree $\sigma^2 a$ the same set, i.e., $q^{\text{in}}(\sigma^2 a) \downarrow_M = S$; in general $\tau_{\text{OI}}(M) = \tau(N) \circ \tau(\text{set}_{\Omega, +, \theta})$.

Note that the function par-tr is necessary, because $+$ and θ in a parameter position of a state have to be modelled by auxiliary states (q_+ and q_θ , respectively) in order to achieve the correct order of copying followed by nondeterminism and deletion followed by partialness. The reason for having tr besides par-tr and not modelling everything only with the mapping par-tr is justified by the fact that if an input symbol of rank 0 is read, no “dummy” input-variable for the auxiliary states is available. The division of Definition 6.5 (first interpret $+$ and θ directly by non-determinism and partialness, respectively, i.e., applying tr , and then, once a parameter position of a state in $\mathcal{Q}_N$ is reached, inserting auxiliary states, i.e., applying par-tr) offers a simple construction in the proof of $MT_{\text{OI}} \subseteq D_t MT \circ \text{SET}$ (Theorem 6.10). Nevertheless, the next lemma shows that M behaves in the “same way” on the images of both functions par-tr and tr .

6.7. LEMMA. Let N and M be as in Definition 6.5 and M be the $(+, \theta)$ -interpretation of N . Let $m \geq 1$. Then, for all $t \in \text{RHS}(Q_N, \Delta, m, n)$, $\pi \in T_{\Sigma}^m$, and $\rho \in \mathcal{P}(T_{\Omega})^n$,

$$(\text{par-tr}(t))\langle \downarrow, \pi, \rho \rangle_M = \bigcup \{t' \langle \downarrow, \pi, \rho \rangle_M \mid t' \in \text{tr}(t)\}.$$

Proof. The proof is done by straightforward induction on the structure of t . We only mention two typical cases, namely (iii) and (iv) of Definition 6.5. Note that $m \geq 1$. Furthermore note that case (v) is trivial. For the rest of this proof abbreviate $\langle \downarrow, \pi, \rho \rangle_M$ by $\langle \rangle$.

(iii) Let $\delta \in \Omega_k$ with $k \geq 0$ and $\xi_1, \dots, \xi_k \in \text{RHS}$.

$$\begin{aligned} & (\text{par-tr}(\delta(\xi_1, \dots, \xi_k)))\langle \rangle \\ &= (\delta(\text{par-tr}(\xi_1), \dots, \text{par-tr}(\xi_k)))\langle \rangle && \text{by Definition 6.5,} \\ &= \{\delta(\xi_1, \dots, \xi_k) \mid \zeta_i \in \text{par-tr}(\xi_i)\langle \rangle \text{ for all } i \in [k]\} && \text{by Definition 3.16,} \\ &= \{\delta(\xi_1, \dots, \xi_k) \mid \zeta_i \in \bigcup \{t'_i \langle \rangle \mid t'_i \in \text{tr}(\xi_i)\} \text{ for all } i \in [k]\} && \text{by I.H.,} \\ &= \bigcup \{\{\delta(\xi_1, \dots, \xi_k) \mid \zeta_i \in t'_i \langle \rangle \mid t'_i \in \text{tr}(\xi_i) \text{ for all } i \in [k]\} \\ &= \bigcup \{\delta(t'_1, \dots, t'_k) \langle \rangle \mid t'_i \in \text{tr}(\xi_i) \text{ for all } i \in [k]\} \\ &= \bigcup \{t' \langle \rangle \mid t' \in \text{tr}(\delta(\xi_1, \dots, \xi_k))\}. \end{aligned}$$

(iv) Let $\xi_1, \xi_2 \in \text{RHS}$. Then

$$\begin{aligned} & (\text{par-tr}(+(\xi_1, \xi_2)))\langle \rangle \\ &= q_+(x_1, \text{par-tr}(\xi_1), \text{par-tr}(\xi_2))\langle \rangle \\ &= q_+(s_1, \text{par-tr}(\xi_1)\langle \rangle, \text{par-tr}(\xi_2)\langle \rangle)\downarrow_M \quad \text{where } s_1 = \pi(1), \\ & \quad \text{by Definition 3.16,} \\ &= y_1 \langle \downarrow, \pi', \rho' \rangle_M \cup y_2 \langle \downarrow, \pi', \rho' \rangle_M, \end{aligned}$$

where π' is the sequence of sons of s_1 and $\rho' = (\text{par-tr}(\xi_1)\langle \rangle, \text{par-tr}(\xi_2)\langle \rangle) \in \mathcal{P}(T_{\Omega})^2$,

$$\begin{aligned} &= \text{par-tr}(\xi_1)\langle \rangle \cup \text{par-tr}(\xi_2)\langle \rangle && \text{by Definition 3.16,} \\ &= \bigcup \{t'_1 \langle \rangle \mid t'_1 \in \text{tr}(\xi_1)\} \cup \bigcup \{t'_2 \langle \rangle \mid t'_2 \in \text{tr}(\xi_2)\} && \text{by I.H.,} \\ &= \bigcup \{t' \langle \rangle \mid t' \in \text{tr}(+(\xi_1, \xi_2))\}. \end{aligned}$$

■

Now we are in a position to prove the central lemma.

6.8. LEMMA. Let N and M be as in Definition 6.5 and let M be the $(+, \theta)$ -interpretation of N . Then, $\tau_{\text{OI}}(M) = \tau(N) \circ \tau(\text{set}_{\Omega, +, \theta})$.

Proof. We abbreviate $\text{set}_{\Omega, +, \theta}$ by set and use all other already mentioned abbreviations concerning the translation functions of Definition 6.5. The following

statements (1) and (2) are proved by simultaneous induction on s and π . Taking $q = q_N^{\text{in}}$, (1) is the statement of the lemma:

(1) For all $s \in T_{\Sigma}$, $q \in Q_N$ of rank $n + 1$ with $n \geq 0$, and $t_1, \dots, t_n \in T_A$, $\text{set}(q(s, t_1, \dots, t_n) \uparrow_N) = q(s, \text{set}(t_1), \dots, \text{set}(t_n)) \downarrow_M$.

(2) For all $\pi \in T_{\Sigma}^m$ with $m \geq 0$, $t \in \text{RHS}$ with $n \geq 0$, $\rho \in T_A^n$, and $\rho' \in \mathcal{P}(T_{\Omega})^n$, if $\rho'(i) = \text{set}(\rho(i))$ for all $i \in [n]$, then $\text{set}(t \langle \uparrow, \pi, \rho \rangle_N) = \bigcup \{t' \langle \downarrow, \pi, \rho' \rangle_M \mid t' \in \text{tr}(t)\}$.

Note that $\uparrow_N$ and $\langle \uparrow, \pi, \rho \rangle_N$ are total functions.

The proof of (2) $\Rightarrow$ (1) is easy and left to the reader. (1) $\Rightarrow$ (2) is proved, as usual, by induction on the structure of t . Let $\pi = (s_1, \dots, s_m)$, $\rho = (t_1, \dots, t_n)$, and $\rho' = (\text{set}(t_1), \dots, \text{set}(t_n))$. We only work out the case in which t is of the form $q(x_i, \xi_1, \dots, \xi_k)$ with $q \in Q_N$ of rank $k + 1$ ($k \geq 0$), $x_i \in X_m$, and $\xi_1, \dots, \xi_k \in \text{RHS}$. Note that $m \geq 1$. Abbreviate $\langle \uparrow, \pi, \rho \rangle_N$ and $\langle \downarrow, \pi, \rho' \rangle_M$ by $\langle N \rangle$ and $\langle M \rangle$, respectively:

$$\begin{aligned}
 & \text{set}(q(x_i, \xi_1, \dots, \xi_k) \langle N \rangle) \\
 &= \text{set}(q(s_i, \xi_1 \langle N \rangle, \dots, \xi_k \langle N \rangle) \uparrow_N) && \text{by Definition 3.14,} \\
 &= q(s_i, \text{set}(\xi_1 \langle N \rangle), \dots, \text{set}(\xi_k \langle N \rangle)) \downarrow_M && \text{by (1),} \\
 &= q(s_i, \bigcup \{t' \langle M \rangle \mid t' \in \text{tr}(\xi_1)\}, \dots, \bigcup \{t' \langle M \rangle \mid t' \in \text{tr}(\xi_k)\}) \downarrow_M && \text{by I.H.,} \\
 &= q(s_i, \text{par-tr}(\xi_1) \langle M \rangle, \dots, \text{par-tr}(\xi_k) \langle M \rangle) \downarrow_M && \text{by Lemma 6.7,} \\
 &= q(x_i, \text{par-tr}(\xi_1), \dots, \text{par-tr}(\xi_k)) \langle M \rangle && \text{by Definition 3.16,} \\
 &= \text{par-tr}(q(x_i, \xi_1, \dots, \xi_k)) \langle M \rangle && \text{by Definition 6.5,} \\
 &= \bigcup \{t' \langle M \rangle \mid t' \in \text{tr}(q(x_i, \xi_1, \dots, \xi_k))\} && \text{by Lemma 6.7. } \blacksquare
 \end{aligned}$$

Similar to Definition 6.5 we can define for $N \in D_i MT_a$ and $M \in {}_i MT_a$ the $+$ -interpretation and for $N \in D_i MT_a$ and $M \in DMT_a$ the θ -interpretation. Roughly speaking, we only have to omit θ and $+$, respectively. For these modified definitions we obtain a lemma similar to the previous one.

6.9. LEMMA. *Let M and N be as in Definition 6.5 and furthermore let M be total (or deterministic). Let M be the $+$ -(or θ -) interpretation of N (respectively). Then, $\tau_{\text{OI}}(M) = \tau(N) \circ \tau(\text{plus}_{\Omega,+})$ (or $\tau_{\text{OI}}(M) = \tau(N) \circ \tau(\text{empty}_{\Omega,\theta})$, respectively).*

Proof. The proof can be obtained by an obvious modification of the proof of Lemma 6.8. $\blacksquare$

Using these lemmata we can prove the announced characterization.

6.10. THEOREM.

$$\begin{aligned}
 MT_{\text{OI}} &= D_i MT \circ \text{SET}, \\
 {}_i MT_{\text{OI}} &= D_i MT \circ \text{PLUS}, \\
 DMT_{\text{OI}} &= D_i MT \circ \text{EMPTY}.
 \end{aligned}$$

Proof. We only prove the first equality. By obvious modifications we obtain proofs for the other two.

“ $\subseteq$.” Let $M = (Q_M, \Sigma, \Omega, q_M^{\text{in}}, R_M) \in MT_a$. Add two dummy states $q_+^{(3)}$ and $q_\theta^{(1)}$ to Q_M and the rules $q_+(\sigma(x_1, \dots, x_m), y_1, y_2) \rightarrow y_1 | y_2$ for every $\sigma \in \Sigma_m$ with $m \geq 0$ to R_M . We obtain $M' = (Q_M, \Sigma, \Omega, q_M^{\text{in}}, R'_M)$. Obviously, $\tau_{\text{OI}}(M) = \tau_{\text{OI}}(M')$, because q_+ and q_θ do not occur in right-hand sides of rules in R_M .

Now construct $N = (Q_N, \Sigma, \Delta, q_N^{\text{in}}, R_N) \in D_t MT_a$ with $Q_N = Q_M$, $q_N^{\text{in}} = q_M^{\text{in}}$, and $\Delta = \Omega \cup \{ +^{(2)}, \theta^{(0)} \}$. Let R_N contain the following rules:

(i) If there are no (q, σ) -rules in R_M , where $q \in Q_M$ of rank $n+1$ ($n \geq 0$), and $\sigma \in \Sigma_m$ with $m \geq 0$, then $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow \theta$ is in R_N .

(ii) If $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t_1 | \dots | t_r$ are all (q, σ) -rules in R_M , where $q \in Q_M$ of rank $n+1$ ($n \geq 0$), $\sigma \in \Sigma_m$ with $m \geq 0$, and $t_1, \dots, t_r \in \text{RHS}(Q_M, \Omega, m, n)$ with $r \geq 1$ and,

if $r = 1$ then $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t_1$ is in R_N , and if $r \geq 2$ then $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow +(t_1, +(t_2, \dots, +(t_{r-1}, t_r) \dots))$ is in R_N .

Obviously, M' is the $(+, \theta)$ -interpretation of N . Note that $t_1, \dots, t_r$ in case (ii) do not contain $+$ or θ . Hence, $\text{tr}(t_i) = \{t_i\}$. From Lemma 6.8 it follows that $\tau_{\text{OI}}(M') = \tau(N) \circ \tau(\text{set}_{\Omega, +, \theta})$.

“ $\supseteq$.” Let $N = (Q_N, \Sigma, \Delta, q_N^{\text{in}}, R_N) \in D_t MT_a$ and $\tau(\text{set}_{\Omega, +, \theta}) \in \text{SET}$ with $\Omega = \Delta - \{+, \theta\}$. Construct $M = (Q_M, \Sigma, \Omega, q_M^{\text{in}}, R_M) \in MT_a$ such that M is the $(+, \theta)$ -interpretation of N (using Definition 6.5). Then, by Lemma 6.8, $\tau(N) \circ \tau(\text{set}_{\Omega, +, \theta}) = \tau_{\text{OI}}(M)$. ■

Since top-down tree transducers are special macro tree transducers, Definition 6.5 provides the notion of $(+, \theta)$ -interpretation also for top-down tree transducers (just by omitting the auxiliary states q_+ and q_θ). For this transducer class a lemma similar to Lemma 6.8 can be shown by disregarding the cases in which parameters are involved. (Note that no lemma corresponding to Lemma 6.7 is needed, because $\text{tr}(q(x_i)) = \{q(x_i)\}$.) Using constructions similar to those in Theorem 6.10, we obtain a characterization for top-down tree transducers.

6.11. COROLLARY.

$$T = D_t T \circ \text{SET},$$

$$_t T = D_t T \circ \text{PLUS},$$

$$DT = D_t T \circ \text{EMPTY}.$$

Proof. The proof is similar to that of Theorem 6.10. Note that the auxiliary states are not needed. This is due to the fact that $+$ and θ in the top part of a right-hand side, i.e., not in a parameter position, are interpreted by tr directly as non-determinism and partialness. ■

One immediate consequence of this characterization is the following composition result.

6.12. COROLLARY.

$$\begin{aligned} D_t MT \circ T &= MT_{OI}, \\ D_t MT \circ {}_t T &= {}_t MT_{OI}, \\ D_t MT \circ DT &= DMT_{OI}. \end{aligned}$$

Proof.

$$\begin{aligned} D_t MT \circ T &= D_t MT \circ D_t T \circ \text{SET} && \text{by Corollary 6.11,} \\ &= D_t MT \circ \text{SET} && \text{by Theorem 4.12,} \\ &= MT_{OI} && \text{by Theorem 6.10.} \end{aligned}$$

The proofs of the other equalities are analogous. ■

We finally note the following. In general, when constructing the $(+, \theta)$ -interpretation of a given $N \in D_t MT_a$ (as in the second part of the proof of Theorem 6.10), linearity is not preserved. However, if $+$ and θ do not occur in the parameter position of a state, then linearity is preserved. This holds in particular in case N corresponds to a YIELD-mapping (see Example 4.5). Hence, $\text{YIELD} \circ \text{SET} \subseteq LMT_{OI}$. Let QT denote the class of quoted tree languages (obtained by generalizing the quoted macro languages of [23] to trees). It is shown in [33] that $QT = \text{SET}(IOT)$. Hence, we obtain the following result.

6.13. COROLLARY. $QT \subseteq LMT_{OI}(\text{RECOG})$.

Proof. $QT = \text{SET}(IOT) = \text{SET}(\text{YIELD}(\text{RECOG})) \subseteq LMT_{OI}(\text{RECOG})$. ■

6.2. OI-Macro Tree Transducers with Regular Look-ahead

Extending the characterization of OI-macro tree transducers (Theorem 6.10) to OI-macro tree transducers with regular look-ahead, and using the fact that total deterministic macro tree transducers have the regular look-ahead ability (Theorem 4.21), we can prove that OI-macro tree transducers also have that ability (Theorem 6.15). Actually, for deterministic macro tree transducers using OI-derivation mode the same holds, i.e., $DMT_{OI}^R = DMT_{OI}$. Thus, whilst deterministic IO-macro tree transducers have only a “deterministic” top-down regular look-ahead (Theorem 5.23), deterministic OI-macro tree transducers possess the “full” regular look-ahead ability (Theorem 6.15).

First, we characterize OI-macro tree transducers with regular look-ahead (Definition 4.17) similar to the ones without (Theorem 6.10). The notion of $(+, \theta)$ -interpretation can be easily transferred to the extended transducers. For this pur-

pose let both sets of regular look-ahead states (from $M \in MT_a^R$ and $N \in D_t MT_a^R$) coincide, i.e., $P_M = P_N$, let $\delta_N = \delta_M$ and let R_M be the rule set

$$\begin{aligned} & \{ (q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t', \langle p_1, \dots, p_m \rangle) | \\ & \quad (q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle) \text{ is in } R_N \text{ and } t' \\ & \quad \in \text{tr}(+, \theta, m, n)(t) \} \cup \{ (q_+(\sigma(x_1, \dots, x_m), y_1, y_2) \\ & \quad \rightarrow y_1 | y_2, \langle p_1, \dots, p_m \rangle) | \\ & \quad \sigma \in \Sigma_m \text{ with } m \geq 0 \text{ and } p_1, \dots, p_m \in P_M \}. \end{aligned}$$

Similarly to Lemma 6.8, a lemma for this extension can be shown by means of which the characterization of MT_{OI}^R can be proved.

6.14. THEOREM.

$$\begin{aligned} MT_{OI}^R &= D_t MT^R \circ \text{SET}, \\ {}_t MT_{OI}^R &= D_t MT^R \circ \text{PLUS}, \\ DMT_{OI}^R &= D_t MT^R \circ \text{EMPTY}. \end{aligned}$$

Proof. “ $\subseteq$.” We extend an $M = (Q_M, P_M, \Sigma, \Omega, q_M^{\text{in}}, R_M, \delta_M) \in MT_a^R$ with two dummy states $q_+^{(3)}$ and $q_\theta^{(1)}$ and the rules $(q_+(\sigma(x_1, \dots, x_m), y_1, y_2) \rightarrow y_1 | y_2, \langle p_1, \dots, p_m \rangle)$ for every $\sigma \in \Sigma_m$ with $m \geq 0$ and $p_1, \dots, p_m \in P_M$ and we obtain M' such that $\tau_{OI}(M') = \tau_{OI}(M)$.

The rule set for $N = (Q_N, P_N, \Sigma, A, q_N^{\text{in}}, R_N, \delta_N) \in D_t MT_a^R$ is defined similarly as in the proof of Theorem 6.10, but the case analysis now depends on q, σ , and a sequence of look-ahead states rather than only on q and σ . Note that in case (ii), $r \geq 2$, we consider all (q, σ) -rules $(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t_1 | \dots | t_r, \langle p_1, \dots, p_m \rangle)$ with the same look-ahead states. These can clearly be matched into one rule for N . With the extension of Lemma 6.8 the inclusion “ $\subseteq$ ” is proved, because M' is the $(+, \theta)$ -interpretation of N .

“ $\supseteq$.” The construction of $M = (Q_M, P_M, \Sigma, \Omega, q_M^{\text{in}}, R_M, \delta_M) \in MT_a^R$ follows directly the modified Definition 6.5. ■

6.15. THEOREM. $MT_{OI}^R = MT_{OI}$, ${}_t MT_{OI}^R = {}_t MT_{OI}$, and $DMT_{OI}^R = DMT_{OI}$.

Proof. Immediate from Theorems 4.21, 6.10, and 6.14. ■

Since deterministic OI-macro tree transducers have the regular look-ahead ability, we can derive that deterministic bottom-up tree transductions are contained in DMT_{OI} (cf. Corollary 4.22, Theorem 5.16, and Corollary 5.18).

6.16. COROLLARY. $DB \subsetneq DMT_{OI}$.

Proof.

$$\begin{aligned}
 DB &\subseteq DT^R && \text{by Theorem 3.2 in [14],} \\
 &\subseteq DMT_{OI}^R && \text{by Proposition 3.6,} \\
 &= DMT_{OI} && \text{by Theorem 6.15. } \blacksquare
 \end{aligned}$$

Note that also $DT^R \subseteq DMT_{IO}^R$ holds, but $DMT_{IO} \subsetneq DMT_{IO}^R$ (Corollary 5.20). With respect to the regular look-ahead ability of DMT_{OI} it is reasonable to expect the closure of DMT_{OI} under left-composition with FTA (cf. Lemma 5.21).

6.17. LEMMA. $FTA \circ DMT_{OI} \subseteq DMT_{OI}$.

Proof. Let $B = (P, \Sigma, \delta, F) \in DB\text{-}FTA_a$, see Section 2.4, and $N = (Q, \Sigma, \Delta, q^{\text{in}}, R_N) \in DMT_a$. Now we can construct $M = (Q \cup \{q_M^{\text{in}}\}, P, \Sigma, \Delta, q_M^{\text{in}}, R_M, \delta) \in DMT_a^R$, where R_M contains the following rules:

(i) If $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t$ is in R_N with $q \in Q_{n+1}$, $n \geq 0$, and $\sigma \in \Sigma_m$ with $m \geq 0$, then for all $p_1, \dots, p_m \in P$ the rule $(q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow t, \langle p_1, \dots, p_m \rangle)$ is in R_M .

(ii) If $\delta_\sigma(p_1, \dots, p_m) = p \in F$ and $q^{\text{in}}(\sigma(x_1, \dots, x_m)) \rightarrow t$ is in R_N , then the rule $(q_M^{\text{in}}(\sigma(x_1, \dots, x_m)) \rightarrow t, \langle p_1, \dots, p_m \rangle)$ is in R_M .

Now it is easy to prove that $\tau(B) \circ \tau_{OI}(N) = \tau_{OI}(M)$, which proves the lemma, using Theorem 6.15. $\blacksquare$

Similarly to Theorem 5.23 we obtain a characterization of DMT_{OI} . For the proof we need the fact that the domain of DMT_{OI} is the class of recognizable tree languages, which is proved in Theorem 7.4(2).

6.18. THEOREM. $DMT_{OI} = FTA \circ D_t MT$.

Proof. We only have to prove $DMT_{OI} \subseteq FTA \circ D_t MT$; “ $\supseteq$ ” holds by Lemma 6.17. Note that $\text{dom}(DMT_{OI}) \subseteq \text{RECOG}$. Hence, for an $M = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in DMT_a$ there is an $N \in FTA_a$ such that $\text{dom}(M) = \text{dom}(N)$. (Note that $\tau(N) = \text{dom}(N) \times \text{dom}(N)$.) Then construct $M' = (Q, \Sigma, \Delta, q^{\text{in}}, R) \in D_t MT_a$ by completion of the rule set in the following sense (cf. Lemma 5.22): $R \subseteq R'$ and if for $q \in Q_{n+1}$ with $n \geq 0$ and $\sigma \in \Sigma_m$ with $m \geq 0$ there is no (q, σ) -rule in R , then the rule $q(\sigma(x_1, \dots, x_m), y_1, \dots, y_n) \rightarrow a$ is in R' for an arbitrary $a \in \Delta_0$. It should be obvious that for all $s \in \text{dom}(M)$, $q^{\text{in}}(s) \downarrow_M = q^{\text{in}}(s) \downarrow_{M'}$. Hence, $\tau_{OI}(M) = \tau(N) \circ \tau(M')$. $\blacksquare$

7. CONSEQUENCES

In this final section we collect a few consequences of the results of this paper, to show the differences and similarities between IO- and OI-macro tree transducers. We start with comparing the deterministic macro tree transducer classes.

7.1. THEOREM. (1) $D_iMT \subsetneq DMT_{IO} \subsetneq DMT_{OI} = DMT_{IO}^R$

(2) $\text{out}(D_iMT) = \text{out}(DMT_{IO}) = \text{out}(DMT_{OI}) = D_iMT(\text{RECOG}) = DMT_{IO}(\text{RECOG}) = DMT_{OI}(\text{RECOG})$.

Proof. The inclusions in (1) follow immediately from $DMT_{IO} = DT - \text{FTA} \circ D_iMT$ and $DMT_{OI} = \text{FTA} \circ D_iMT$ (Theorems 5.23 and 6.18, respectively). Furthermore, $DMT_{IO} \subseteq DMT_{OI}$ implies $DMT_{IO}^R \subseteq DMT_{OI}^R = DMT_{OI}$ (Theorem 6.15), and $DMT_{OI} = \text{FTA} \circ D_iMT$ implies $DMT_{OI} \subseteq \text{FTA} \circ DMT_{IO} \subseteq DMT_{IO}^R$ (by a proof similar to that of Lemma 6.17). This shows that $DMT_{OI} = DMT_{IO}^R$.

To prove (2), it suffices to show that $DMT_{OI}(\text{RECOG}) \subseteq \text{out}(D_iMT)$. Since $DMT_{OI} = \text{FTA} \circ D_iMT$ and RECOG is closed under intersection [38], $DMT_{OI}(\text{RECOG}) \subseteq D_iMT(\text{RECOG})$. It remains to show that $D_iMT(\text{RECOG}) \subseteq \text{out}(D_iMT)$. Clearly $\emptyset \in \text{out}(D_iMT)$: take an empty input alphabet. Now let $M \in D_iMT_a$, $L \in \text{RECOG}$, and assume $\tau(M)(L) \neq \emptyset$. Let t_0 be a fixed tree in $\tau(M)(L)$. Since D_iMT is closed under regular look-ahead (Theorem 4.21), there is an $M' \in D_iMT_a$ such that for every input tree s , $\tau(M')(s) = \tau(M)(s)$ if $s \in L$, and $\tau(M')(s) = t_0$ if $s \notin L$. Clearly, $\text{out}(\tau(M')) = \tau(M)(L)$. ■

Next we give, in Fig. 9, two diagrams which show inclusions of some of the classes we have considered. For the inclusion $\text{out}(B) \subseteq \text{IOT}$, see [12, 20]. To show the correctness of these diagrams (i.e., all inclusions are proper, and non-connected classes are incomparable), it remains to prove that the following classes are not empty:

- (1) $B - MT_{OI}$, $MT_{OI} - MT_{IO}$
- (2) $OIT - \text{out}(MT_{IO})$, $\text{out}(MT_{OI}) - \text{out}(MT_{IO})$, $QT - D_iMT(\text{RECOG})$, $D_iMT(\text{RECOG}) - QT$.

We leave this to the next paper.

One of our main efforts was to decompose macro tree transducers into simple translations. It follows from Theorems 4.8, 5.12, and 6.10 (together with $\text{res-}T = \text{res-}$

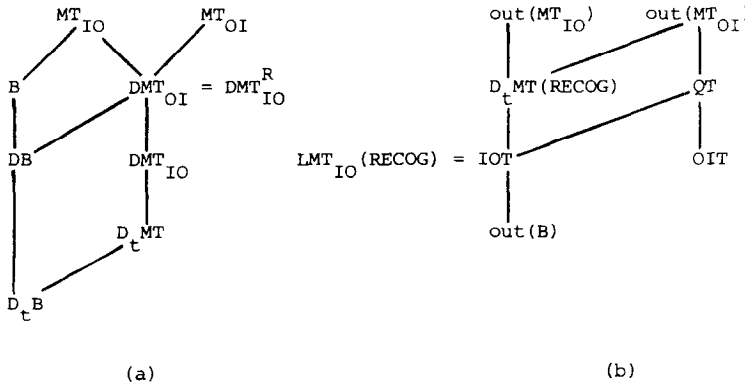


FIG. 9. Inclusion diagrams of classes of translations and their ranges.

$D_i T \circ \text{SET}$, cf. Corollary 6.11) that $MT_{OI} = D_i T \circ \text{YIELD} \circ \text{SET}$ and $MT_{IO} = \text{res-}D_i T \circ \text{SET} \circ \text{YIELD}$. This shows that the difference between IO and OI lies mainly in the order of application of YIELD and SET (cf. similar remarks in [20]). This is even more clear for the classes of ranges.

7.2. THEOREM. $\text{out}(MT_{OI}) = \text{SET}(\text{YIELD}(D_i T(\text{RECOG})))$, $\text{out}(MT_{IO}) = \text{YIELD}(\text{SET}(D_i T(\text{RECOG})))$.

Proof. The first equality follows from the facts that $MT_{OI} = D_i T \circ \text{YIELD} \circ \text{SET}$ and $MT_{OI}^R = MT_{OI}$. For the second equality, it follows from Theorem 5.15 that $\text{YIELD}(\text{SET}(D_i T(\text{RECOG}))) = MT_{IO}(\text{LHOM}(\text{RECOG}))$. Since RECOG is closed under linear tree homomorphisms and $MT_{IO}^R = MT_{IO}$, this class is equal to $MT_{IO}(\text{RECOG}) = \text{out}(MT_{IO})$. ■

We have decomposed the macro tree transducers into top-down tree transducers and relations in YIELD. This implies the following result, where, for a class of relations K , K^* denotes $\bigcup_n K^n$, i.e., the closure of K under composition.

7.3. THEOREM. (1) $MT_{IO}^* = MT_{OI}^* = (T \cup \text{YIELD})^* = (D_i T \cup \text{SET} \cup \text{YIELD})^* = (D_i MT \cup \text{SET})^*$
 (2) $D_i MT^* = (D_i T \cup \text{YIELD})^* = D_i T \circ \text{YIELD}^*$.

Proof. Immediate from the various decomposition results. Note that $\text{SET} \subseteq T$. The second equality in (2) is Corollary 4.13. ■

From this the following consequences are obtained.

7.4. THEOREM. Let F be the composition of a finite number of relations in $MT_{IO} \cup MT_{OI}$:

- (1) RECOG is closed under F^{-1} .
- (2) $\text{dom}(F) \in \text{RECOG}$.

Proof. (1) holds for $F \in T$ (Lemma 1.2 of [14]) and for $F \in \text{YIELD}$ (Lemma 6.1 of [20]), and hence for any composition of such relations. From this, (2) follows as a particular case. ■

Now we can show that the output languages of macro tree transducers are decidable.

7.5. THEOREM. The general membership problem for the output languages of macro tree transducers is decidable, i.e., there is an algorithm which decides for any $M \in MT_a$ and any tree t whether $t \in \text{out}(\tau_\mu(M))$ or not ($\mu \in \{\text{IO}, \text{OI}\}$).

Proof. Let $M \in MT_a$, $\mu = \text{OI}$, and t be a tree. Define $N \in T\text{-FTA}_a$ such that $\tau(N) = \{(t, t)\}$. Then, $t \in \text{out}(\tau_{\text{OI}}(M))$

$$\text{iff } \text{out}(\tau_{\text{OI}}(M)) \cap \{t\} \neq \emptyset$$

$$\text{iff } \text{out}(\tau_{\text{OI}}(M)) \cap \text{dom}(\tau(N)) \neq \emptyset$$

$$\text{iff } \text{out}(\tau_{\text{OI}}(M) \circ \tau(N)) \neq \emptyset$$

$$\text{iff } \text{dom}(\tau_{\text{OI}}(M) \circ \tau(N)) \neq \emptyset.$$

Since $\tau(N) \in MT_{\text{IO}} \cup MT_{\text{OI}}$, $\text{dom}(\tau_{\text{OI}}(M) \circ \tau(N)) \in \text{RECOG}$ (Theorem 7.4(2)), and since it is decidable whether $L \in \text{RECOG}$ is empty or not [38], it is decidable whether $t \in \text{out}(\tau_{\text{OI}}(M))$ or not. The same argumentation holds for $\mu = \text{IO}$. ■

Finally, we turn to composition results. In the next theorem we consider classes $K_1 \circ K_2$, where K_1 consists of deterministic transducers and K_1 or K_2 consists of top-down tree transducers.

7.6. THEOREM.

$$\begin{aligned}
 (1) \quad & D_t MT \circ D_t T = D_t MT & D_t T \circ D_t MT &= D_t MT \\
 & D_t MT \circ DT = DMT_{\text{OI}} & DT \circ D_t MT &= DMT_{\text{IO}} \\
 & D_t MT \circ ,T = ,MT_{\text{OI}} \\
 & D_t MT \circ T = MT_{\text{OI}} \\
 (2) \quad & DMT_{\text{IO}} \circ D_t T = DMT_{\text{IO}} \\
 & DMT_{\text{IO}} \circ DT = DMT_{\text{OI}} & DT \circ DMT_{\text{IO}} &= DMT_{\text{IO}} \\
 & DMT_{\text{IO}} \circ ,T = ,MT_{\text{OI}} \\
 & DMT_{\text{IO}} \circ T = MT_{\text{OI}} \\
 (3) \quad & DMT_{\text{OI}} \circ DT = DMT_{\text{OI}} & DT \circ DMT_{\text{OI}} &= DMT_{\text{OI}} \\
 & DMT_{\text{OI}} \circ ,T = ,MT_{\text{OI}} \\
 & DMT_{\text{OI}} \circ T = MT_{\text{OI}} \\
 (4) \quad & & DT \circ ,MT_{\text{OI}} &= ,MT_{\text{OI}} \\
 & & DT \circ MT_{\text{OI}} &= MT_{\text{OI}} \\
 (5) \quad & & D_t T \circ MT_{\text{IO}} &\neq MT_{\text{IO}}.
 \end{aligned}$$

Proof. (1) The left side is Theorem 4.12 together with Corollary 6.12, and the first equality at the right side is Corollary 4.10. The second equality at the right is shown as follows:

$$\begin{aligned}
 DMT_{\text{IO}} &= DT - \text{FTA} \circ D_t MT && \text{by Theorem 5.23,} \\
 &\subseteq DT \circ D_t MT \\
 &\subseteq DT - \text{FTA} \circ D_t T \circ D_t MT && \text{by Lemma 5.22,} \\
 &= DT - \text{FTA} \circ D_t MT && \text{by Corollary 4.10.}
 \end{aligned}$$

(2) Since $DMT_{IO} = DT - FTA \circ D_i MT$ (Theorem 5.23), we obtain the equalities at the left from those under (1) by left-composing with $DT - FTA$ and the use of regular look-ahead (Theorems 4.21 and 6.15, cf. Lemma 6.17). For the equality at the right, we first note that $D_i T \circ DT = DT$ (because $DT = D_i T \circ \text{EMPTY}$ by Corollary 6.11, and $D_i T$ is closed under composition [34]). Hence, by Theorem 5.24, $D_i T \circ DMT_{IO} = D_i T \circ DT \circ \text{YIELD} = DT \circ \text{YIELD} \circ DMT_{IO}$. We can now use the same reasoning as under (1) to show that $DT \circ DMT_{IO} = DMT_{IO}$.

(3) The equalities at the left follow easily from $DMT_{OI} = FTA \circ D_i MT$ (Theorem 6.18), the left-equalities under (1) and the use of regular look-ahead to get rid of FTA. To prove the equality at the right, note that, since $DMT_{OI} = D_i MT \circ \text{EMPTY}$ (Theorem 6.10), $D_i T \circ DMT_{OI} = D_i T \circ D_i MT \circ \text{EMPTY} = D_i MT \circ \text{EMPTY} = DMT_{OI}$ (using Corollary 4.10). Hence, $DT \circ DMT_{OI} \subseteq DT - FTA \circ D_i T \circ DMT_{OI} = DT - FTA \circ DMT_{OI} \subseteq DMT_{OI}$ (by Lemma 6.17).

(4) follows from (3): $MT_{OI} = DMT_{OI} \circ T = DT \circ DMT_{OI} \circ T = DT \circ MT_{OI}$, and similarly for ${}_i MT_{OI}$.

(5) From Example 5.10 we know that $MT_{IO} \not\subseteq T \circ \text{YIELD} = \text{LHOM} \circ MT_{IO}$ (Theorem 5.15). Note that $\text{LHOM} \subseteq D_i T$. ■

Considering these composition results one might say that OI-macro tree transducers behave very regularly (as opposed to IO): $DT \circ (D)MT_{OI} \subseteq (D)MT_{OI}$ and $DMT_{OI} \circ (D)T \subseteq (D)MT_{OI}$.

8. CONCLUSION

We have investigated the classes of translations realized by total deterministic, IO- and OI-macro tree transducers, i.e., $D_i MT$, MT_{IO} , and MT_{OI} , respectively, and have shown that each of them is decomposable into the simple translation classes $D_i T$, YIELD , and SET . Since it has become apparent that macro tree transductions are not closed under composition, we have studied the composition with top-down tree transducers. Considering the extension with regular look-ahead it has turned out that most of the translation classes (except DMT_{IO}) are closed under regular look-ahead. For further investigations many areas offer their services:

— To study the transformational power of macro tree transducers more compositions can be considered. Is, e.g., $D_i MT$ closed under right-composition with $D_i T^R$? Is MT_{OI} closed under right-composition with LT ?

— Consider restricted classes of macro tree transducers, e.g., static or dynamic restrictions on the nesting of states (cf. [39] for bounded nesting in macro grammars), linearity or bounded copying with respect to input (sub-)trees, copying power of macro tree transducers with a restricted set of states (cf. [21] for the copying power of 1-state top-down tree transducers), tree-to-string macro tree transducers, and monadic input trees (cf. [19] for ETOL-systems and top-down tree-to-string transducers).

— What is the complexity of deciding the membership problem and the general membership problem for the output tree languages of macro tree transducers (viewed as expressions over the alphabet with two additional symbols “(“and”)”? In [5] it was shown that the output languages of top-down tree transducers are context-sensitive.

— Compare the hierarchies T^n , YIELD^n , $D_i MT^n$, MT_{IO}^n , MT_{OI}^n , $(T \cup \text{YIELD})^n$, $(T \circ \text{YIELD})^n$, $(T \circ \text{yield})^n$ (see [9, 17]).

— The tree register pushdown transducer (TRPT [22]) provides an automaton model for MT_{IO} . Work out the suggestions done in [16] for an automaton modelling MT_{OI} : generalize the checking-tree pushdown machine (CT-PD, [19]) to a checking-tree nested-stack machine (CT NS) that has a nested-stack memory [1] available.

— The gsm-mappings are characterized in [25]. To obtain a better insight in the character of the mappings induced by macro tree transducers it would be worthwhile to have a similar result for MT_a (or even T_a).

REFERENCES

1. A. V. AHO, Nested stack automata; *J. Assoc. Comput. Mach.* **16** (1969), 383–406.
2. A. V. AHO AND J. D. ULLMAN, Translations on a context-free grammar, *Inform. and Control* **19** (1971), 439–475.
3. A. V. AHO AND J. D. ULLMAN, “The Theory of Parsing, Translation, and Compiling,” Vols. I, II, Prentice-Hall, Englewood Cliffs, N.J., 1973.
4. B. S. BAKER, Composition of top-down and bottom-up tree transductions, *Inform. and Control* **41** (1979), 186–213.
5. B. S. BAKER, Generalized syntax directed translation, tree transducers, and linear space; *SIAM J. Comput.* **7**, No. 3 (1978), 376–391.
6. L. M. CHIRICA AND D. F. MARTIN, An order-algebraic definition of Knuthian semantics, *Math. Systems Theory* **13** (1979), 1–27.
7. B. COURCELLE, Attribute grammars: Theory and applications, in “Proc. Colloq. on Formalization of Programming Concepts” (J. Dias and I. Ramos, Eds.), Lecture Notes in Computer Science Vol. 107, pp. 75–95, Springer-Verlag, Berlin, 1981.
8. B. COURCELLE AND P. FRANCHI-ZANNETTACCI, Attribute grammars and recursive program schemes I, II, *Theoret. Comput. Sci.* **17** (1982), 163–191; 235–257.
9. W. DAMM, The IO- and OI-hierarchies, *Theoret. Comput. Sci.* **20** (1982), 95–208.
10. W. DAMM AND J. ENGELFRIET, Higher level tree transducers, in preparation.
11. P. DOWNEY, “Formal languages and recursion schemes,” Report TR-16-74, Harvard University, 1974.
12. J. ENGELFRIET, Bottom-up and top-down tree transformations — A comparison, *Math. Systems Theory* **9** (1975), 198–231.
13. J. ENGELFRIET, “Tree Automata and Tree Grammars,” Datalogisk Afdeling Report DAIMI FN-10, Aarhus University, 1975.
14. J. ENGELFRIET, Top-down tree transducers with regular look-ahead, *Math. Systems Theory* **10** (1977), 289–303.
15. J. ENGELFRIET, Some open questions and recent results on tree transducers and tree languages, in “Formal Language Theory; Perspectives and Open Problems” (R. V. Book, Ed.), Academic Press, New York, 1980.

16. J. ENGELFRIET, "Tree Transducers and Syntax-Directed Semantics," TW Memorandum 363, Twente University of Technology, 1981.
17. J. ENGELFRIET, Three hierarchies of transducers, *Math. Systems Theory* **15** (1982), 95–125.
18. J. ENGELFRIET AND G. FILÈ, The formal power of one visit attribute grammars, *Acta Inform.* **16** (1981), 275–302.
19. J. ENGELFRIET, G. ROZENBERG, AND G. SLUTZKI, Tree transducers L -systems, and two-way machines, *J. Comput. System Sci.* **20** (1980), 150–202.
20. J. ENGELFRIET AND E. M. SCHMIDT, IO and OI, *J. Comput. System Sci.* **15** (1977), 328–353; **16** (1978) 67–99.
21. J. ENGELFRIET AND S. SKYUM, The copying power of one-state tree transducers, *J. Comput. System Sci.* **25** (1982), 418–435.
22. G. FILÈ, personal communication, 1982.
23. M. J. FISCHER, "Grammars with Macro like Productions," Ph. D. thesis, Harvard University, 1968; in "Proc. 9th Sympos. on SWAT," pp. 131–142, 1968.
24. P. FRANCHI-ZANNETTACCI, "Attributs semantiques et schemas de programmes," Thèse d'Etat, Université de Bordeaux I, 1982.
25. S. GINSBURG AND G. F. ROSE, A characterization of machine mappings, *Canad. J. Math.* **18** (1966), 381–388.
26. M. J. C. GORDON, "The Denotational Description of Programming Languages; An Introduction," Springer-Verlag, Berlin, 1979.
27. J. A. GOGUEN, J. W. THATCHER, E. G. WAGNER, AND J. B. WRIGHT, Initial algebra semantics and continuous algebras, *J. Assoc. Comput. Mach.* **24** (1977), 68–95.
28. E. T. IRONS, A syntax directed compiler for ALGOL 60, *Comm. ACM* **4** (1961), 51–55.
29. D. E. KNUTH, Semantics of context-free languages, *Math. Systems Theory* **2** (1968), 127–145; Correction, *Math. Systems Theory* **5** (1971), 95–96.
30. T. S. E. MAIBAUM, A generalized approach to formal languages, *J. Comput. System Sci.* **8** (1974), 409–439.
31. J. MEZEI AND J. B. WRIGHT, Algebraic automata and context-free sets, *Inform. and Control* **11** (1967), 3–29.
32. M. NIVAT, On the interpretation of recursive program schemes, *Sympos. Math.* **15** (1975), 255–281.
33. V. J. RAYWARD-SMITH, Hypergrammars: An extension to macro grammars, *J. Comput. System Sci.* **14** (1977), 130–149.
34. W. C. ROUNDS, Mappings and grammars on trees, *Math. Systems Theory* **4** (1970), 257–287.
35. W. C. ROUNDS, Tree-oriented proofs of some theorems on context-free and indexed languages, in "2nd Ann. ACM Sympos. on Theory of Computing," Northampton, Mass., 1970.
36. J. W. THATCHER, Generalized² sequential machine maps, *J. Comput. System Sci.* **4** (1970), 339–367.
37. J. W. THATCHER, Tree automata: An informal survey, in "Currents in the Theory of Computing" (A. V. Aho, Ed.), pp. 143–172, Prentice-Hall, Englewood Cliffs, N. J., 1973.
38. J. W. THATCHER AND J. B. WRIGHT, Generalized finite automata theory with an application to a decision problem of second-order logic, *Math. Systems Theory* **2** (1968), 57–81.
39. J. ENGELFRIET AND G. SLUTZKI, Bounded nesting in macro grammars, *Inform. and Control* **42** (1979), 157–193.